UNISON

RTOS for Microchip PIC32

version 5.2

Tutorial Guide

Unison Version 5.2
Tutorial Guide for Microchip PIC32

RoweBots Research Inc.
119 Church St.
Kitchener/Waterloo, Ontario N2G 2S3
Canada

Technical Support
Web: www.rowebots.com/support

This manual is, in its entirety, copyright © RoweBots Research Inc. 1994-2013 and protected by international copyright laws.

All rights are reserved. No part of this book may be reproduced in any form, except brief excerpts for the purpose of review, without the written permission of RoweBots Research Inc.

All trademarks are the property of their respective owners.

Chapter 1 Getting Started With Unison viii
Chapter 2 Building a Simple Application 4
The Operating System 5
A Simple Application 15
Chapter 3 Adding Unison 20
Unison I/O 21
POSIX and Unix I/O 26
Chapter 4 Memory Maps and Management 32
Overview 33
Memory Maps 34
Pools 36
Partitions 37
Chapter 5 Moving to the Target 38
Target Startup 39
Compiling and Downloading 42
Chapter 6 Synchronizing and Communicating With Other Threads 46
Communication Overview 47
Semaphores 49
Unison Rendezvous 64
Communicating With aChild Thread 66
Example of a Rendezvous 68
Synchronization Methods 71
Mutex 73
Chapter 7 Server Design and Interrupt Processing 76
Server Design 77
Chapter 8 Using Sockets 82
Ethernet Protocols 83
Setting Up Sockets 84
Creating a Socket 86
Using Stream Sockets 91
Chapter 9 Unison Servers 98
Using Unison Servers 99
File Servers 106

APPENDICES:

Appendix A **Chip (and Board) Support Packages (CSPs) 118**
Appendix B **Microchip Processors and Solutions 126**
Appendix C **Chip and Board Support for Microchip 132**
Appendix D **Device Specific Servers 134**
Appendix E **RemedyTools and Applications 140**
Appendix F **Error Messages 144**
Appendix G **libc 152**
Appendix H **Glossary 154**

TABLES:

Table t1 Unison Host Tools 2
Table t2 Unison Target Tools 2
Table t3 Illustrating Potential Server Options 6
Table t3 (continued) Illustrating Potential Server Options 7
Table t4 Nanoexec Initialization 14
Table t5 System Calls Used in the Simple Application (Sample No. 1) 16
Table t6 System Calls Used in Creating a Unison Application (Sample No. 2 – mrendez) 23
Table t7 System Calls Used in the Unison/Nanoexec Application (Sample No. 3 - tty) 27
Table t8 Directory System Calls 30
Table t9 Stack Filling Parameters and Functions 35
Table t10 New System Calls Used Creating a Socket on the Server Side (Sample No. 7) 86
Table t11 New System Calls Used Creating a Socket on the Client Side (Sample No. 5) 87
Table t12 New System Calls Used with Stream Socket on the Server Side (Sample No. 6) 92
Table t13 New System Calls Used for Stream Socket on the Client Side (Sample No. 8) 96
Table t14 Servers Requiring Specific Names 101
Table t15 Unison Error Codes 145
Table t15(continued) Unison Error Codes 146
Table t15(continued) Unison Error Codes 147
Table t15(continued) Unison Error Codes 148
Table t15(continued) Unison Error Codes 149
Table t15(continued) Unison Error Codes 150
Table t16 Unison Fatal Error Codes 151

FIGURES:

Figure f1 Different Layers of the Operating System 5
Figure f2 Thread Diagram for a Simple Application 16
Figure f3 Interaction Between a Client and Server 21
Figure f4 Standard Unison Message 22
Figure f5 Initialization Sequence For a Unison/ Nanoexec Application In Setup.c 40
Figure f6 Thread Interaction using Two Threads and Two Queues .53
Figure f7 Thread States When Using the Rendezvous 64
Figure f8 Server Structures 77
Figure f9 Setting up UDP Sockets 84
Figure f10 Steps Required to Communicate Via TCP/IP Protocol . .92
Figure f11 Buffers Used by the Unison I/O Package 100
Figure f12 Sequence of Events That Occur Upon Execution of an Open Command 101
Figure f13 Sequence of Events When Using NFS Server 102
Figure f14a Sequence of Events When Using the NFS Server 105
Figure f14b Sequence of Events When Using the File Server 106
Figure f15 How Unison Determines a Device Number 111
Figure f16 How Unison's File Server Uses the Disk 115
Figure f17 Remedy RED Hardware Architecture 143
Figure f18 Remedy RED Software Architecture 143

EXAMPLES:

Example e1 Setup.c Example for ARM Cortex-M 7
Example e1(cont'd) Setup.c Example for ARM Cortex-M 8
Example e1(cont'd) Setup.c Example for ARM Cortex-M 9
Example e1(cont'd) Setup.c Example for ARM Cortex-M 10
Example e1(cont'd) Setup.c Example for ARM Cortex-M 11
Example e2 Incorporating Region Zero Inside the BSS Section 13
Example e3 A Simple Application (Sample No. 1 - sema) 16
Example e3 (cont'd) A Simple Application (Sample No. 1 - sema) 17
Example e3 (cont'd) A Simple Application (Sample No. 1 - sema) 18
Example e4 main() for a Simple Application (Sample No. 1) 19
Example e5 Creating a Unison Application (Sample No.2 - mrendez) 23
Example e5 (cont'd) Creating a Unison Application (Sample No.2 - mrendez) 24
Example e5 (cont'd) Creating a Unison Application (Sample No.2 - mrendez) 25
Example e6 Opening a Channel for Standard I/O 27
Example e7 A Unison/Nanoexec Application (Sample No. 3 - tty) . . .27
Example e7 (cont'd) A Unison/Nanoexec Application (Sample No. 3 - tty) . . .28
Example e7 (cont'd) A Unison/Nanoexec Application (Sample No. 3 - tty) . . .29
Example e8 Message Queue Test 60
Example e8 (cont'd) Message Queue Test 61
Example e8 (cont'd) Message Queue Test 62
Example e8 (cont'd) Message Queue Test 63
Example e9 Using Primitives 66
Example e10 Sending a Message to a Child thread 66
Example e11 A Child thread Receiving a Message 67
Example e12 Unison Server 79
Example e12(cont'd) Unison Server 80
Example e13 Creating a Socket on the Server Side (Sample No. 7) . . .86
Example e13(cont'd) Creating a Socket on the Server Side (Sample No. 7) . . .87
Example e14 Creating a Socket on the Client Side 87
Example e14(cont'd) Creating a Socket on the Client Side (Sample No. 5) . . .88
Example e14(cont'd) Creating a Socket on the Client Side (Sample No. 5) . . .89
Example e14(cont'd) Creating a Socket on the Client Side (Sample No. 5) . . .90
Example e14(cont'd) Creating a Socket on the Client Side (Sample No. 5) . . .91
Example e15 Stream Socket on the Server Side (Sample No. 6) 93
Example e15(cont'd) Stream Socket on the Server Side (Sample No. 6) 94
Example e15(cont'd) Stream Socket on the Server Side (Sample No. 6) 95

Example e16 Stream Socket on the Client Side (Sample No. 8) 96
Example e16(cont'd) Stream Socket on the Client Side (Sample No. 8) 97
Example e17 Creating a NFS Server 103
Example e17(cont'd) Creating a NFS Server 104
Example e18 Mounting a Directory on the Host 105
Example e19 Creating a File Server 109
Example e20 A Typical Device Table 110
Example e21 A Typical Minor Device Table Definition 111
Example e22 Creating a RAM Disk 112
Example e23 Building a Device Table 113
Example e24 Creating a File System on RAM Disk 114
Example e25 Allocating a Contiguous Region to a File 116

Chapter 1
Getting Started With Unison

This document is designed to guide users through the different areas of Unison, while providing examples along the way. The overall install directory is found at

```
$RRI_DIR/Unison
```

where $RRI is a placeholder for the path to the release directory Unison.

The examples in this manual are provided as a part of the standard installation and can be found in the standard release in the following directory:

```
$RRI_DIR/Unison/demo
```

Unison is a real-time operating system with a development environment or an integrated development environment (IDE); it has two distinct sections:

- host side
- target side

The host based software contains source code, libraries, and tools that assist in assembling, compiling, downloading, and debugging an application. The target side software contains a number of modules that provide a wide variety of "off the shelf services".

Unison is intended to look like Linux from an applications programming interface point of view. It provides a single process multiple thread model with or without I/O satisfying the real time POSIX specifications and Linux compatibility. Unison appears to the programmer as an ultra tiny embedded Linux compatible solution.

DSPnano is offered on a collection of 16 bit processors. It is identical to the Unison operating system and, with minor exceptions, all documentation and applications programming interfaces are identical.

Unison is a real-time operating system designed for system on a chip (SOC) processors. It works with various integrated development environments (IDE), generally from the semiconductor manufacturer of the chip.

Unison Host Tools

Table **t1** (see pg.2) shows the host based tools that are discussed in this manual.

Table t1
Unison Host Tools

Host Tools	Description
Eclipse IDE	a front end to the compilers, linkers, debuggers and help system including cheat sheets and books
gcc	a compiler toolchain for Linux, Solaris and other hosts with cross development support for most hosts
MinGW	a compiler toolchain for Windows based on gcc
gdb	the real-time and host based debugger
MPLAB	a debug, compile and build environment for the PIC24, dsPIC, and PIC32 families
HEW	a debug, compile and build environment for the H8, M16C, R32C, and SH2A families
hardware	real-time and host based hardware debugger support, processor specific

Unison Target Components

On the target, the Unison operating system consists of the following major components:

Table t2
Unison Target Tools

Target Tools	Description
Nanoexec	a real-time multi-threaded POSIX operating system (kernel)
Unison Servers	libraries and extensions for POSIX I/O services
DSP libraries	generic and application specific libraries*

The host tools operate in a generic fashion and allow for mixing and matching of processor types and architectures.

** Contact the factory for details*

Chapter 2
Building a Simple Application

The Operating System

In order to build the simplest of applications, the user must become familiar with the Nanoexec kernel. The Nanoexec is the lowest level of Unison and is the foundation on which all target software rests. It is designed to run in a consistent fashion on all supported hardware. The kernel will run:

- either by itself, or,
- with many kernels connected via thin wires (to produce a distributed-processing system) or shared memory.

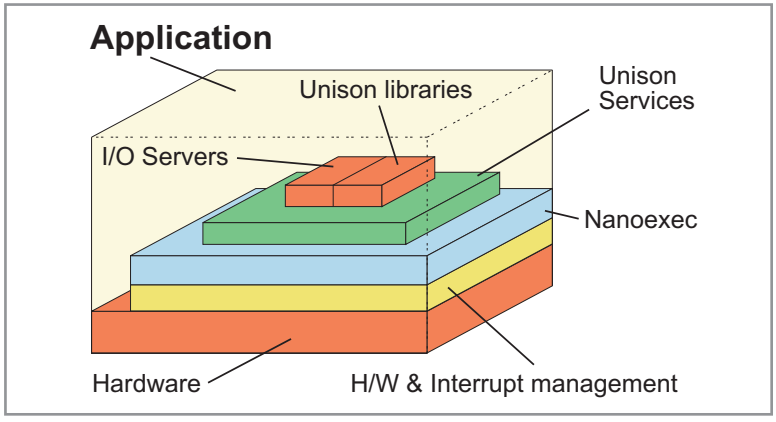
Each processor in a system will run its own copy of the kernel.

** For more information on Nanoexec features, refer to the I/O and library examples in the "Unison Programmer's Guide" or other general POSIX references.*

The kernel is responsible for maintaining queues, semaphores, mutexes, local thread operations, and memory management. It was designed to work independently of the hardware architecture underneath it. Unison is designed to enhance the Nanoexec and provide features not present inside the kernel.*

The features provided by Unison RTOS include synchronization and communication operations, interrupt management, directory services, transparent I/O services, processor, chip and board support packages, and remote debugging. A simplified system architecture of Unison is shown in the following figure:

Figure f1
Different Layers of the Operating System



In this section, we will be building a simple Nanoexec application. To remove all complications concerned with the hardware we will be using the processor simulator that runs on your workstation. First we will discuss the general building or linking of a Unison application. This will include some discussion of the various services available.

After this we will begin by describing the functions that are required to configure the Nanoexec and its setup file. The kernel will not automatically allocate all of its resources upon initialization as a DSP kernel should. The system should be initialized with the queues, threads, semaphores, etc., so that the kernel may allocate all the memory that

those resources require from the system's memory pool at start up. There is nothing to force the user to do so – it is simply POSIX style. It is up to the user to ensure static resource management, a good practice for DSP systems.

The system's memory pool is called Region Zero. The Nanoexec will allocate **all** of the memory it requires from Region Zero before the first user thread is created. After the kernel has initialized, the balance of this region may be used by the application. Note that in some of our target examples we will use a simpler scheme than a full fledged memory manager to save space and minimize cost and complexity. This will also demonstrate the great flexibility of the Unison architecture as many components and functions can simply be replaced by the user (e.g. idle thread, kfatal, …).

Linking Services

When linking an application, you must know:

- the processor's exact configuration
- a list of all servers that will be used on the target processor

The user simply has to include the correct libraries and services at link time to use these Unison components. Note that as the user you may have to adjust the linking order to ensure that libraries are linked correctly.

The following servers are part of Unison but all might not be available for your released processor and/or board. Please consult the detailed processor, chip and board support documentation to understand which servers are available.

Table t3
Illustrating Potential Server Options

Options	Library
Athena toolkit	athena
Backplane Driver	bptcpd
CAN	can
Ethernet Server (TCP/IP)	tcpd
Fat File System	fatfs
File System Server	fsys
LIT	lit
Network File Server (NFS)*	nfs
POSIX Shell	posh
Performance Meter	perf

** Registration for nfs or NFS must use this name.*
Other servers can be renamed.

Table t3 (continued)
Illustrating Potential Server Options

Options	Library
Remote Procedure Call (RPC)	rpc
Serial Server (TTY)	tty
telnetd	telnetd
tftp Daemon	tftpd
USB embedded host	usb_host
USB client host interface device	usb_client_hid
USB client mass storage	usb_client_ms
web server	httpd
X11 Client Services	X11
X11 Server	x11server
Other Options	**Library**
Cause the compiler to generate relocatable code	compiler specific
Remote Reset	rreset
Turn Compiler Optimization On	compiler specific

Example e1
Setup.c Example for ARM Cortex-M

```
/*****************************************************/
              /* Firmware Includes */

/*****************************************************/
#include <CMSIS/a2fxxxm3.h>
#include <drivers/mss_gpio/mss_gpio.h>
#include <drivers/mss_watchdog/mss_watchdog.h>
#include <drivers/mss_uart/mss_uart.h>
#include <drivers/mss_ace/mss_ace.h>




#define DEBUG_TO_BUF   0
  // 1 - xprintf will printout debug information to
  //     debug memory buffer ÔøΩdebug_bufferÔøΩ
  // 0 - xprintf will printout debug information to
  //     serial port (UART). Do not use this options, when
using TTY server in your demo!
#define DEBUG_BUFSIZE  128
  // Size for "debug_buffer" in bytes
```

Example e1(cont'd)
Setup.c Example for ARM Cortex-M

```
#define CLOCK_TIME_PERIOD      1000000
        // nanoseconds per tick - period 1ms
#define SYS_TIMER_PERIOD       75000
        // SystemCoreClock 75MHz * 0.001s - ticks per 1ms

int main()
{
  /* Disable all interrupts */
  __disable_irq();

  /* 1 - fill task stack by 'S' during initialization for
debug purpose */
  pthreadStackFill = 1;

  /* 1 - fill isr stack by 'I' during initialization for debug
purpose */
  _isrStackFill = 1;

  /* initializes stack for interrupt, sizes = MAIN_STACK_SIZE,
see .ld file for detail */
  _isr_init();

  /* heap for kernel malloc initialization */
  malloc_init();

  /* all hardware initialization */
  hw_init();

  /* Start RTOS */
  Nano_Start();

  return 0;
}
```

Example e1(cont'd)
Setup.c Example for ARM Cortex-M

```
//*******************************************************
// This is the handler for this SysTick interrupt.
//*******************************************************
int timer_cnt = 0;
        void SysTickHandler(uint_32 arg)
        { timer_tick(CLOCK_REALTIME);
           if(++timer_cnt > (1000000000/CLOCK_TIME_PERIOD/4))
              {       timer_cnt = 0;
                      LEDToggle(0);
              }
        }


static void hw_processor_init_cortexm3()
{
  int iseterror;
  clockid_t  my_timer_clockid;
  struct timespec my_timespec;

        // processor specific initialization with clock
        // first setup the interrupt vector, clock parameters
and then setup                    the timer hardware
  my_timespec.tv_sec = 0;
  my_timespec.tv_nsec = CLOCK_TIME_PERIOD;
  my_timer_clockid = CLOCK_REALTIME;
  iseterror = clock_init(my_timer_clockid, &my_timespec);

  if(iseterror != 0)
  {
        xprintf("clock_init error\n");
  }
}


// this routine initializes the three aspects of the hardware
// and enables interrupts.  The next immediate thing must be
// to start the kernel so it can initialize before the first
// timer interrupt occurs.

void hw_init(void)
{
  // Disable the Watch Dog Timer
  MSS_WD_disable( );
```

Example e1(cont'd)
Setup.c Example for ARM Cortex-M

```
  // Init GPIO for LEDs
  Led_Init();

  // Configure main (SysTick) timer.
  i_set_interrupt((uint_32)SysTick_IRQn, SysTickHandler, 0);
  NVIC_SetPriority(PendSV_IRQn, 0xF);
  SysTick_Config(SYS_TIMER_PERIOD);
  //by default, SysTick_Config will setup Priority = 0xF,
  //but we need Priority val < then PendSV_IRQn, so we
will set one more time.
  NVIC_SetPriority(SysTick_IRQn, 0xE);
#if DEBUG_TO_BUF == 0
        UART0_Init();
#endif

        hw_processor_init_cortexm3();
}


/*******************************************************
Function Name: DEBUG_UARTInit
Description:   Setup UARTx module.
Parameters:    none
Return value:  none

*******************************************************/
#if DEBUG_TO_BUF == 0
void UART0_Init()
{
  /* Initialize and configure UART0.*/
  MSS_UART_init
  (     &g_mss_uart0,
        MSS_UART_115200_BAUD,
        MSS_UART_DATA_8_BITS | MSS_UART_NO_PARITY |
MSS_UART_ONE_STOP_BIT
  );

#else
  uchar debug_buffer[DEBUG_BUFSIZE];
  static int debug_index=0;
#endif
```

Example e1(cont'd)
Setup.c Example for ARM Cortex-M

```
void xputchar (const int c)
{
  #if DEBUG_TO_BUF == 0
  uint_32 status;
  /* Wait for UART to become ready to transmit. */
  do    {
        status = g_mss_uart0.hw_reg_bit->LSR_THRE;
        } while ( (status & 0x1) == 0 );
        /* Send next character in the buffer. */
        g_mss_uart0.hw_reg->THR = c;



#else
        debug_buffer[debug_index]=(uchar)c;
        if (++debug_index >= DEBUG_BUFSIZE) debug_index = 0;
#endif
}
```

When the kernel starts up, it is provided files called *setup.c* and *malloc.c* which then provide all the information to initialize the hidden elements of the executive startup. *Setup.c* includes the following elements to startup the system:

- The file *malloc.c* provides Region Zero, malloc and kalloc. It configures the memory management for the system.
- timer information and setup
- main function
- kfatal

Sometimes the interrupt stack size is also changed to tune memory performance. It is found in *interrupt.c*. The root thread is specified with the function itself.

Interrupt Stack Size

All the rest of the configuration information is contained in the actual code consistent with a complete POSIX implementation style.*

** See #define SYSSTK and char_isr_stack[SYSSTK]*

The first important parameter which is required is the Interrupt Stack Size. On most systems this will be used as the overall interrupt stack; however, on some systems there may not be a separate interrupt stack and individual threads will have to have enough stack space to support all nested interrupts.

The interrupt stack size is found in the file *interrupt.c* and is part of the kernel port. It is generally only modified as a system is fine tuned.

SYSSTK specifies the size of the kernel stack or the size of the interrupt stack. The size of this stack is proportional to the number of nested interrupts that your application has. The more nested interrupts that occur, the more times the kernel will be re-entered and the larger this value should be.

The interrupt stack is a significant benefit generally. It means that only one stack is required to handle interrupts – not every thread. It may be slightly slower but offers significant space reduction if there are many levels of interrupt or larger interrupt handlers.

Note that not all processor configurations support an interrupt stack and/or nested interrupts. In some cases, an interrupt stack is too expensive and the same may be true for nested interrupts. Generally interrupt processing is done quickly to avoid missed interrupts and interrupt service routines should be minimal for these reasons.

Region Zero (System Memory)

Region Zero is the system memory that the operating system will generally create all objects from for the kernel. Actually, kalloc allocates the kernel's memory and some systems use a more sophisticated allocation scheme. See the "Unison Programmers Guide" for details. It is possible to have memory management for other pools of memory (variable size allocation) and for memory partitions (fixed size buffer management). This region (as previously mentioned) is the location of the kernel's memory. It must be large enough for the kernel to allocate all of its resources. In our example Setup.c, the malloc_init function performs this initialization operation.

*** BSS is explained in the Memory Maps and Management chapter in this guide and is also explained in the Unison Programmer's Guide.*

There are two ways to locate the kernel's memory. The first method is to devise the system's memory map and then to place the text, data, BSS, and Region Zero within those regions of memory. With this method it can be difficult to ensure that Region Zero does not overlap one of these areas. If a user were to map out a section of memory for Region Zero, then that user would constantly have to check the link map to ensure that the boundaries of the compiler/linker do not overlap this area.**

A simpler method is to simply incorporate Region Zero inside of the BSS section. This is done by declaring a large array and then placing a pointer to this array in the startup table. In some environments this may be more difficult and special linker scripts are required. By doing this, the linker will resolve where to place Region Zero. The code to do this would be:

Example e2
Incorporating Region Zero Inside the BSS Section

```
#define RN0LEN (1024*20)       /*define a 20K Region Zero */
char rn0[RNOLEN];              /* define the memory for Region
                                Zero using an array */
/* these values must then be used to initialize the memory for the
system during startup or the first time that it is called. */

/* and initialize values using pool calls */
#include <sys.h>

#define MEMSIZE_M3 2000
static unsigned int _Heap_start[MEMSIZE_M3];

void malloc_init()
{
  /* can only fail if pool already created */
  pool_create(0, (unsigned long)&_Heap_start, (unsigned
long)&_Heap_start[MEMSIZE_M3-1]);
}

void *malloc(size_t size)
{
  return pool_alloc(0, size);
}

void free(void *ptr)
{
  pool_free(0, ptr);
}
```

Nanoexec Objects

The routines that are used by the kernel to allocate storage and return it are: *kalloc* and *kfree*. Often these are simply calls for *malloc* and *free* if there is a single memory pool.

Nanoexec main() function

In many real-time executives, particularly those that do DSP applications, the rest of the objects of the kernel are statically defined in the startup table. In the case of POSIX, this is done in the actual code using dynamic memory allocation. It is up to the user to ensure that all the static allocation is done before the main processing is started to manage predictability of the time responses.

The main function has the responsibility to configure the system for startup, define the first user thread and then start the system. It never returns after calling *Nano_Start* and it is only called once at startup.

Nanoexec Timing and Timer

All Nanoexec structures and options are defined in the include file:

$RRI_DIR/Unison/include/sys.h

Table t4
Nanoexec Initialization

Step	Description	Function or Variable
1	Define first user thread by providing table parameters.	ConfigTableStruct ConfigTable = { first_thread, root_stack_size, root_priority};
2	Make sure kfatal is specified to handle catastrophic errors. This is a user provided routine in target systems.	Void *kfatal()*;
3	Initialize the memory management using Region Zero information. These routines can be replaced by the user or simpler routines can be used. Note that the kernel uses kalloc and kfree.	You can use pools to provide memory management and regions or you can use your own kalloc routines.
4	Initialize the hardware remembering that each server thread will initialize its own hardware. This function must be provided by the user and should start the timer last after the interrupt vectors are set up. The timer defines for the processor, part and board are also used to configure the timer.	Void *hw_init(void)*;
5	And then start the first user thread, an operation which never returns	Void *Nano_Start()*;

Although the kernel and all of Unison (except for the TCP/IP server) is capable of running without a clock, most real-time applications require the use of one for accurate timing, etc. Setting up the hardware is the responsibility of the user. If the user wants the kernel to have a clock then programming of a suitable hardware device is required.

Each time a clock interrupt occurs, the kernel must be notified. Since the kernel did not set up the clock, it has no way of knowing its accuracy. The user must call a POSIX routine to setup the clock and also configure and start the clock within the hw_init routine.

```
#include <sys.h>
int clock_init(clockid_t clockid, const struct timespec *resolution);
```

The tick information is passed to the kernel using the interrupt structure of the processor, ensuring that it gets setup within hw_init knowing timer_tick.

```
#include <sys.h>
   //for some processors
   //use i_set_interrupt
void _Nanoexec_ISR timer_isr(void)
{
    timer_tick();    /* clockid must be of Clock_Real_Time type */
}
```

The timer initialization is not enough by itself and also needs the specific clock setup for timers used on the application level.

The POSIX call is used to inform the kernel of the number of clock interrupts that will occur in one second. Remember, each time the kernel is notified of a clock tick by calling timer_tick it will have to adjust its internal timers and the ready-to-run queues.

The greater the number of clock ticks the more accurate the clock, but also the greater amount of time is spent inside the kernel deciding which thread should run next. The accuracy of the clock is always a series of tradeoffs that are highly dependent upon your application.

The kernel is not capable of performing time-slicing. Time-slicing is generally not suitable for real-time applications. If users need a feature like this for low priority threads they can use sched_yield to accomplish the same thing. With sched_yield, if no threads of equal priority are ready-to-run, the current thread will continue to run.

Nanoexec Root Thread and Nanoexec ConfigTableStruct

The ConfigTableStruct elements ROOTADDR, ROOTSTKSIZE, and ROOTPRIO are used to describe the first thread that will be started. In Unison systems, these fields are supplied by the Nanoexec main initialization and will be used in any host simulation environment. Unison on the target system will not be used in this example, but we will require the use of these fields to create our first thread.

Kfatal

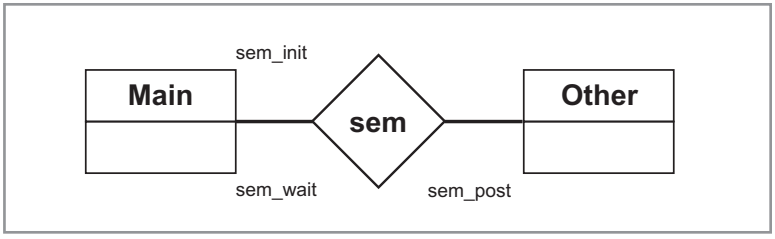
KFATAL is a user supplied function which is to be run when the kernel detects a fatal error. Such an error is serious enough that the kernel is unable to continue. During development this should be handled and it is highly recommended that it is also handled in target systems which are deployed.

A Simple Application

We will now build a simple application (Sample No. 1 - sema). It can be found in $RRI_DIR/Unison/demo/simple_sema. This example will have two threads and one semaphore. The first thread will be created by the kernel, after it has initialized. The ROOT thread will then create the balance of the Nanoexec objects. After all objects are created, the root thread will wait on the semaphore relinquishing the processor. The second thread will then run and post to the semaphore and then clean up by destroying itself. The root thread will be released by the sem_post of the second thread and it will run again. It will exit. Each Nanoexec thread must destroy itself. (If the thread returns from its root function and does not destroy itself, then the kernel will consider that to be a fatal error and the entire system will be aborted.).

The system architecture is shown in the following figure:

Figure f2
Thread Diagram for a Simple Application



Application Code

Each Nanoexec object is given an ID when the object is created. Once the object is created with this ID, then any thread may interact with the object if it has the object id. The object ids can be shared in many ways. For simple applications, global variables may be used. For larger applications the directory provides a good way to link names to Ids allowing users to get Ids by name. Global variables will be used here and in many other examples for simplicity.

The system calls for the simple application are:

Table t5
System Calls Used in the Simple Application (Sample No. 1)

Name	Description
pthread_create()	create a Nanoexec thread
xprintf(), xputs()	create generic polled formatted output
kfatal()	terminate processing
sema_init()	initialize semaphore
sema_wait()	wait for semaphore
sema_post()	post to a semaphore
pthread_exit	delete myself
dir_register	register thread id

The application code is:

Example e3
A Simple Application (Sample No. 1 - sema)

```
#include <sys.h>
#include <semaphore.h>

int errno;
#define SYSSTK  200

THREAD Main(), OtherThread();

ConfigTableStruct ConfigTable = {Main, 0, 8};   /* setup the
root thread with default stack size and prioirity 8 */

void main()
{

#if VERBOSE
pthreadStackFill = 1;
#endif
```

Example e3 (cont'd)
A Simple Application
(Sample No. 1 - sema)

```
/* malloc_init() might generally be seen here but we are using
a minimal kalloc memory manager given below */

void hw_init(void(*timer)());  /* hardware initialization for
kernel and clocks */

/* malloc_init()  is not done here because kalloc and kfree
are defined below in minimal fashion */

Nano_Start();
}

sem_t sema;           /* define a semaphore structure

this is the root thread that will create all other objects and
allocate all space for performance reasons */
THREAD Main(){
pthread_t tid;

xputs("Main thread here!\n");
#ifdef DEBUG
                dir_register("Main", pthread_self(),
TYPE_THREAD);
#endif

sem_init(&sema, 0, 0);   // initialization of semaphore
#ifdef DEBUG
                dir_register("sema", &sema, TYPE_SEMAPHORE);
#endif

/* create the "other" thread */
if(pthread_create(&tid, 0, OtherThread, 0))
        /* error */
       xputs("pthread_create OtherThread failed.\n");
else
{
       xputs("Created "); xphex(tid); xputchar('\n');
}
/* let the other thread run */
xputs("Main: sem_wait...\n");
if(sem_wait(&sema) == -1)
      xputs("sem_wait failed\n");
xputs("Main back.\n");
pthread_exit(0);   /* threads must clean up after themselves –
nobody else can do it */
}

THREAD OtherThread()
{
  xputs("OtherThread here!\n");

  xputs("OtherThread: sem_post...\n");
  if(sem_post(&sema) == -1)
        xputs("sem_post failed\n");
```

Example e3 (cont'd)
A Simple Application
(Sample No. 1 - sema)

```
  xputs("OtherThread dies!\n");
  pthread_exit(0);
}
kfatal(unsigned long error)
{
  Status ps;
  ps = i_disable();
  xprintf("FATAL %ld\n", error);
  while(1) ;
}
#endif

/* doing a minimal implementation, in hope of setting a heap
which works in minimal space check out the KALLOC and MEMSET
code */
#define KALLOC1       /* ultra simple alloc - removes need for
malloc */
#define MEMSET1       /* removes need for TI libc */

/* This is the complete definition for one system.
 * It doesn't get any better than this. */

#if KALLOC
  /* You could play with this size to minimize it by running
once
   * and checking heapp.
   */
#  define MEMSIZE             1000
unsigned heap[MEMSIZE];
unsigned heapp;

void *kalloc(int type, int size)
{
if(heapp + size < MEMSIZE)
  {
  void *ptr = &heap[heapp];
  heapp += size;
  return ptr;
  }
return 0;
}
  /* never called */
  void kfree(int type, void *ptr) {}
  #endif
```

Kernel Startup

As with all C programs, there must be a function called *main()*. In this system, *main()* is responsible for starting the kernel. In order to do this the *hw_init()* and *Nano_Start()* calls must be made. These should be setup as follows:

Example e4
main() for a Simple Application (Sample No. 1)

```
/*————————————————————-*/
/* main startup                                        */
/*————————————————————-*/
int main(int argc, char ** argv)
{
_isr_init() ;
malloc_init() ;
hw_init();   //enables interrupts last
Nano_Start();       /* DOES NOT RETURN */
}
```

Compilation

The kernel configuration and the application are now ready to compile. Simply click on the menu item and select simulator as the build type. Once this is specificed correctly a project build all command will generate the simulated system. Note that the more complex aspects of linking are not used during this phase.

Running

You have now created your first Nanoexec application. The program will terminate after it prints out information about the sequence of execution so you can be sure that the threads all ran in the correct order and the semaphore used for synchronization worked as well. After all threads have completed their execution, the Nanoexec will simply run the idle thread. To stop the execution of the kernel, you must typically type a control c (^c) or use the halt feature in the debugger if you were using that to run the example.

This example application uses the Nanoexec *xprintf()* I/O call. It is a part of every *Nanoexec port* and provides simple I/O capabilities that do not rely on threads. It will work even before the system has completely started. This is done in a chip support package (CSP) specific fashion. Thus those chips with no local I/O capabilities will send output directly to a memory buffer while others with serial chips will use their polled I/O mode.

This example running under the simulator, like any other POSIX application, may be debugged using the host debugger, gdb. It may use the Unix or POSIX file system, sockets, etc. Any facility that is available to a Unix or POSIX application is also available to this application provided an appropriate server has been ported and supported on the simulator. It is recommended that the application be simulated this way using all possible I/O to ensure the proper operation of the logic before extra target complexity creeps in.

In the next section we will remove the *xprintf* I/O facility and replace it with the Unison I/O facility. Unison provides the stdio for all target applications. If the final application does not require any I/O then Unison may be omitted from the target (but Nanoexec will be required).

Chapter 3
Adding Unison

Unison I/O

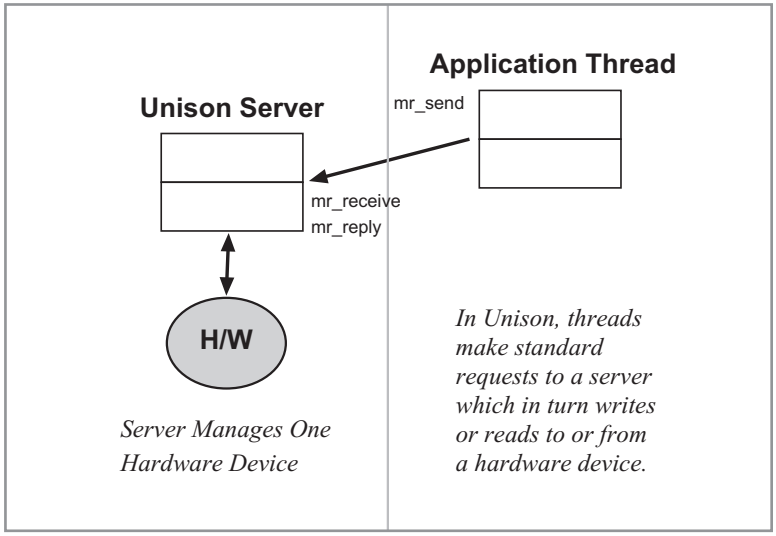
Unison is a collection of libraries that sit on top of Nanoexec. It provides features and functionality that are not present in the Nanoexec. This includes all the I/O facilities, Board Support Packages, Chip Support Packages, and Application libraries for DSP and standard I/O. The Nanoexec operates independently of Unison but the reverse is not true. Unison must have the Nanoexec present for managing queues, semaphores, threads etc. As discussed in the previous section, these services are simply linked as required.

Unison Servers

A Unison application is built with a series of modules or servers. Each provides **one** service and is optionally linked into an application. **Any thread** may use the services provided by a single Unison server. This reduces the overall resource requirements for an application. Resources now become "system" resources.

Unison servers are threads that understand client requests. Each client request is packaged into a message and then sent to the server in a standardized format. The server, being a Nanoexec thread, will process the message once it becomes the next ready to run thread. Servers are the same as any other Nanoexec thread. They have no special privileges within the kernel except that they are typically given the highest priority. Users can create their own servers by conforming to the Unison server protocol.

Figure f3
Interaction Between a Client and Server



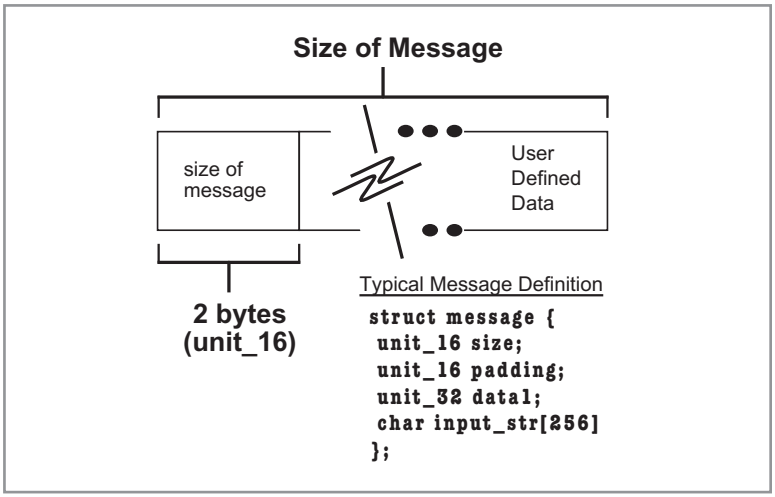
Unison Messaging

Both reads and writes by a client thread result in a synchronous message transfer to a Unison server. This happens if the client is performing a read or a write. The client always calls the server; the server never calls the client. Data is transferred from the server's memory into the client's

memory using the Unison standard synchronous rendezvous. The *synchronous message transfer* is the basis for all Unison server-client communication. The synchronous message is variable size up to a maximum of 64 Kbytes.

Variable size messaging allows each message to be a data structure. The programmer does not have to adjust data structures to the size of a single message. Instead, the operating system adjusts its message to the size of the data structure. The only restriction that is placed on the programmer is that the first two bytes, or unit_16, indicate the size of the message. This is shown in the following diagram.

Figure f4
Standard Unison Message



All Unison threads are capable of having several rendezvous ports. When a thread sends a message to another thread's port using a *mr_send()*, the sender queues up and waits for the receiving thread to receive the message. The receiving thread receives this message by using a *mr_receive()*. Once received, the receiver may then process the message and generate a reply using the *mr_reply()* system call. The sender is blocked from execution until the receiver performs this *mr_reply()* system call. This sequence is referred to as a send/receive/reply.*

** For a rendezvous to happen it is essential that the receiver has initialized using mr_init() prior to any send to it.*

Each thread is capable of waiting for a message from one of its 32 different ports simultaneously. Each port may be given a meaning (for example, all reads use port 1 and all writes use port 2). Unison then allows each thread to selectively choose which of the thread's 32 ports it will receive data from.

In Unison, threads are Nanoexec threads. Because of this, Unison threads and Nanoexec threads can coexist in the same system as they are identical. However, in order to manage threads which use Unison

services, the system requires additional information (for example, system resources and storage) required to do an *mr_receive()*. This is allocated by *mr_init()*. In order for a thread to use Unison services via the *mr_receive* call, it must be properly setup with this additional information using *mr_init()* (before sending any thread to it).

Table t6
System Calls Used in Creating a Unison Application (Sample No. 2 – mrendez)

Name	Description
mr_init()	allocates resources to support server access, required for *mr_receive* only
mr_send	send message to receiver on a port
mr_receive	receive messages and signals selectively
mr_reply	replies to the sender so that the sender may continue

In our example (Sample No. 2) we will only require one port and we will use *mr_init()*. This will cause Unison to attach the Unison data structure to the receiving thread.

Import $RRI_DIR/Unison/demo/rendez_time

Example e5
Creating a Unison Application (Sample No. 2 - mrendez)

```
#include <sys.h>
#include <mrendez.h>
#define NLOOP                 4

typedef struct {
              uint_16 size;
              uint_16 pad;
              } Msg;

THREAD Sender(), Receiver(), Dummy();
ConfigTableStruct ConfigTable = {Sender, 600, 4};

THREAD Sender()
{
  pthread_t dumb;
  pthread_t child;
  pthread_attr_t attr;
  struct sched_param myNewPriority;

  int createError;
  Msg rqst;
  int i;
  int port;
  unsigned start;

  xprintf("RENDEZVOUS TIMING TEST\n\n");

#ifdef DEBUG
              dir_register("Sender”, pthread_self(),
TYPE_THREAD);
#endif
```

Example e5 (cont'd)
Creating a Unison Application (Sample No. 2 - mrendez)

```
  pthread_attr_init(&attr);
  myNewPriority.sched_priority = 2;
  pthread_attr_setschedparam(&attr, &myNewPriority);

  /* create 4 dummy tasks with the highest priority*/
  for(i = 0; i < 4; ++i)
        if ((createError = pthread_create(&dumb, &attr,
&Dummy, 0))!=0)
        {     xprintf("pthread_create = %d\n", createError);
              pthread_exit(0);
        }

  /* create Receive task */
  if ((createError = pthread_create(&child, 0, &Receiver,
0))!=0)
  {     xprintf("pthread_create = %d\n", createError);
        pthread_exit(0);
  }

  if(mr_init(1)!=0)
  {     xprintf("Sender: mr_init = %d\n", errno);
        pthread_exit(0);
  }

  /* create 4 dummy tasks with the lowest priority*/
  for(i = 0; i < 4; ++i)
        if ((createError = pthread_create(&dumb, &attr,
&Dummy, 0))!=0)
        {     xprintf("pthread_create = %d\n", createError);
              pthread_exit(0);
        }

  /* sync up */
  rqst.size = sizeof(Msg);
  sched_yield() ;        // let the receive run and init;
  if(mr_send(&rqst, &rqst, child, 0, FOREVER)!=child)
  {     xprintf("Sender: mr_send = %d\n", errno);
        pthread_exit(0);
  }

  /* do the timings */
  errno = 0;
  TimerRestart();
  for(i = 0; i < NLOOP; ++i)
        mr_send(&rqst, &rqst, child, 0, FOREVER);
  if(mr_receive(0, 0, ALLSIGS, &port, FOREVER)!=INTERRUPT_ID)
  {     xprintf("Sender: mr_receive = %d\n", errno);
        pthread_exit(0);
  }

  if(errno || port)
        xprintf("\nTESTS FAILED\n");
  else
  {     xprintf("%d clock ticks for %d iterations\n",start-
MSS_TIM1_get_current_value(), NLOOP);
```

Example e5 (cont'd)
Creating a Unison Application (Sample No. 2 - mrendez)

```
        xprintf("\nTESTS PASSED\n");
  }
}

THREAD Receiver()
{
  Msg rqst;
  int i;
  int out;
  pthread_t parent; // Used for messaging with another thread

  errno = 0;
  rqst.size = sizeof(Msg);

  if(mr_init(1)!=0)
  {     xprintf("Sender: mr_init = %d\n", errno);
        pthread_exit(0);
  }

  sched_yield();   // let Sender run

  for(i = 0; i < NLOOP + 1; ++i)
  {     parent = mr_receive(&rqst, 1, 0, &out, FOREVER);
        mr_reply(&rqst, parent);
  }

  /* return status - if this fails, Main will timeout and
report an error */
  mr_sigpost(parent, errno ? 1 : 0);
}

/* This task is just to fill space and test the algo for
finding TDs */
THREAD Dummy(void *arg)
{#if RTOS_DEBUG
        char name[] = "Dummy( )";
        name[6] = dummy_numb++ + 0x30;
        dir_register(name, pthread_self(), TYPE_THREAD);
  #endif
        /* just block ourselves forever */
        mr_init(1);
        mr_receive(0,0,0,0,FOREVER);
        pthread_exit(0);
        return 0;
}
```

Note that the call *mr_init()* has been performed, so we may now use any Unison services. All Unison services use **standard POSIX or Unix I/O calls wherever possible**. Since we want to use the Unison stdio package rather than the low level *xprintf()* based I/O we must rename all *xprintf()* to *print()* for the next example.

POSIX and Unix I/O

stdin & stdout

The Unison operating system makes no assumptions about the hardware or the environment that the threads will be running on. Thus, Unison threads are **not** created with a pre-determined standard-in (stdin) or standard-out (stdout) as are Unix processes; however they can be mapped using the calls *stdin_init* and *stdout_init*. Stdin and stdout do not exist for these threads without creating these descriptors.

The Unison environment is a single process multiple thread environment. Streams must be opened for **every** thread that is created which wishes to use I/O and these streams are defined using a file name and the directory structure. This is done to minimize memory usage on the target.

Complete character and network based I/O services are available to the programmer to ensure that off-the-shelf services are available and that users do not have to re-create the wheel.

TTY Server

Opening an I/O channel is done with the standard *open()* command. The first argument of the *open()* command is the name of the server. The *open()* call listed below opens a connection with the server called "*/dev/ttys1*". The last number or character is server specific and is interpreted only by the server. In the case of the file server, the string after the server name is the file name you wish to open. In the case of the TTY server, the number after the *ttys* is the particular channel number or comm port you wish. (Each TTY server is capable of handling one serial peripheral. Typically each serial device is capable of handling multiple channels. To open a particular channel on that chip, you must specify the channel number). In the example below, we are opening the first channel on that chip (i.e. channel 0).

If the *open()* request is successful, the caller will be returned a *POSIX File Descriptor* or **fd**. This handle can be thought of as identical to a *POSIX file structure* file descriptor. There is no limit to the number of files that may be opened by a thread; however, there is a soft limit specified in header files to reduce space requirements.

To reduce message traffic between the server and the client, each fd contains a reference to a block of memory. When the thread performs file I/O functions (e.g. *write()*, *read()*, etc.), Unison uses this block of memory to store the results. From the application programming point of view, the user can assume that I/O is immediate because the data read or write is complete from the thread's point of view when the calls return. The server may still have internal buffers of data waiting to be sent, but the data has been copied and will be sent as resources allow.

The stdio package adds yet another layer of buffering, and it will send a message to the server only after enough I/O has occurred to fill this block of memory. This approach is called *buffered* I/O and has the advantage that only large blocks of data will be sent to the server. Commands are available to flush each time called *perprintf* and *noperprintf* to disable flushing. Future releases will add POSIX compliant calls for *perprintf*. Users can also flush streams (ie *fflush*).

In our simple example, we want to see the output immediately after the thread performs a *printf()*. To allow this to happen, we will perform a *perprintf()*. This command causes the Unison stdio package to flush the output after every stdio request.

To summarize, to open a channel for standard I/O the following code must be executed:

Example e6
Opening a Channel for Standard I/O

```
fd = open("/dev/ttys1", O_READ | O_WRITE, 0, 0);
if ( (result= fdopen(fd, "w")) == NULL) xprintf("error open
file, %X”, errno);
perprintf( result );
```

Unison Configuration

Making all the changes listed above will result in the example shown below. This new code can be found in $RRI_DIR/demo/tty. Note that the *xprintf()s* have been removed in favor of Unix style *printf()s*.

Table t7
System Calls Used in the Unison/Nanoexec Application (Sample No. 3 - tty)

Name	Description
open()	open a connection to a Unison server
pthread_exit	delete myself

Example e7
A Unison/Nanoexec Application (Sample No. 3 - tty)

```
#include <sys.h>
#include <fcntl.h>

#define VERBOSE 1

THREAD Main(), tty_shell(), Other();
extern THREAD tty_shell();

// this section initializes the system
// first the first thread is defined, then the
// Nanoexec kernel configuration parameters are defined,
// main starts and it actually initializes the hardware
// and starts the first user thread along with checking
// for a correct cputype and setting up the interrupt stack.
// as well as the system memory pool

// start the first user thread as Main
ConfigTableStruct ConfigTable = {Main, 512, 4};
#define SYSSTK 200
```

Example e7 (cont'd)
A Unison/Nanoexec Application (Sample No. 3 - tty)

```
// start the system with main and do initialization
void main()
{ pthreadStackFill = 1; // fill stack with known values
  hw_init();          // initialize hardware
  Nano_Start();    // start Nanoexec, memory init not
                      //included this should never exit
}

// this is the main section of the program, the above just
initialized
// the system and creates all the threads

THREAD Main()
{ pthread_t pid;
    pthread_attr_t pattr;
  int fd;
    int p_return;
  ssize_t n;

#if VERBOSE
  xputs("Main here!\n");
#endif
#ifdef DEBUG
                 dir_register("Main", pthread_self(),
TYPE_THREAD);
#endif
// we actually create the thread that does the I/O
// it is responsible for initializing its hardware
// and setting appropriate interrupt vectors as it is
// initialized
if(pthread_create(&pid, &attr, &tty_shell, 0)!=0)
{ xprintf("pthread_create = %d\n", errno);
        #if RTOS_DEBUG
              dir_deregister("Main thread");
        #endif
              pthread_exit(0);
}

sched_yield();
  // let tty server initialize before any sends to it
if(dir_register("/dev/ttyS", pid, TYPE_SERVER)==0)
{ xprintf("dir_register = %d\n", errno);
  #if RTOS_DEBUG
        dir_deregister("Main thread");
  #endif
        pthread_exit(0);
}

#if OTHER
if(pthread_create(&pid, 0, &Other, 0)!=0)
{ xprintf("pthread_create = %d\n", errno);

  #if RTOS_DEBUG
```

Example e7 (cont'd)
A Unison/Nanoexec Application (Sample No. 3 - tty)

```
        dir_deregister("Main thread");
  #endif
        pthread_exit(0);
}
#endif

  // note that this is call based on mr_send which will block
  if((fd = open("/dev/ttyS0", O_RDWR, 0)) == -1)
  {     xprintf("open = %d\n", errno);
  #if RTOS_DEBUG
        dir_deregister("Main thread");
  #endif
        pthread_exit(0);
  }

#if DEBUG_READ

  if((n = write(fd, “Hello world!\n”, 13)) == -1)
        xprintf(“write = %d\n”, errno);
  else if(n != 13)
        xprintf(“write returned %d expected 13\n”, n);

  if((n = write(fd, “Hello world!\n”, 13)) == -1)
        xprintf(“write = %d\n”, errno);
  else if(n != 13)
        xprintf(“write returned %d expected 13\n”, n);
#else
  write(fd, “Hello world!\n”, 13);
  write(fd, “Hello world!\n”, 13);
  write(fd, “Hello world!\n”, 13);
#endif

pthread_exit(0);
}

THREAD Other()
{
  int fd;
  int n = 0;
  char str[40];

  strcpy(str, “0: This is from other thread\n”);

  if((fd = open(“/dev/ttys1”, O_WRONLY, 0)) == -1)
  { #if VERBOSE
        xprintf(“Other open = %d\n”, errno);
    #endif
pthread_exit(0);
  }

  while(1);
}
```

Table t8
Directory System Calls

Name	Description
dir_register	register a name with a value and type
dir_deregister	remove registration by name
dir_lookup	search by name
dir_lookup_type	search by type
dir_lookup_string	search by partial name

Directory

The Directory is used to register values which need to be shared among users. In our previous example you can see how dir_register was used to create an entry for the tty_shell thread so that the "Other" thread could get access to the same id. The five routines associated with directory allow users to manipulate directory information and maintain it properly.

Server Threads

In many systems during initialization, all the I/O server threads are created in the background. Users simply use them. In a system like Unison, this is not nearly so desirable for two reasons:

- Only a few I/O ports generally exist, so they are easy enough to manage in application code.
- If it is done automatically, flexibility is sacrificed and in systems like this the main constraint is memory resources. It is best to have maximum flexibility to ensure memory can always be used to best advantage.

Users then must start server threads as done above. After they are started, POSIX calls can be used to do I/O associated with the servers. It is not limited to the few calls used in our last example, a broad set of calls exist including: stdio, sockets, rpc, file I/O and more.

Comments On POSIX I/O and Thread Destruction

In Example e7, the pthread_exit() calls were made to terminate threads if errors occur printing out a message to the polled I/O port. This approach works well for simple systems but it has an inherent weakness. Due to the small size of the system, it does not track all open resources and leaves the responsibility of clean up to the user. In practical terms this means, ***to avoid memory leaks you must:***

- *close all open files before exiting*
- *return all memory buffers to the pool excluding those allocated by the system*
- *return all partition buffers to the partition*
- *remove all directory entries*
- *delete system objects after use*

All thread states will be cleaned up and if error processing is done correctly by other threads, the thread exit should be graceful.

Chapter 4
Memory Maps and Management

Overview

Although it is not required to move an application to the target system, it is useful to have a basic understanding of the various requirements and limitations of memory management systems in the Unison system. For that reason, this information is presented here. It is not strictly required to move forward, so if you want to skip this short section, you can do so.

There are two types of allocation of memory for the system: variable size memory blocks and fixed size blocks. Partitions are fixed size buffer allocation routines and are discussed below. There are several types of routines for variable size block allocation.

Within the variable size allocation, there is a need to support two types of allocation: kernel memory allocation using kalloc and kfree, and thread resource allocation using malloc and free or the pool routines. The malloc and free routines are mapped directly onto the pool routines and the kalloc and kfree routines are mapped onto the malloc and free routines or provided independently. This can become extremely important on DSPs with X and Y memory requirements for certain buffers and instructions.

The main thing to remember is that in such systems there are generally several kinds of memory used for different purposes. One view discussed in the last chapter was:

- program memory
- uninitialized data memory or bss
- text or initialized data memory (which might be split again into constants and dynamically changed variables and which might serve as several different heaps for the system)
- other memory not allocated by the compiler but managed by the user. This would likely offer several different heaps for the system with different characteristics.

Another view would be the hardware oriented and processor oriented view, because each could provide different memory services:

- flash
- ROM
- RAM (potentially several areas with different speeds)
- ePROM
- on chip
- off chip

These two ideas come together and are resolved by the system Memory Map – as discussed next.

Memory Maps

Before we can move the software to the target, we must understand the different sections of code and where the compiler will place them. There are three different sections of memory:

- text
- data
- bss

The first section of memory is the **text** section. This is the section of software that will be executed by the application (i.e. the actual machine instructions). The second section of memory is called the **data** section. The data section contains all *initialized* variables. The last section of memory is called the **bss** section and contains all *un-initialized* data.

When a programmer declares the following code:

```
char * string = "Hello Joe";
int      var1 = 32;
int      var2;
```

The linker will place *string*, and *var1* in the **data** section. The variable *var2* will be placed in the **bss** section since it was not initialized.

Before the application starts, C programs assume that the **bss** section has been zeroed. Since we are going to be initializing the target it will be the our responsibility to ensure that this occurs before starting the application. The code that does this typically resides in a routine called **CRT0** and is supplied for most processors. In our case, we can assume that we will do this initialization even if our emulation/debug hardware may do it as part of its crt0. If our **CRT0** does not do it, the functionality will be found in ***malloc_init()***.

Using the above information, we must design a memory map. The application and kernel must operate out of a fixed, user definable memory. You must define where the **text**, **data**, and **bss** sections of the application will be located. Also, special considerations must be made depending on your target hardware.

For instance, some processors do not have general purpose RAM. In the case of the M16C and R32C, the text segment must go in the program memory while the **bss** must go in the RAM. In the case of the dsPIC, static strings and other status data can go into the text segment too, in which case the initialized data segment is split into two parts.

You should become familiar with all the limitations of your particular hardware before attempting to design a memory map. This includes all devices such as processor firmware, special math library requirements, as well as the MCU and board requirements and locations of all memory and I/O peripherals.

Stack Filling

When looking at memory maps it is often useful to see what the memory was used for. In the case of Nanoexec it has two parameters and one routine that are very useful.

Table t9
Stack Filling Parameters and Functions

Name	Description
pthreadStackFill	if equal to 1, fill thread stack with “S” at creation.
isrStackFill	if equal to 1, fill interrupt with “I” at creation.
checkIstack	check thread stack, isr stack and pool zero bounds when called.

These parameters let you establish what memory was used for what purpose. The checkIstack call dynamically checks stack usage to allow for stack size tuning.

Kalloc and Kfree

These two routines are required to allocate memory for the Nanoexec kernel and Unison environment on the target system. They allocate variable size buffers. All the kernel space is allocated using these routines.

Often simple static memory allocations use a simple version of these routines (see example 1). This is sufficient for many small systems. Larger systems require more elaborate schemes. For example, if memory is deallocated then more sophisticated management is required. Completely static systems are recommended for simple environments.

```
// the default definitions for these routines are as follows
void *kalloc(int type, size_t size)   { return malloc(size); }

void kfree(int type, void *ptr)       { free(ptr); }
```

By default, malloc and free are used for kernel allocation unless the user overwrites these routines. You can see this in the previous paragraph.

Malloc and Free

Malloc and free are the POSIX routines that are generally used for variable size buffer allocation. For that reason, they should be used to make your application more portable. However, in some circumstances, specifically those dealing with either fixed size buffers and performance requirements for those or multiple memory types and heaps, other routines are required.

malloc_init initializes the memory pool for malloc and free. It is easily mapped into a pool create routine for a specific pool that malloc and free will use.

```
void *malloc(size_t size);
void free(void *ptr);
void malloc_init(void);
```

Pools

Memory pools are used for dynamic allocation of variable size buffers. Up to 8 memory pools may be created. Valid *poolno* values are 0 through 7 inclusive. This provides the user with the ability to create regions for all special memory types and manage a collection of heaps for them.

A memory pool is created with a call to ***pool_create()***. *startaddr* and *endaddr* indicated the pool limits. It is up to the caller to provide a contiguous block of memory for the memory pool. *poolno* is a handle which will be used to access the pool. Memory pool *poolno* can be destroyed with a call to *pool_destroy. pool_alloc* returns a pointer to a dynamically allocated block of size bytes from memory pool *poolno*. *pool_free* returns the block of memory, allocated with a call to *pool_alloc*, to memory pool *poolno*.

pool_create() and ***pool_destroy()*** return 0 if sucessful. If they fail, they set errno and return -1.

pool_alloc() returns 0 on failure and a pointer to the allocated buffer if successful.

pool_create() will set the following errors:

```
EINVAL
  poolno is not in the range 0 to 7 or is already in use.
```

pool_destroy() will set the following errors:

```
EINVAL
  poolno is not in the range 0 to 7 or has outstanding memory
allocated.
```

As discussed in the last section, pools often have malloc and free macros to implement POSIX compatible calls for them. These are <u>not</u> included as part of the standard sys.h but are found in *stdlib.h*.

```
#include <sys.h>
int pool_create( int poolno,
unsigned long startaddr , unsigned long endaddr );
int pool_destroy( int poolno );
void *pool_alloc( int poolno , size_t size );
int pool_free( int poolno , void *buf );
pool_alloc() and pool_free()
```

are callable from an ISR.

Partitions

pt_create() creates a partition from which memory can be dynamically allocated in fixed blocksize blocks. *numblocks* blocks will be available in the partition. If sucessfull, ***pt_create()*** returns *ptno*, which is a system identifier which will be used by threads to access the partition. ***pt_getblock()*** is called to get a fixed sized buffer from the partition *ptno*. A pointer to the buffer is returned if the call was successful. ***pt_freeblock()*** returns a fixed size buffer, pointed to by *block*, to the partition *ptno*. *block* must be the value from a successful *pt_getblock* call. *pt_destroy* will delete the partition with the system identifier *ptno* All allocated buffers must be returned to the partition before it can be successfully deleted.

If ***pt_create()*** or ***pt_destroy()*** fails, they will return -1 and set ***errno***. If ***pt_getblock()*** fails, it will return 0.

pt_create() will return the following errors:

```
EINVAL
  blocksize or numblocks are 0.
ENOSPC
  Not enough memory to allocate the partition.
EAGAIN
  All 8 partitions have been allocated.
```

pt_destroy() will return the following errors:

```
EBADID
  ptno was not a valid partition.
EBUSY
  ptno has outstanding blocks allocated.
```

Note that the routine pt_create() calls kalloc(KA_PT, ...) to allocate the memory for the partition. The size passed to kalloc will contain all the overhead for the partition. If you want memory allocated from a specific region, you must modify kalloc to make sure that the type you want is allocated from the pool that you want.

Chapter 5
Moving to the Target

Now that the application is working on the host, we will want to move it to the target. Any of the samples could be used for these purposes. New code for this can be found in $RRI_DIR/Unison/demo. Due to the large number of processor configurations that RoweBots supports, it is impossible to modify this document for all possible configurations. Instead the balance of this chapter will be written with the assumption that the end processor will have a support package and that you the user are familiar with the tools in your environment. As user you will re-configure the compiler and linker for your target system.

Target Startup

No changes are required to move the application from the host to the target in any of the three previous examples. Simply load the software into the IDE, select the correct options for the target and build the target system.

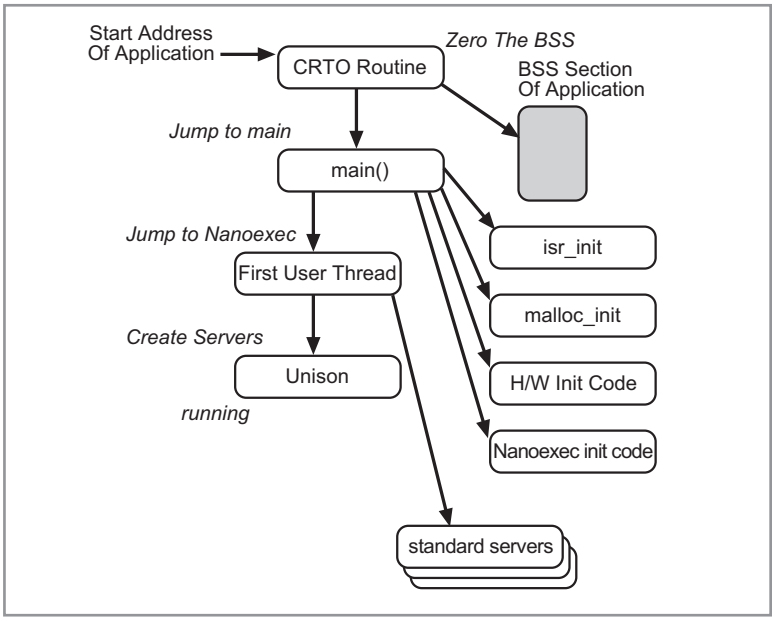
Now you should run on the actual target hardware without changes.

The approach that you take depends on your preferences as well as the target processor in question – not all processors have the instruction level simulator.

We recommend that you first run with the instruction level simulator if this is an option for you. It will allow you to execute all your target code without worrying that the target hardware is not working. One limitation that it generally has is that you must do a considerable amount of work if you want to simulate target peripheral devices. If hardware is readily available and you need to check out peripherals, it is best to move to the target.

In most systems, RoweBots can write a generic startup procedure for all applications. This is because the target is well known. In some target systems, a good understanding of your target system is not known and you will be responsible for ensuring that the application starts correctly. The diagram shown below illustrates the startup sequence for an application that is running on the target. The elements of this were previously discussed and revisited here.

Figure f5
Initialization Sequence For a Unison/ Nanoexec Application In Setup.c



The *main()* used for a simulated configuration is identical to the one required to configure a target version of the application. Both versions have the argc and argv parameters removed.

In a simple system, ***main()*** has the following functions:

- Call *isr_init*.
- Call *malloc_init*.
- Call the hardware initialization routines for the hardware.
- Call the initialization routines for the Nano_Start which never returns.

From here, the user will see Nanoexec start the first user thread, which will in turn complete the creation of system objects and start the I/O servers required.

ISR Initialization and Generic ISRs

Nano_Start is called to pass control into the executive to finish initialization and start the first user thread. The first user thread is then started using the information in the Nanoexec ConfigTable.

At this point, servers will be created and as part of their initialization they will create additional ISR handlers for the system. However, there may be many unhandled ISRs. If this is the case in your system it is a good idea to provide generic ISRs that are installed for unexpected errors.* Whenever an interrupt occurs, this generic ISR will be called

• In some processors i_set_interrupt is replaced by other macros.

unless a call to i_set_interrupt() has been done by a thread. If this interrupt handler is not called, the program will fail without notification because nothing will catch the exception.

Memory Initialization

As Nanoexec starts and creates threads, one of the features is that it zeros uninitialized data. The user does not need to do this.

Also, before any initialization is done, it is required that the user provide kalloc and kfree routines as well as malloc and free if the user wants to use full POSIX calls. It is recommended for larger systems that the pool routines be used for this purpose and that kalloc and kfree are mapped onto malloc and free. Malloc and free should then be mapped in turn onto pool_alloc and pool_free.

Often systems like this have many different types of memory in a variety of places. For example, flash can be used for program memory, static constants and flash disk storage. Various types of EPROM might exist along with various types and speeds of RAM in various locations. These differences are best managed using pools as discussed in the memory chapter. Typically all pools and partitions are initialized before the system starts.

Initializing the Hardware

To initialize the hardware, a call to the standard *hw_init()* routine must be done. The hardware initialization routines require no arguments. The hw_init routine will initialize the system clock and other parts of the system. This includes the initialization of interrupt handling for timer_tick.

The Number of Clock Ticks

The number of clock ticks per second that are required must be specified as the clock is initialized. Only Clock_Real_Time is supported. This information is provided by the user as part of hw_init.

The Clock ISR

The argument to hw_init is the clock ISR that should be called when a clock tick occurs. In most systems, only the kernel is required to handle this call and a pointer to the function *timer_tick()* is passed as an argument.

Kernel Traps

For efficiency, all calls to the Nanoexec are done directly and do not use trap handlers or a trap table.

Kernel Start

Nano_Start is called to pass control into the executive to finish initialization and start the first user thread. The first user task is then started using the information in the NanoexecConfigTable.

Compiling and Downloading

Compiling

All that remains is to compile the application and download it. To compile the application we must inform the IDE that we will be changing from the host based system to the target based system. Typically this is done by changing one of the target options. This will cause the IDE to build the target version rather than the host version.

This will result in an application that may be downloaded to the configuration of hardware you have specified. If you have purchased a complete package from RoweBots, it may include an Ethernet downloading flash memory. Generally though, the user will use a jtag or similar hardware debugging and programming solution to flash the target component.

If you are using various processors from the same family, then you may substitute the processor option for another processor name. A simple rebuild will support your new processor assuming that your memory map and initialization has not changed. If you change processor families, your code will still port, but likely your complete IDE will change.

Downloading

In general, *downloading* is an application that consists of two parts:

- one part that resides on the host workstation as part of the IDE.
- one part on the target or emulator.

The host side reads the executable and sends the text and data sections to the target via the host target communications link. The target side is a simple Unison application that copies the content of the packets into RAM or flash as the case might be, or an emulator or flash programmer. Once the application has completely downloaded it will optionally be told to jump to CRT0 to start *main()* if you have selected this.

To download our application, the download menu item must be selected. Errors will be displayed in the console window.

Controlling the Application

At this point, we have an application that is ready to run sitting on the target system. We have two choices: either run it or run it with a debugger. Unless we specified the debug option to the IDE during the build step, we will only be able to run it. The debug version has extra information placed in the files which allow us to look at all the memory as the system runs and provide a symbolic display of that information. This includes breakpoints by source code line number as well as understanding which variables are on the stack and where.

Unison uses two different types of debuggers: hardware based and software based. Hardware based debuggers are called in circuit emulators and today there is a new flavor which is mostly hardware and some software. We will call both of these hardware debuggers. Typically, hardware debuggers are provided by chip manufacturers and they allow the user to flash or download the memories, and start the program running.

Sometimes, some configurations use debuggers which are much more software than hardware. They simply use trap instructions planted in the code to provide breakpoints and have capabilities to peek and poke memory. The gdb stubs debugger is a good example of this. These debuggers will be called software debuggers. They often have the download built into them and both download and start the debug session.

Regardless of the type of debugger that you have, the process is the same. You go to the debug environment with a current build of the target system. You download or flash the memory and start the debugger. Generally it will automatically halt at the first line of *main()* so you have control from the beginning of your program. If it does not, you will have to place a breakpoint at the first executable line of *main()*.

The debugger generally has little knowledge of the target threads and objects. It is up to the user to look at these objects with the debugger and determine their state and state changes. The user can use the RTOS viewer to do this if available. If this is a limitation in your current toolkit for Unison, it will be upgraded in the future.

Note that when a thread hits a breakpoint in these debuggers it is a stop the world approach. All activity stops on the processor until the user tells the debugger to continue execution. This means that interrupts can be missed which means that the execution of the system will be modified. You must be extremely careful to control interrupts and I/O when using a debugger like this to ensure tha the system operates as you want.

Slow Motion Debugging

Effective debugging of the target system involves both the target hardware and software running under control of the debugger, as well as controlling the external environment. What you need to do is put the world into slow motion and be able to analyze and detect each and every event. By doing this, you can see how the system responds from its various states and how the I/O and timer in the system affect the behavior. It is often a good idea to completely slow down or even disable the timer at some points in order to get an accurate picture of the state transitions (or mode changes if you prefer this expression).

Agile Development

As part of the development, test cases should be developed along with the code. You have been running some of our test cases which were used for initial shakeout of this system as examples. We believe strongly in agile development and would encourage you to practice it. It means that you have reasonable test cases for your system when you move to the target system and try and debug under the most difficult circumstances.

Chapter 6
Synchronizing and Communicating With Other Threads

Communication Overview

Until now we have been concerned with getting a basic system running. Now we will concentrate our efforts on communicating between the different threads on the system. In the previous sections, we have used semaphores. We also demonstrated the rendezvous mechanism which is the underlying mechanism to support servers. We will now explore some of the other ways in which two threads can communicate and more detailed interaction examples for the previously discussed methods.

An Actor Model (Homomorphic Programming)

When designing multi-threaded programs, it is often advantageous to think in terms of humans doing the work that each thread does and seeing the interaction between threads in terms of human interactions. This is a simple principle but it is extremely effective for systems design.

One simple example of human interaction is a producer consumer model. The producer generates an item and hands it to a consumer of the item, then goes to make a new item. The consumer accepts the item and then uses it, upon which the consumer goes back to wait for another item. Thread interactions work in exactly the same way as human interactions.

Actors communicate and synchronize their behavior using a variety of methods in real life and the same is true in our homomorphic programming world. If you need some general service performed, sending a package for example, you fill out a form, provide the package and wait for the clerk to say that all is good, and off your package goes. Using the standard I/O library and I/O Lib is very much like this model. All transactions are standardized into a specific format and processed in the same way, regardless of the details of the transaction. In real world package sending, the differences would be 1 day, 2 day or surface. In the world of actors and communication, the differences might be RAM disk, SD card, hard drive, or serial port.

Actor basic communication and synchronization primitives are quite straightforward and will be explained briefly in the following paragraphs and then in enough detail to allow users to use each method successfully.

Actor Communication and Synchronization Overview

In homomorphic programming, actors need to do several things to work together just like humans do. Sometimes they just need to send a message, other times they need to send an urgent message and yet other times they need to send a message and wait for a response. The methods used for these three types of communication are:

- message queues
- message queues with priority messages
- rendezvous mechanism

Think of the message queue as a simple version of the priority queue. The priority queue is best represented by the queue created when boarding airplanes with different classes of service. First class gets the head of the queue immediately. Business class is second priority and economy class gets the lowest priority.

Similarly, actors need to synchronize and they have some mechanisms for this in the real world as well as the homomorphic world. They need to signal an event, indicate how many of an item or resource are available for processing and manage this count, and make sure records are changed in an orderly fashion and are coherent. The methods used for this are:

- event signals
- semaphores – for counting resources
- mutex – for ensuring orderly records

In this case, an event might be something like, we have an over sized bag at ticket wicket 10, please send someone to get it. A resource would be how many open seats we have on the plane, and orderly record keeping would ensure that all the agents have the same picture of who is traveling on the flight and only one agent can assign new crew members at a time (mutex).

The one last case is that of autonomous robots that assist the humans in the real world. They might carry bags, unload and load baggage and signal events to the humans so they can deal with the issues that arise from the events. Interrupt service routines are like these autonomous robots. They are pushed into service immediately by unexpected events, service them at a low level and then ask the intelligent humans to handle the problem. Typically they use signals but they can also use semaphores, mutexes and message queues. Don't forget, they are not smart enough to block and wait for a resource or response – they must always be ready to do their job if the system is to function.

Now we will look at the details of using these methods.

Semaphores

We saw semaphores used in the very first example but the operation of these semaphores was not discussed at any length. Now there will be more discussion about how they work and why they work in that fashion.

Semaphores can be opened and closed. They can have their value checked. They can be unlinked – that is, no longer used by the current thread. They can be destroyed. They can be acquired or waited on and they can be released or posted. You can also try to wait for a semaphore but continue if it is not available.

Generally, semaphores are used to arbitrate access to shared resources or data structures or regulate access to a specific number of resources. A thread with a semaphore has access and a thread without the semaphore does not have access. In this way, access can be serialized and contained access by a specific number of threads can be provided. This means that the typical execution sequence of these routines would be: initialization, *sema_wait*, use resource, *sema_post*, *sema_wait*, use resource, *sema_post* and so on.

Semaphores can be used to share access to large data areas that many threads need access to but that should not be duplicated. This is a common way to save memory space in these small systems.

To initialize a semaphore, a user can open it. ***sem_open()*** establishes a connection to a semaphore with name, returning the address of the semaphore to the calling thread for subsequent calls to *sem_wait()*, *sem_trywait()*, *sem_post()*, and *sem_close()*. The semaphore remains usable until the semaphore is closed.

name points to a string naming a semaphore object. The *name* argument should conform to the construction rules for a pathname. If a thread makes multiple successful calls to ***sem_open()*** with the same value for *name,* the same semaphore address will be returned for each such successful call, provided that there have been no calls to ***sem_unlink()*** for this semaphore.

oflag determines whether the semaphore is created or merely accessed by the call to ***sem_open()***. The three valid values for *oflag* are **0**, **O_CREAT**, or the bitwise inclusive **OR** of **O_CREAT** and **O_EXCL**. Setting the *oflag* bits to **O_CREAT** will create the semaphore if it does not already exist. Setting both **O_CREAT** and **O_EXCL** will fail if the semaphore already exists. The check for the existence of the semaphore and the creation of the semaphore if it does not exist is atomic with respect to other threads executing ***sem_open()***. After the semaphore named name has been created by ***sem_open()*** with the **O_CREAT** flag, other threads can connect to this semaphore by calling ***sem_open()*** with the same value of *name*, *oflag* set to 0.

Using the **O_CREAT** flag requires a third and a fourth argument: *mode* which is not used and can be set to **0**, and value. The semaphore is created with an initial count of value. value must be less than or equal to **{SEM_VALUE_MAX}**.

If successful, ***sem_open()*** returns the address of the semaphore, otherwise it returns **-1** and sets ***errno*** to indicate the error condition.

After using the semaphore, to release the memory associated with it, the semaphore must be closed by the thread that opened it. ***sem_close()*** is used to indicate that the calling thread is finished using the named semaphore *sem*. ***sem_close()*** deallocates any system resources for use by this thread for this semaphore. If the semaphore has not been removed with a successful call to *sem_unlink()*, then *sem_close()* has no effect on the state of the semaphore. If *sem_unlink()* has been successfully invoked for *name* after the most recent call to *sem_open()* with **O_CREAT** for this semaphore, then when all threads that have opened the semaphore close it, the semaphore will no longer be accessible.

sem_close() should not be called for an unnamed semaphore initialized by *sem_init()*.

If successful, ***sem_close()*** returns **0**, otherwise it returns **-1** and sets ***errno*** to indicate the error condition.

sem_destroy() is used to destroy the unnamed semaphore, sem, which was initialized by *sem_init()*. If successful, ***sem_destroy()*** returns **0**, otherwise it returns **-1** and sets ***errno*** to indicate the error condition.

Often, the value of a semaphore is needed to make intelligent decisions about how to handle a particular case in the system. ***sem_getvalue()*** updates the location referenced by *sval* to have the value of the semaphore referenced by *sem* without affecting the state of the semaphore. The updated value represents an actual semaphore value that occurred at some unspecified time during the call to ***sem_getvalue()***, but may not be the actual value of the semaphore when ***sem_getvalue()*** is returned to the caller.

The value set in *sval* may be 0 or positive. If *sval* is 0, there may be other threads waiting for the semaphore; if *sval* is positive, no one is waiting. If successful, ***sem_getvalue()*** returns **0**, otherwise, it returns **-1**, and sets ***errno*** to indicate the error condition.

It is often good to know what resources are available during initialization, and it is often necessary to set a semaphore to a given value. ***sem_init()*** is used to initialize the unnamed semaphore, referred to by *sem*, to *value*. This semaphore may be used in subsequent calls to *sem_wait()*, *sem_trywait()*, *sem_post()*, and *sem_destroy()*. This semaphore remains usable until the semaphore is destroyed.

If *pshared* is non-zero, then the semaphore is sharable between threads. If the semaphore is not being shared between threads, the application should set *pshared* to **0**.

If successful, ***sem_init()*** returns **0** and initializes the semaphore in *sem*; otherwise it returns **-1** and sets ***errno*** to indicate the error condition.

If, prior to the call to ***sem_post()***, the value of *sem* was **0**, and other threads were blocked waiting for the semaphore, then one of them will be allowed to return successfully from its call to *sem_wait()*. The thread which has been waiting the longest is unblocked.

If, prior to the call to **sem_post()**, no other threads were blocked for the semaphore, then its value is incremented by one. If successful, ***sem_post()*** returns **0**, otherwise it returns **-1**, and sets ***errno*** to indicate the error condition. It is callable from an ISR.

After using a semaphore, *sem_unlink* is used to clean up resources associated with access to the semaphore without actually closing the semaphore until all open connections are closed. ***sem_unlink()*** removes the semaphore named by the string *name*. If the semaphore, *name*, is currently referenced by other threads, then ***sem_unlink()*** has no effect on the state of the semaphore. If one or more threads have the semaphore open when ***sem_unlink()*** is called, destruction of the semaphore is postponed until all references to the semaphore have been destroyed by calls to *sem_close()*. Calls to *sem_open()* to re-create or re-connect to the semaphore will refer to a new semaphore after ***sem_unlink()*** is called. ***sem_unlink()*** does not block until all references have been destroyed; rather, it returns immediately.

If successful, ***sem_unlink()*** returns **0**; otherwise, the function returns **-1**, sets ***errno*** to indicate the error condition, and the semaphore is left unchanged.

sem_wait() and ***sem_trywait()*** are the functions by which a calling thread waits or proceeds depending upon the state of a semaphore. A synchronizing process can proceed only if the value of the semaphore it accesses is currently greater than **0**.

If at the time of a call to either ***sem_wait()*** or ***sem_trywait()***, the value of *sem* is positive, these functions decrement the value of the semaphore, return immediately, and allow the calling process to continue.

If the semaphore's value is 0, ***sem_wait()*** blocks, awaiting the semaphore to be released by another thread.

sem_trywait() fails, returning immediately.

If at the time of a call to either ***sem_wait()*** or ***sem_trywait()***, the value of *sem* is positive, these functions return **0** on success. If the call was

unsuccessful, the state of the semaphore is unchanged, the calling function returns **-1**, and sets ***errno*** to indicate the error condition.

The syntax of the calls is:

```
#include <semaphore.h>
sem_t *sem_open(const char *name , int oflag ,
/* unsigned long mode , unsigned int value */ ... );
typedef struct {
...
} sem_t; /*opaque POSIX semaphore*/
int sem_close(sem_t *sem );
int sem_destroy(sem_t *sem );
int sem_getvalue(sem_t *sem , int *sval );
int sem_init(sem_t *sem , int pshared , unsigned int value );
int sem_post(sem_t *sem );
int sem_unlink(const char *name );
int sem_wait(sem_t *sem );
int sem_trywait(sem_t *sem );
```

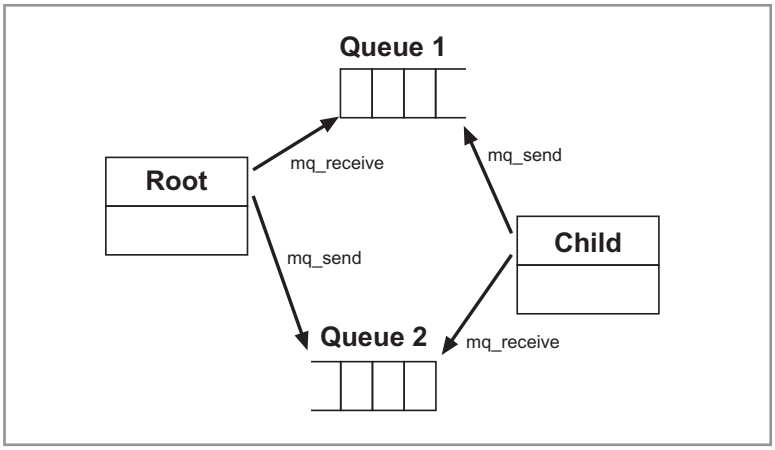
Messages/Queues

Sending messages to another thread using queues has many advantages:

- It is supported directly by the Nanoexec and so is one of the fastest methods for inter-thread communication.
- It is also an asynchronous method of communication which allows a thread to post a message to the other thread without having to give up the CPU.
- It will continue to execute provided that the thread waiting for the message on the queue did not have a higher priority.
  - queues have a maximum message size of 64K bytes and variable size transfers are possible with Nanoexec sorting out the message size on the fly. (In order to send larger messages, the user must use pointers to blocks of memory.)
  - the user can be signalled so they can wait for a combination of messages, signals and synchronous rendezvous calls
  - timed messages are allowed
  - each queue has a separate setable message size and maximum queue length
  - messages are sorted by priority

The main disadvantage of message queues is that communication by queues is asynchronous which may cause the programmer to provide an elaborate series of hand shakes between two communicating threads. The receiver, for instance, may have to inform the sender that the message has been removed from a block of memory passed to it and that block may be re-used. These problems are not insurmountable but care must be taken to ensure the proper exchange of data.

Figure f6
Thread Interaction using Two Threads and Two Queues



The details of the message queue calls are given next. The idea is go through each call so that the user can get some idea of how they fit together. Basically they work like all POSIX calls. First a message queue is created and initialized with the correct attributes. After this, the queues have messages sent to them and received from them. The interaction sequence is up to the user. Many options exist for each queue including:

- message length
- queue length
- notification using signals
- timed sends or receives
- priority of messages

This next section talks about each of the calls in turn. Each one has the variables defined and their operation discussed. The combination of the above section which provides general understanding is intended to be augmented by this section which provides the specifics of each call.

mq_close() removes the association between the message queue descriptor, *mqdes,* and its message queue. Upon successful completion, ***mq_close()*** returns **0** ; otherwise, the function returns **-1** and sets ***errno*** to indicate the error condition.

mq_notify()* provides an asynchronous mechanism for a thread, in our single process multiple thread model, to receive notice that messages are available in a message queue, rather than synchronously blocking (waiting) in ***mq_receive()***.

** mq_notify is not available on all processor configurations. Check procesor specific sections to find out if it is on your processor.*

If *notification* is not **NULL**, this function registers the calling process to be notified of message arrival at an empty message queue associated

with the message queue descriptor. The notification specified by *notification* will be sent to the process when the message queue becomes non-empty. At any time, only one thread may be registered for notification by a specific message queue. Also, if the calling thread or any other thread has already registered for notification of message arrival at the specified message queue, subsequent attempts to register for that message queue will fail.

When the notification is sent to the registered thread, its registration is removed. The message queue is then available for registration.

If a thread has registered for notification of message arrival at a message queue and some thread is blocked in ***mq_receive()*** waiting to receive a message when a message arrives at the queue, the arriving message will be received by the appropriate ***mq_receive()***, and no notification will be sent to the registered thread. The resulting behavior is as if the message queue remains empty, and this notification will not be sent until the next arrival of a message at this queue.

Any notification registration is removed if the calling process either closes the message queue or exits.

Upon successful completion, ***mq_notify()*** returns **0**; otherwise, it returns a value of **-1** and sets errno to indicate the error condition.

mq_open() establishes a connection to a named message queue, name, returning the address of the message queue descriptor to the caller for subsequent calls to *mq_send()* or *mq_receive()*. The message queue once opened remains usable until the message queue is closed by a successful call to *mq_close()*.

name points to a string naming a message queue. The *name* argument must conform to the construction rules for a pathname. If *name* is not the name of an existing message queue and its creation is not requested, ***mq_open()*** fails and returns an error.

oflag requests the desired receive and/or send access to the message queue. The requested access permission to receive messages or send messages is granted if the calling thread is granted read or write access, respectively, to a file with the equivalent permissions.

The value of *oflag* is the bitwise inclusive **OR** of values from the following list. Applications must specify exactly one of the first three values (access modes) below in the value of *oflag*:

O_RDONLY

Open the message queue for receiving messages. The thread can use the returned message queue descriptor with *mq_receive()*, but

not *mq_send()*. A message queue may be open multiple times in the same or different threads for receiving messages.

O_WRONLY

Open the queue for sending messages. The thread can use the returned message queue descriptor with *mq_send()* but not *mq_receive()*. A message queue may be open multiple times in the same or different thread for sending messages.

O_RDWR

Open the queue for both receiving and sending messages. The thread can use any of the functions allowed for O_RDONLY and O_WRONLY. A message queue may be open multiple times in the same or different threads for sending messages.

Any combination of the remaining flags may additionally be specified in the value of *oflag*:

O_CREAT

This option is used to create a message queue, and it requires two additional arguments: *mode*, which is of type ***mode_t***, and *attr*, which is a pointer to a ***mq_attr*** structure. If the pathname, *name*, has already been used to create a message queue that still exists, then this flag has no effect, unless combined with O_EXCL (see below). Otherwise, a message queue is created without any messages in it. The mode field is not used in the current implementation.

** C30/40 users or users on processors with similar addressing constraints may use the BSIZEOF() function instead of sizeof to do the conversion, or a similar function.*

If *attr* is **NULL**, the message queue is created with the default message queue attributes. These can be set by the user in the global variable mqAttrDefault. If *attr* is **non-NULL**, the message queue ***mq_maxmsg*** and ***mq_msgsize*** attributes are set to the values of the corresponding members in the ***mq_attr*** structure referred to by *attr* . The ***mq_msgsize*** must be specified in 8-bit bytes.*

O_EXCL

If both O_EXCL and O_CREAT are set, ***mq_open()*** will fail if the message queue name exists. The check for the existence of the message queue and the creation of the message queue if it does not exist are atomic with respect to other processes executing ***mq_open()*** naming the same *name* with both O_EXCL and O_CREAT set.

O_NONBLOCK

The setting of this flag is associated with the open message queue descriptor and determines whether a calling ***mq_send()*** waits for message buffer space or a calling ***mq_receive()*** waits for messages that are not currently available; or whether the calling function fails, thereby setting ***errno*** to ***EAGAIN***.

Upon successful completion, ***mq_open()*** returns a message queue descriptor; otherwise the function returns ***(mqd_t)(-1)*** and sets ***errno*** to indicate the error condition.

mq_receive() is used to receive the oldest of the highest priority message(s) from the message queue specified by *mqdes* . If the size of the buffer in bytes, specified by *msg_len*, is less than the ***mq_msgsize*** attribute of the message queue, the function fails and returns an error. Otherwise, the selected message is removed from the queue and copied to the buffer pointed to by *msg_ptr*.

If *msg_prio* is not **NULL**, the priority of the selected message is stored in the location referenced by *msg_prio* .

If the specified message queue is empty and O_NONBLOCK is not set in the message queue description associated with *mqdes*, (see *mq_open()* and *mq_setattr())*, *mq_receive()* blocks, waiting until a message is enqueued on the message queue. If more than one thread is waiting to receive a message when a message arrives at an empty queue, then the thread of highest priority that has been waiting the longest is selected to receive the message. If the specified message queue is empty and O_NONBLOCK is set in the message queue description associated with *mqdes*, no message is removed from the queue, and *mq_receive()* returns an error.

Upon successful completion, ***mq_receive()*** returns the length of the selected message in bytes and the message will have been removed from the queue. Otherwise, no message is removed from the queue, the function returns a value of **-1**, and sets ***errno*** to indicate the error condition.

mq_send() adds the message pointed to by *msg_ptr* to the message queue specified by *mqdes. msg_len* specifies the length of the message in bytes pointed to by *msg_ptr* . The value of *msg_len* must be less than or equal to the ***mq_msgsize*** attribute of the message queue, or ***mq_send()*** will fail.

If the specified message queue is not full, ***mq_send()*** behaves as if the message is inserted into the message queue at the position indicated by *msg_prio* . A message with a larger numeric value of *msg_prio* is inserted before messages with lower values of *msg_prio* . A message is

inserted after other messages in the queue, if any, with equal *msg_prio priority*. The value of *msg_prio* must be greater than **0**, and less than or equal to **{MQ_PRIO_MAX}**.

If the specified message queue is full and if O_NONBLOCK is not set in the message queue description associated with *mqdes* (see *mq_open()* and *mq_setattr()*), ***mq_send()*** blocks, waiting until space becomes available to enqueue the message, or until ***mq_send()*** is interrupted by a signal. If more than one thread is waiting to send when space becomes available in the message queue, then the thread of the highest priority which has been waiting the longest is unblocked to send its message. If the specified message queue is full and O_NONBLOCK is set in the message queue description associated with *mqdes*, the message is not queued, and ***mq_send()*** returns an error.

Upon successful completion, ***mq_send()*** returns a value of **0** ; otherwise, no message is enqueued, the function returns **-1**, and sets ***errno*** to indicate the error condition

mq_setattr() is used to set attributes associated with the message queue specified by *mqdes* .

The message queue attributes corresponding to the following members defined in the *mq_attr* structure are set to the specified values upon successful completion of ***mq_setattr()*** :

mq_flags The value of this member is either **0** or **O_NONBLOCK**.

The values of ***mq_maxmsg***, ***mq_msgsize***, and ***mq_curmsgs*** are ignored by ***mq_setattr()***.

If *omqstat* is **non-NULL**, ***mq_setattr()*** stores, in the location referenced by *omqstat*, the previous message queue attributes and the current queue status. These values are the same as would be returned by a call to ***mq_getattr()*** at that point. ***mq_getattr()*** is used to get status information and attributes associated with the message queue specified in *mqdes* . Upon return, the ***mq_flags*** member of the ***mq_attr*** structure referenced by mqstat has the value that was set when the message queue was created but also with modifications made by subsequent ***mq_setattr()*** calls.

The following attributes were set at message queue creation:

- ***mq_maxmsg***
- ***mq_msgsize***

Upon return, the ***mq_curmsgs*** (the number of messages currently on the queue) member of the ***mq_attr*** structure referenced by *mqstat* is set according to the current state of the message queue.

Upon successful completion, these function(s) return **0** ; otherwise, they return **-1**, and set ***errno*** to indicate the error condition.

mq_setattr(), if successful, also changes the attributes of the message queue as specified.

mq_timedreceive() receives the oldest of the highest priority messages from the message queue named by the descriptor *mqdes*.

If the size of the buffer specified by *msg_len* is less than the *mq_msgsize* attribute of the message queue, ***mq_timedreceive()*** fails and returns an error. Otherwise, the message from the message queue named by *mqdes* is copied to the buffer pointed to by *msg_ptr* and removed from the queue.

If the argument *msg_prio* is not **NULL**, the priority of the selected message will be stored in the field pointed to by *msg_prio*.

If the message queue named by *mqdes* is empty and O_NONBLOCK is not set in the message queue description associated with *mqdes*, ***mq_timedreceive()*** blocks until a message is queued on the message queue or until the time interval specified in *timeout* has elapsed. If more than one thread is waiting to receive a message when a message arrives at an empty queue, the thread of highest priority that has waited the longest receives the message. If the specified message queue is empty and O_NONBLOCK is set in the message queue description associated with *mqdes*, no message is removed from the queue, and ***mq_timedreceive()*** returns an error with ***errno*** set to **[EAGAIN]**.

Upon successful completion, ***mq_timedreceive()*** returns the length of the selected message in bytes and the message will have been removed from the queue. Otherwise, no message is removed from the queue, the function returns a value of **-1**, and sets ***errno*** to indicate the error condition.

mq_timedsend() adds the message pointed to by *msg_ptr* to the message queue specified by *mqdes*.

msg_len specifies the length of the message pointed to by *msg_ptr*. If *msg_len* is greater than the *mq_msgsize* attribute of the message queue, ***mq_timedsend()*** returns an error.

If the specified message queue is not full, ***mq_timedsend()*** behaves as if the message is inserted into the message queue at the point indicated by *msg_prio*. A message with a larger numeric value of *msg_prio* is inserted before messages with lower values of *msg_prio*. A message is inserted after other messages in the queue, if any, with equal *msg_prio*. The value for *msg_prio* must be in the range **0** through **_POSIX_MQ_PRIO_MAX**.

If the specified message queue is full and O_NONBLOCK is not set in the message queue description associated with *mqdes*, ***mq_timedsend()*** blocks until space becomes available to enqueue the message or until the time interval specified in timeout has elapsed. If more than one thread is waiting to send when space becomes available in the message queue, the highest priority thread sends its message. If the specified message queue is full and O_NONBLOCK is set in the message queue description associated with *mqdes*, the message is not queued, and ***mq_timedsend()*** returns an error.

Upon successful completion, ***mq_timedsend()*** returns a value of **0** ; otherwise, no message is enqueued, the function returns **-1**, and sets ***errno*** to indicate the error condition.

mq_unlink() removes the message queue named by *name* . After a successful call to ***mq_unlink()***, a call to *mq_open()* with the same name will fail if the flag O_CREAT is not set in flags . If one or more threads have the message queue open when ***mq_unlink()*** is called, destruction of the message queue is postponed until all references to the message queue have been closed. Calls to *mq_open()* to re-create the message queue may fail until the message queue is actually removed. However, ***mq_unlink()*** does not block (wait) until all references have been closed; it returns immediately.

Upon successful completion, ***mq_unlink()*** returns a value of **0** ; otherwise, the named message queue is not changed by this function call, the function returns a value of **-1** and sets ***errno*** to indicate the error condition.

The specific syntax of the routines is given below along with the structures that are used with these routines.

```
#include <mqueue.h>

int mq_close(mqd_t mqdes );

int mq_notify(mqd_t mqdes, const struct sigevent *notification
);   **
struct sigevent {
  int sigev_notify /* notification type */
  int sigev_signo; /* signal number */
  union sigval sigev_value; /* signal value */
};
union sigval {
  int sival_int; /* integer value */
  void *sival_ptr; /* pointer value */
};

mqd_t mq_open(const char *name , int oflag ,
```

*** Not implemented on all processors. Check before use.*

```
  /* unsigned long mode , mq_attr attr */ ... );
struct mq_attr {
  int mq_flags; /* message queue flags */
  int mq_maxmsg; /* maximum number of messages */
  int mq_msgsize; /* maximum message size */
  int mq_curmsgs; /* number of messages currently queued */
...
};

ssize_t mq_receive(mqd_t mqdes , char *msg_ptr,
  size_t msg_len,
  unsigned int *msg_prio );

int mq_send(mqd_t mqdes , const char *msg_ptr,
  size_t msg_len, unsigned int msg_prio );

int mq_setattr(mqd_t mqdes , const struct mq_attr *mqstat,
  struct mq_attr* omqstat );

int mq_getattr(mqd_t mqdes , struct mq_attr *mqstat );

ssize_t mq_timedreceive(mqd_t mqdes , char *msg_ptr,
  size_t msg_len,
  unsigned int *msg_prio,
  const struct timespec *timeout );

nt mq_timedsend(mqd_t mqdes , const char *msg_ptr,
  size_t msg_len, unsigned int msg_prio
  const struct timespec *timeout );

int mq_unlink(const char *name );
```

Below is an example of a message queue based program that illustrates the basic operation of the queues. It is found in $RRI_DIR/Unison/demo/mqueue.

Example e8
Message Queue Test

```
#include <sys.h>
#include <mqueue.h>

THREAD Main(), Child(), Send();

ConfigTableStruct ConfigTable = {
  Main, 1024, 4,           /* root task */
};

extern void checkIstack();
THREAD Main()
{
  struct mq_attr attr;
  mqd_t mq;
```

Example e8 (cont'd)
Message Queue Test

```
  char msg[8];
  pthread_t pthrId;

  xprintf("Main here!\n");

#ifdef DEBUG
                 dir_register("Main", pthread_self(),
TYPE_THREAD);
#endif

  attr.mq_flags     = 0;
  attr.mq_maxmsg    = 3;
  attr.mq_msgsize   = 16;
  if((mq = mq_open("Q001", O_CREAT | O_RDWR, 0, &attr))
== -1)
  {
       xprintf("mq_open = %d\n", errno);
       pthread_exit(0);
  }
  else xprintf("mq = %x\n", &mq);

#ifdef DEBUG
                 dir_register("Q001", mq, TYPE_MQUEUE);
#endif

  memset(msg, 'V', 8);
  if(mq_send(mq, msg, 8, 0) == -1)
       xprintf("mq_send = %d\n", errno);

  memset(msg, 'W', 8);
  if(mq_send(mq, msg, 8, 0) == -1)
       xprintf("mq_send = %d\n", errno);

  memset(msg, 'X', 8);
  if(mq_send(mq, msg, 8, 0) == -1)
       xprintf("mq_send = %d\n", errno);

  if(pthread_create(&pthrId, 0, &Child, 0)!=0)
  {
       xprintf("pthread_create Child = %d\n", errno);
       pthread_exit(0);
  }
#ifdef DEBUG
                 dir_register("Child", pthrId,
TYPE_THREAD);
#endif
  /* Should block */
  xprintf("send Y\n");
  memset(msg, 'Y', 8);
  if(mq_send(mq, msg, 8, 0) == -1)
       xprintf("mq_send = %d\n", errno);
```

Example e8 (cont'd)
Message Queue Test

```
  else xprintf("sent Y\n");

  if(pthread_create(&pthrId, 0, &Send, 0)!=0)
  {
        xprintf("pthread_create Send = %d\n", errno);
        pthread_exit(0);
  }
#ifdef DEBUG
                 dir_register("Send", pthrId,
TYPE_THREAD);
#endif

  if(mq_close(mq) == -1)
        xprintf("mq_close failed\n");

  xprintf("Main done\n");

  checkIstack();
  pthread_exit(0);
}


THREAD Child()
{
  char msg[16];
  ssize_t len;
  mqd_t mq;
  int i;
  char expect;

  xprintf("Child here!\n");

  if((mq = mq_open("Q001", O_RDONLY, 0, 0)) == -1)
  {
        xprintf("mq_open = %d\n", errno);
        pthread_exit(0);
  }
  else xprintf("mq = %x\n", &mq);

  for(expect = 'V'; expect <= 'Z'; ++expect)
  {
        memset(msg, 0, sizeof(msg));
        xprintf("Child waiting...\n");
        if((len = mq_receive(mq, msg, sizeof(msg), 0))
== -1)
        {
              xprintf("mq_receive = %d\n", errno);
              pthread_exit(0);
        }
        else xprintf("GOT MESSAGE %c!\n", expect);
```

Example e8 (cont'd)
Message Queue Test

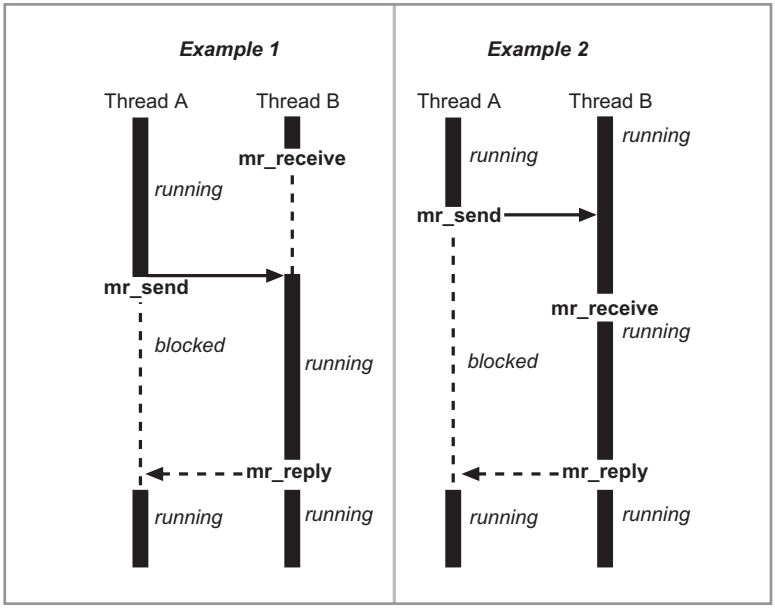
```
        if(len != 8)
              xprintf("expected len 8 got %d\n", len);

        for(i = 0; i < 8; ++i)
              if(msg[i] != expect)
                     xprintf("msg[%d] got %c expected
%c\n", i, msg[i], expect);
  }

  if(mq_close(mq) == -1)
        xprintf("mq_close failed\n");

  xprintf("Child done\n");

  checkIstack();
  pthread_exit(0);
}


THREAD Send()
{
  mqd_t mq;
  char msg[8];

  xprintf("Send here!\n");

  if((mq = mq_open("Q001", O_WRONLY, 0, 0)) == -1)
  {
        xprintf("mq_open = %d\n", errno);
        pthread_exit(0);
  }
  else xprintf("mq = %x\n", &mq);

  /* task will be blocked waiting for message */
  memset(msg, 'Z', 8);
  if(mq_send(mq, msg, 8, 0) == -1)
        xprintf("mq_send = %d\n", errno);

  if(mq_close(mq) == -1)
        xprintf("mq_close failed\n");

  xprintf("Send done\n");

  checkIstack();
  pthread_exit(0);
}
```

Unison Rendezvous

An alternative approach to message queues and semaphores is called the rendezvous and it is available to Nanoexec threads which have used the *mr_init()* call to initialize the thread for receiving. In a rendezvous, the sending thread is immediately blocked until the receiving thread receives, processes the message, and then replies to it. This is shown in the following figure:

Figure f7
Thread States When Using the Rendezvous

Note: These timing diagrams reflect ready to run status. On a uniprocessor further scheduling policies and priorities affect when threads actually run.



The Rendezvous communication technique consists of four steps:

- ***mr_init()*** (non blocking)*
- ***mr_send*** (sender blocks),
- ***mr_receive***, and
- ***mr_reply*** (sender unblocks).

** mr_init must be called by all threads which do mr_ receive before anyone sends to them. A minimum of 1 port is required.*

The rendezvous mechanism between thread A and B is illustrated in Figure f7.

In Example 1, thread B is ready to receive a message before thread A is ready to send it, which causes it to block. thread B unblocks only after thread A has performed the send. Once this has occurred, thread A becomes blocked waiting for a reply. thread A only unblocks again after it has received its reply.

In Example 2, thread A is ready to send before thread B is ready to receive. This causes it to block waiting for thread B to perform the receive and then reply. Only after thread B performs the receive and

does a subsequent reply does the sending thread A unblock. We saw earlier that this was the entire basis for the POSIX style communication in the system dealing with I/O and Unison servers. This was illustrated in Example e7: A Unison/Nanoexec Application (Sample No.3 – tty).

Messages in this system, both synchronous and asynchronous, become a series of C data structures. This is different from message passing that is based on fixed size messages. When using fixed sized messages, C structures become modeled after messages.

In our example we are not that interested in the contents of the message but rather the simple act of performing the transmission. Our data structure will simply be the size of the message and the loop counter. Our data structure will be:

```
struct MSG{
uint_16 size;
uint_16 padding;
uint_32 loopCount;
};
```

Using Primitives

Notice in the data structure above, there is an extra field called padding. This padding field ensures that all long words are *long word aligned* – essential when dealing with different processor types. RoweBots recommends that all your data structures follow this rule of thumb unless you are certain you will not be porting your software to a new architecture.

mr_init() must be called before using ***mr_receive()***. In fact, doing an ***mr_send()*** to a thread without its ports initialized will generate an error. At least one port must be created for each thread using ***mr_receive()***. Ports are numbered from 0–31. You should take care not to create more than you need because each one requires extra data space.

Whenever you wish to send a message or receive a message using the primitives, ***mr_send()***, mr_receive() and ***mr_reply()***, you must **always** set the size of the message to be transmitted or received. Don't forget to call ***mr_init()*** before anyone sends to you though. When the transmission of the message between threads starts, Unison examines the size of the message to be transmitted and the maximum size of the message to be received. If the size of the message is too large, then Unison will truncate the message to fit into the receiver's buffer. Since all messages are based on C data structures, setting the size of the message is simple – simply use the C compiler pseudo function ***sizeof***. The code to set the size of the data structure would then be:

Example e9
Using Primitives

```
#include "sys.h"
#include "stdio.h"

struct MSG{
  uint_16    size;
  uint_16    padding;
  uint_32    loopCount;
};

... rest of the declarations...
void root_thread()
{

  struct MSG rend_msg;
  rend_msg.size = sizeof(rend_msg);
  ... rest of the thread's code...
  mr_send(&rend_msg, &anothermsgrply, tid, 0, FOREVER);
  mq_send(mqdes , &rend_msg, rend_msg.size, msg_prio );
}
```

Communicating With a Child Thread

The advantage of the above code is that as the size of the data structure changes, so does the size of that message. This allows the data structure to change without our having to determine if it will fit into a fixed size message.

Transmitting a Message to a Child thread

To transmit a message to the child, we will be using the *mr_send()* call. This call requires 5 parameters.

- a pointer to the data structure that we wish to pass
- a pointer to the data structure we wish to receive a reply into (In our case we will assume that there is no data in the reply and as a result will use the same data structure for both the request and the response.)
- the thread id of the thread we wish to communicate with (In our case it will be the child's id.)
- the port we wish to have the message posted to (In the example listed so far, we have created the child with only one port. Thus the child will communicate with port number 0. Ports are numbered 0–31).
- a time out (This feature is currently not implemented. Set it to FOREVER.)

The code to send to the child would be:

Example e10
Sending a Message to a Child thread

```
rend_msg.size = sizeof(rend_msg);
rend_msg.loopCount = i;
mr_send((char*)&rend_msg, (char*)&remd_msg,
  tid,0,FOREVER);
```

Receiving a Message from a Child thread

To receive the message on the child side, we must perform an *mr_receive()*. This call requires five parameters.

- a pointer to a block of data where the contents of the sender message will be copied (Since there is only one type of message being transmitted or received, it will simply be a pointer to the same type of data structure. If there were many different types of data structures to be received, it would be important to create a union of all the possible messages types.)
- the port mask (This mask lists all the ports that we are interested in receiving a message from. In our case there is only one port and this value will be 0x1. Notice that to send to port 0 we specify number 0 but to receive from port 0 we specify a mask of 0x1. This is because bit 0 represents port 0. A simple macro to ensure the correct port setting is:

  ```
  #define PORT_MASK(x) (0x1 << (x))
  ```

- a mask of all the signals or events that we are currently interested in receiving (We will not be using events in this example and as a result we will set this field to 0.)
- the signal number that was received, if the thread receives a signal from another thread. (Since we are not interested in this feature this will be set to 0.)
- a time out (The time out is specified in multiples of 100 micro seconds.)

The code required in the child thread to receive the message would be:

Example e11
A Child thread Receiving a Message

```
..The Rest Of the Declarations...

/* The code for the child */
void child_thread()
{
  int i;
  uint_32 msg[4];
  ObjId tid;
  struct UCB * ucb;
  struct MSG rend_msg;
  mr_init(PORTS);   //PORTS>=1
...Initialization Code For Child thread....
rend_msg.size = sizeof(rend_msg);
tid = mr_receive(&(char*)rend_msg,PORT_MASK(0),
  NO_SIG, 0, FOREVER);
...Process The Request...
/* child has to reply to the request */
rend_msg.size = sizeof(rend_msg);
  mr_reply((char*)&rend_msg, tid);
...Rest Of the Child Code...
```

In the above code the reply was also shown. Whenever a thread sends a message that thread is blocked until the reply has been issued. Data may be attached both to the request and to the reply. In the above case we are not concerned with sending any data back to the parent thread.

Example of a Rendezvous

The code for the rendezvous was provided in Sample 2 and can be found in $RRI_DIR/Unison/demo/mrendez.

This message passing scheme was intended for threads which make a request for a service and then wait for a response before continuing execution. Without the rendezvous, two queues would have to be set up in the system: one for the request and one for the response. The rendezvous copies the entire message from the sender's message space into the receiver's message space. No additional memory needs to be allocated. This will reduce the dynamic memory requirements of your system.

Unison Ports

Each Unison thread has a maximum of 32 ports (the exact number is determined at port creation or *mr_init* time). Each thread can determine the set of ports from which it is interested in receiving a message. When the thread executes a *mr_receive()* system call, it passes in a port mask. Each bit in this mask represents one port. By selectively turning these bits on and off, a thread can perform a *"selective"* accept. This allows a thread to receive on any combination of ports based on internal thread conditions. For example, if the thread wants to receive from ports 0 and 1, it simply sets the port mask to 0x3. The exact port the message comes in on is non-deterministic.

By associating a *"meaning"* to each port (e.g. reads use port 0; writes use port 1; and exceptions use port 2), a thread can use the operating system to block unwanted requests. For example, if the thread wants to prevent reading while another thread is writing, then it simply adjusts the port mask to block all readers. Readers remain queued by the operating system until all writing is done. Afterward, the thread re-adjusts its port mask to allow reading to continue.

If you go back to Example e5: Creating a Unison Application (Sample No. 2 – mrendez), you can see how the rendezvous mechanism works in practice.

It is also possible to integrate signals with the rendezvous mechanism. The ability to do this allows the user to wait for events as well as synchronous messages. This is the simple mechanism often used in servers to wait for interrupts as well as wait for user requests. This mechanism can be extended further by setting up signal notification on an asynchronous message queue. Using this extension, a thread could wait on synchronous and asynchronous messages as well as signals.

The rest of this section will focus on messages and their details.

The ***mr_init()*** primitive is used to create and initialize ports for a thread that can be used for receiving messages with the *mr_receive()* call. *nports* defines the number of ports to create. A thread can have a maximum of 32 ports.

** mr_init() must be called before any send is done to the thread.*

If successful, ***mr_init()*** returns **0**, otherwise it returns **-1** and sets ***errno*** to indicate the error condition. The **EINVAL** is provided if the *nports* specifies a value of **0** or greater than **31**.*

The primitive ***mr_send()*** is used to send a message to a thread which is expected to perform an *mr_receive()*. The argument request is a pointer to a message to be sent to the thread identified by *pthread_id*. The *port* argument specifies the port on the receiver's end through which the message is to be routed.

When the receiver performs a *mr_reply()*, it can pass back data in *response*.

The timeout must be a pointer to a timespec struct, which indicates that the sender is willing to block until the *mr_reply()* is forthcoming or the timeout value has been exceeded. If it is 0, the timeout is **FOREVER**.

The first two bytes (16 bits) of the *request* and *response* argument are taken to be the total size, in bytes, of the appropriate message. The lesser of this size and the size obtained from the receiver's messages represents the number of bytes actually copied by the system.

A positive value for *pthread_id* is the id of the thread which received the message. A value of 0 indicates an error. The receiver may return data to the caller in the area pointed to by the *response* argument. The error **EBADID** is reported if the calling thread has attempted to send to a *pthread_id* which does not represent a thread in the system.

EBADPORT is reported if the calling thread has attempted to route the message to a port number which exceeds the number of ports specified for the receiver.

The ***mr_receive()*** primitive is used to receive messages and/or signals from threads. ISRs can also send signals. If both signals and messages are available at the time of the call, the highest priority signal is accepted; otherwise, the first suitable signal or message is accepted.

A message can be sent to the calling thread via the *mr_send()* primitive. The message is received into the area pointed to by *request*.

The *portmask* implements a selective accept mechanism. This argument is a 32 bit port mask indicating which ports the thread is willing to accept a message through. The ports are numbered 0 through 31. To accept from port *n*, bit (**1** << *n*) must be set in the *portmask*.

The *mr_init()* function must be called before ***mr_receive*** can be called. The *ports* argument to a *mr_init()* limits how many ports can actually be received through. It is not an error to set a port in the *portmask* higher than *ports*, but no messages will come through the port.

A signal may be sent to the calling thread by threads or ISRs via the *mr_sigpost()* primitive.

The *sigmask* implements a selective accept mechanism. This argument works just like the *portmask* described above. All threads can accept all 32 signals, there is no limit as in ports.

The actual port or signal received is set in *outport*. The value returned in *outport* is the port or signal number, not its mask. The argument *outport* may be **NULL** if this value is not needed.

The *timeout* argument specifies the time that the caller is willing to wait before the system should indicate that no signal or message was received. The define ***FOREVER*** can be defined if the call should never timeout.

*** Note that signals have priority with low signal values the highest priority.*

Signals of each value are not queued for the thread by the system; therefore, a thread cannot have more than one outstanding signal of each value at any time. The lower a signal's value, the higher its priority; the system does not interpret the value of signals in any other way.**

The contents of the *rqst* are undefined when a signal is received.

There is no guarantee as to which order messages are received from multiple ports, if more than one port contains a message, other than that a fair scheduling algorithm is implemented. Messages are queued FIFO within each particular port.

The first 16 bits of the *rqst* argument are taken to be the maximum size, in bytes, of the message which may be written into *rqst*. The lesser of this size and the size obtained from the sender's *rqst* argument represents the number of bytes actually copied by the system. Note that since the size bytes are the first two bytes transferred, it is possible to determine if the *rqst* was truncated.

If a signal is received, ***mr_receive()*** returns *INTERRUPT_ID*. If a message is received, *mr_receive* returns the sending thread's id. A 0 is returned if the call failed.

The number of the signal received, or the number of the port through which the message was received, is returned in *outport*. The sender's request, if any, is returned in the area pointed to by *rqst*.

The ***mr_reply()*** primitive is used to unblock the thread, specified by *pthread_id*, from which a message has been received with *mr_receive()*

but not yet replied to. Data may be transmitted to that thread via the *response* argument. It is copied into the sender's *response* area.

The ***mr_reply()*** primitive is non-blocking.

The first two bytes (16 bits) found in the response argument are taken to be the size, in bytes, of the entire message pointed to by *response*. The lesser of this size and the size obtained from the sender's *response* argument represents the number of bytes actually copied by the system.

The return value is the *pthread_id* of the thread being replied to, or **0** on error. The ***EBADID*** error means the *pthread_id* specified to be replied to does not represent any thread which is reply blocked on the caller.

The syntax of the routines is provided here.

```
int mr_init( int nports );
pthread_t mr_send( void *request , void * response,  pthread_t
pthread_id , int port , struct timespec *timeout );
pthread_t mr_receive( void *rqst,  BitMask portmask, BitMask
sigmask, int *outport, struct timespec *timeout);
pthread_t mr_reply( void *response , pthread_t pthread_id );
```

Synchronization Methods

More discussion of signals will be provided in the next section.

Signals

Signals are intimately tied to the mr_receive function but they can be used separately. The three functions that are used related to signals are:

- mr_receive
- mr_sigpost
- mr_sigrecv

A signal can be generated using the primitive ***mr_sigpost().*** ***mr_sigpost()*** is used to send a signal to a thread which is expected to perform a *mr_sigrecv()* or *mr_receive()*. *signal_num* specifies the signal number to send. ***mr_sigpost()*** returns **0** on success and returns an error number if it fails. It does not set ***errno***. It is callable from an ISR.

The ***mr_sigrecv()*** primitive is used to receive signals from threads or ISRs. A signal can be sent to the calling thread via the *mr_sigpost()* primitive.

The *sigmask* implements a selective accept mechanism. The argument is a 32 bit mask indicating which signals the thread is willing to accept. The signals are numbered **0** through **31**. To accept signal *n*, bit (**1** << *n*) must be set in the *sigmask*. All threads can accept all 32 signals and a call to *mr_init()* is not required.

The signal received is returned.

An alternative method of catching the signal is to use the rendezvous *mr_receive*. It can wait for multiple events and messages. The ***mr_receive()*** primitive is used to receive messages and/or signals from threads. ISRs can also send signals. If both signals and messages are available at the time of the call, the highest priority signal is accepted; otherwise, the first suitable signal or message is accepted.

A message can be sent to the calling thread via the *mr_send()* primitive. The message is received into the area pointed to by *request*. This was the subject of much discussion above. Let it suffice to say that messages can be received on ports and signals and only the signal portion will be discussed here. Remember though that the *mr_init()* function must be called before ***mr_receive*** can be called. The *ports* argument to a *mr_init()* limits how many ports can actually be received through. It is not an error to set a port in the *portmask* higher than *ports*, but no messages will come through the port. Signals do not need this initialization if they are not using ports in conjunction with them.

A signal may be sent to the calling thread by threads or ISRs via the mr_sigpost() primitive. The sigmask implements a selective accept mechanism. All threads can accept all 32 signals, there is no limit as in ports. The actual port or signal received is set in outport. The value returned in *outport* is the port or signal number, not its mask. The argument *outport* may be **NULL** if this value is not needed.

The *timeout* argument specifies the time that the caller is willing to wait before the system should indicate that no signal or message was received. The define *FOREVER* can be defined if the call should never timeout.

Signals of each value are not queued for the thread by the system; therefore, a thread cannot have more than one outstanding signal of each value at any time. The lower a signal's value, the higher its priority; the system does not interpret the value of signals in any other way.

The contents of the *rqst* are undefined when a signal is received. If a signal is received, ***mr_receive()*** returns *INTERRUPT_ID*. If a message is received, *mr_receive* returns the sending thread's id. A 0 is returned if the call failed. The number of the signal received, or the number of the port through which the message was received, is returned in *outport*.

The syntax of the calls is as follows:

```
#include <mrendez.h>
int mr_sigpost( pthread_t pthread_id , int signal_num );
int mr_sigrecv( BitMask sigmask );
pthread_t mr_receive( void *rqst,
BitMask portmask, BitMask sigmask, int *outport,
struct timespec *timeout);
```

Examples e5 (pg.23) and e12 (pg.79) illustrate the use of a signal.

Mutex

Mutual exclusion locks (mutexes) prevent multiple threads from simultaneously executing critical sections of code which access shared data (i.e., mutexes are used to serialize the execution of threads). All mutexes must be global. A successful call for a mutex lock via ***pthread_mutex_lock()*** will cause another thread that is also trying to lock the same mutex to block until the owner thread unlocks it via ***pthread_mutex_unlock()***

A mutex must be initialized with a call to *pthread_mutexattr_init()* before it can be used. The ***pthread_mutex_init()*** function initializes the mutex referenced by *mp* with attributes specified. Currently, attr must be **NULL**.

A critical section of code is enclosed by the call to lock the mutex and the call to unlock the mutex to protect it from simultaneous access by multiple threads. Only one thread at a time may possess mutually exclusive access to the critical section of code that is enclosed by the mutex-locking call and the mutex-unlocking call. A thread calling to lock the mutex either gets exclusive access to the code starting from the successful locking until its call to unlock the mutex, or it waits until the mutex is unlocked by the thread that locked it.

Mutexes have ownership, unlike semaphores. Although any thread, within the scope of a mutex, can get an unlocked mutex and lock access to the same critical section of code, <u>only the thread that locked a mutex can unlock it</u>.

A call to ***pthread_mutex_lock()*** locks the mutex object. If the mutex is already locked, the calling thread blocks until the mutex is freed; this will return with the mutex object referenced by *mp* in the locked state with the calling thread as its owner. If the current owner of a mutex tries to relock the mutex, it will result in deadlock.

pthread_mutex_trylock() is the same as ***pthread_mutex_lock()*** except that if the mutex object referenced by *mp* is locked (by any thread, including the current thread), the call returns immediately with an error. For this reason it is callable by an ISR providing a third mechanism to communicate between ISRs and server threads (also semaphores and signals can be used).

pthread_mutex_unlock() is called by the owner of the mutex object referenced by *mp* to release it. The mutex must be locked and the calling thread must be the one that last locked the mutex (the owner). If there are threads blocked on the mutex object referenced by *mp* when ***pthread_mutex_unlock()*** is called, the *mp* is freed, and the scheduling policy will determine which thread gets the mutex. If the calling thread is not the owner of the lock, no error status is returned, and the behavior of the program is undefined. This is also callable from an ISR because if someone else holds the lock, it will generate an error.

pthread_mutex_destroy() destroys the mutex object referenced by *mp*, and the mutex object becomes uninitialized. The space used by the destroyed mutex variable is not freed. It needs to be explicitly reclaimed.

If successful, all of these functions return **0** ; otherwise, an error number is returned.

The syntax of these specific routines is given below.

```
#include <pthread.h>
int pthread_mutex_init(pthread_mutex_t *mp,
const pthread_mutexattr_t *attr );
int pthread_mutex_lock(pthread_mutex_t *mp );
int pthread_mutex_trylock(pthread_mutex_t *mp );
int pthread_mutex_unlock(pthread_mutex_t *mp );
int pthread_mutex_destroy(pthread_mutex_t *mp );
```

pthread_mutex_trylock() and ***pthread_mutex_unlock()*** are callable from an ISR.

Chapter 7
Server Design and Interrupt Processing

This chapter provides more detail on the overall structure of servers and how the previous chapter on inter-thread communication is used for communicating within and between servers and applications. It also provides a high level view of interrupt service routines and their integration into this environment.

Server Design

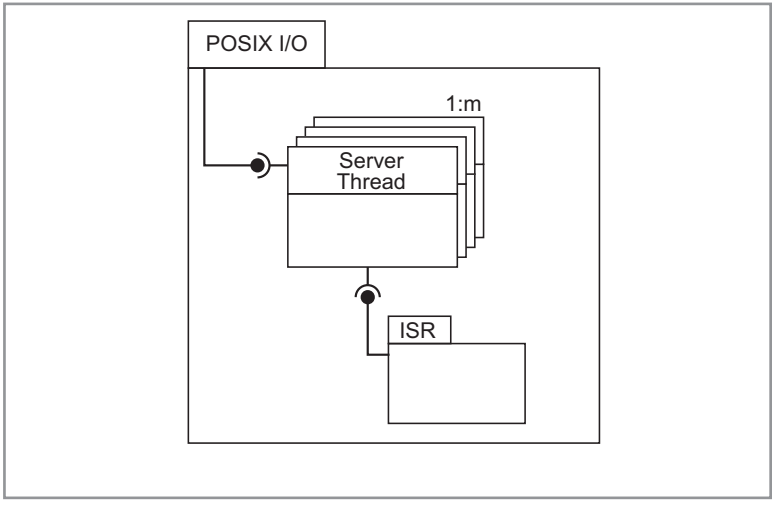
The structure of all servers is similar in some ways. There are basically two classes of servers: ones which directly handle hardware and those that don't. The main difference between these two classes of servers is the interrupt handling support and integration that is required for the servers which handle hardware processing.

From a high level, all servers have the following elements from a requirements point of view:

- Support for POSIX calls – open, close, read, write, ioctl
- Optional support for other networking POSIX calls
- Ability to be created by another thread
- Self initialization

From a design point of view, this means that there is generally one main loop of control which does an *mr_receive* waiting for requests. As requests are made they are processed and clients get the services they requested. In addition, the server internals must process the request.

Figure f8
Server Structures



An extra element might be added here to allow reception of signal events for processing interrupts or receiving notification of *mq_received* messages of an asynchronous nature. Typically, this is how interrupt processing is added to a server. An interrupt service routine is installed

to handle interrupt data and events as they happen, and then as higher level processing is required, the interrupt service routine (ISR) signals the server to do the processing. Typically shared data structures exist between the server and the ISR.

The ISR can do non blocking calls to notify the thread or server that there is work that needs doing. This means that the set of mechanisms that the ISR may use is limited to:

- signals - mr_sigpost
- semaphores - sem_post
- mutex – pthread_mutex_trylock

Other non blocking memory allocation routines are allowed as well.

Of course before the entire server can work, it must be initialized. Often, the initialization information is provided by the creator. In this case, there must be agreement between the server and the creator as to how the information will be passed. Often another mr_receive precedes the main loop in the server for this purpose. The reason that mr_receive is used is that it guarantees that the server is in a good state before the requestor continues, making initialization cleaner and creating the options that are required for a broad set of different operations. During initialization, the ISR is installed and hardware initialized.

Another option for server initialization is the passing of an initialization structure at creation time. This method is generally used by RRI servers. Threads do have initial messages and these can be exploited or a two layer calling scheme may be used to start the server.

The operation of the server is straightforward using mr_receive for initialization synchronization. After initialization, the server waits for requests and processes them. If the server handles hardware, it also receives requests from the ISR. It processes these events too. As requests are serviced, replies are generated to requestors and their requests are completed.

Interrupts

In general, all processors support vectored interrupt handling to allow the user to define a number of interrupt service routines for different devices. Sometimes, these vectored interrupts are nested and sometimes nesting is not possible. Sometimes, they have priorities and other times they do not have priorities. In order to enhance the portability of Unison servers and code, there is a standard method of handling interrupts across all processors.

All interrupts generally require a first and second level handler if they will be using a real-time operating system because the interrupt can cause a change in processing priorities. The interrupt service routine

must be set up in such a way that it supports relinquishing the processor and rescheduling the currently running thread if an interrupt occurs which has forced a higher priority thread to become ready to run.

Another factor modifying how interrupts are processed in systems is the availability of compiler options to support interrupt processing. GNU C and C++ offer attribute arguments to allow this to happen more easily but all compilers do not have this. To allow Unison to be open to a broader set of compilers, a non specific mechanism must be used.

The basic installation of an interrupt vector with an associated priority will be done by the function:

```
void i_set_interrupt( uint_32 vector,
void *handler(),  uint_32 arg)
```

** On some processors an alternate mechanism is used to overcome processor or compiler limitations.*

When the system is ported, this function will be supported so that all servers and interrupt service routines can call it and then they may be more easily moved to other platforms and systems.*

For each processor type and if required each part number, documentation will be provided as an appendix which clarifies the specifics of the interrupt handling on that processor. The appendix will include an overview and a clarification of the implementation to ensure that users understand the operation at a deep level.

Example e12
Unison Server

```
void my_server_example()
{

//  first setup the hardware for device or devices

    if ( my_server_hwrinit() < 0 )
  { xprintf("fatal error setting up my_server hardware \n")
  }
    else {
  // now data structures, thread creation and such for the
server
   ...
        };
     i_set_interrupt( MYSERVERVEC, &myserverISR, 0);
    set_interrupt_priority( HIGHEST); // processor specific, not
                        //provided and then enable interrupts
    _interupt_service(MYSERVERVEC, ENABLE);    /* enable
interrupts for device indicated by vector and now the main
server loop not provided*/

  while( service_requested )
  {
```

Example e12 (cont'd)
Unison Server

```
    ....
    mr_receive( ... ) ;
    switch (message_event_type)
     {
     case  READ:
     ...
     case  ISRSIGNAL:
     ...
    }
    }
  }
```

Processor and Device Specific Information

Each specific processor has its own means of accomplishing different things. It is the organization between these features and the tradeoffs that make processors unique and better purpose suited for some applications over others. For each processor and each server, there is hardware specific information and changes. These details are collected in the appendix of this manual and other manuals for fast and easy reference.

Chapter 8
Using Sockets

Many systems require a means to communicate with another system using a fast data transfer mechanism. One method that has become popular is Ethernet. Ethernet connections allow applications running in an embedded environment to communicate with other applications that may be running on a host computer or in another embedded system.

Ethernet Protocols

Datagrams

Unison supports two different Ethernet protocols. The first protocol is called *datagrams* or *UDP*. Datagrams are the fastest method of Ethernet communication. They take all write requests, package them into an Ethernet packet and then send the packet over the Ethernet. The UDP protocol attaches a checksum to the packet to ensure that *if received by the other machine*, it will be correct. The protocol however **does not** guarantee delivery of that packet. Users are responsible for ensuring that the correct error recover takes place if the packet does not arrive.

Streams

The second method of transmitting a message over Ethernet is to use the *TCP* protocol. This protocol guarantees delivery of the message but is 15-20% slower than UDP. TCP treats all writes or reads as a stream of bytes, giving this communication the nickname streams. Whenever sending a message using TCP, you must incorporate a simple scheme to keep track of the size of the message that was sent. This can be as simple as writing the size of the message before writing the message onto the socket. On the receiving side, the thread would first read the size of the message that was sent and then read the message.

Both TCP and UDP are built on top of the link layer protocol called *IP*. IP is responsible for putting the packet onto the Ethernet. As a result the combined protocol for Ethernet communication is TCP/IP and UDP/IP.

The Ethernet server spends a large amount of time performing checksums on the packets that are sent over Ethernet. The throughput on the Ethernet can be increased by turning off the check summing, in which case another means must be used to ensure that the data sent was not corrupted. Another option would be to do this check summing in hardware off loading the processor. This would require special modifications to the TCP server but is ideal for FPGA implementations.

This section is not intended to describe in detail all the things that one can do with Ethernet. It was designed to get two applications communicating using Ethernet.*

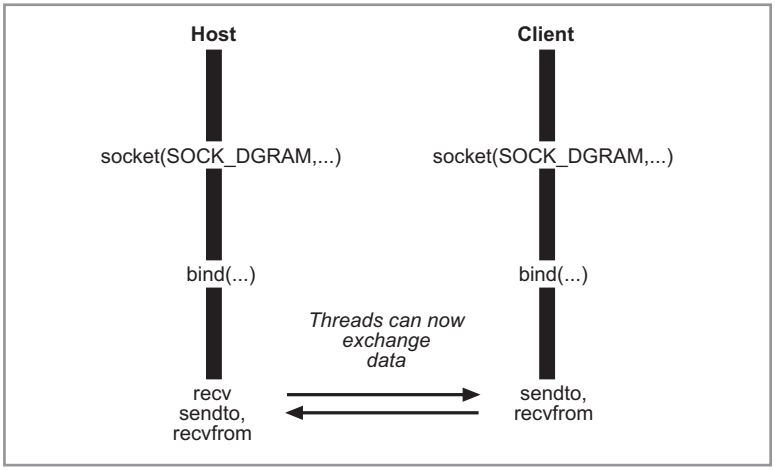
** For a complete description of Ethernet communication, refer to the manual set that came with your workstation or one of several text books that have been written on the subject.*

Setting Up Sockets

Datagram Sockets (UDP)

To get two applications communicating via Ethernet we will first start with UDP (Sample No.5). Code for this can be found in $RRI_DIR/Unison/demo/udp_win. UDP is the simplest method of connecting two applications together. UDP requires two system calls to setup the sockets. When using Ethernet, it is always a *server-client* relationship. That is, one side – the server – is responsible for creating the sockets and the other side – the client – is responsible for connecting to those sockets. The general steps are shown in the following figure:

Figure f9
Setting up UDP Sockets



We will begin by creating the host side of the application. In order to create a socket, one must specify how that socket will be named. The naming convention used is referred to as the *domain*. In most Unix systems, there are two main domains that may be specified:

- Unix
- Internet (INET)

Domains

Unison does not support the Unix domain. In the *Unix* domain, the name for each socket is a file name. Both the server and the client use this file name for establishing a connection with each other. Unix domain names, thus, allow communication between any two processes that reside on the same machine and that are able to access the socket pathnames. This is clearly a restriction for embedded systems since the idea behind using sockets is to communicate with applications not residing on the same machine. It is for this reason that Unix domain sockets are not supported.

Unison supports the Internet domain. Under this domain, addresses consist of machine network address (commonly known as the IP address

of the machine) and an identifying number, called a port. The port is simply an integer that is typically higher than 1023.

To create a UDP socket, first we must make a connection to the Ethernet software using the *socket()* call. The first argument is the domain that the socket will exist in. The only supported format is the ARPA Internet addresses and this parameter must be set to **AF_INET**.

The second field is used to determine if the connection will be streams or datagrams. In this example we will be using datagrams so this field will be set to **SOCK_DGRAM**.

After the *socket()* call, the socket exists in what is referred to as a nameless state. To assign an address to the socket, you use the *bind()* call. This call assigns an address so that other applications may connect to it. The bind call requires a pointer to a ***sockaddr*** structure. A socket address contains three fields that must be filled in. They include the address of the machine, the domain that the name will be in, and the port number of that socket.

Network Addresses

Each machine may have multiple network addresses. When specifying a particular machine address, you must specify one of these valid address or use the wild card value of **INADDR_ANY** (as shown below). When using the INADDR_ANY option the server will be capable of receiving a connection from anywhere on the network. On the client side we will not be able to use a wild card to connect to the server's socket. Instead we will have to specify the IP address of the machine that the server will reside on. It is possible to receive a message from *anywhere* but it is not possible to send a message to anywhere.*

** DHCP is not part of the tcpd server but a separate library which uses tcpd and socket functions.*

Ports

The port number can be thought of as the number of a mailbox, into which the protocol places a message. Certain daemons, offering certain advertised services, have reserved or well-known port numbers. These fall in the range from 1 to 1023. (See the server documentation for port numbers reserved by Unison for dhcp, tftp,thttpd, etc.). Higher numbers are available to general users. Only servers need to ask for a particular number. The system will assign an unused port number when an address is bound to a socket. This may happen when an explicit *bind()* call is made with a port number of 0 or when a *connect()* or *send()* is performed on an unbound socket. Note that port numbers are not automatically reported back to the user. After calling *bind()*, asking for port 0, one may call *getsockname()* to discover what port was actually assigned.

Creating a Socket
Server Side

The code to create a socket on the server side is shown below; note that this example assumes that the server will be running on the host:

Table t10
New System Calls Used Creating a Socket on the Server Side (Sample No. 7)

Name	Description
socket()	creates a socket for network communications
bind()	bind a name to a socket

Example e13
Creating a Socket on the Server Side (Sample No. 7)

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <errno.h>

#define BUFSIZE       65536
#define MAX_PACKET    65536

char sndbuf[BUFSIZE];
char rcvbuf[BUFSIZE];


main()                          // remember this is the host or
server side, not the target
{
  int s, data_s;
  struct sockaddr_in srcaddr;
  int status;
  int nbytes;
  int total = 0;
  int i;

  /* Open sockets */
  s = socket(AF_INET, SOCK_DGRAM, 0);
  printf("Sink creating socket %d\n", errno);

  /* bind the address to the socket - first zero the structure
*/
  memset(&srcaddr, 0, sizeof(srcaddr));
  srcaddr.sin_family = AF_INET;  /* Use the Internet domain */

  srcaddr.sin_port = htons(2000); /* Use port number 2000 */
   /* Take the address of the machine */
  srcaddr.sin_addr.s_addr = INADDR_ANY;

  /* bind the name above to the socket */
  status = bind(s, (struct sockaddr *)&srcaddr,
sizeof(srcaddr));

  printf("Bound address to socket status = %d errno = %d\n",
status,  errno);
```

Example e13 (cont'd)
Creating a Socket on the Server Side (Sample No. 7)

```
/* now begin to communicate with the sockets */

i =0;
do{
  nbytes = rcv(s, rcvbuf, sizeof(rcvbuf), 0);
  if(nbytes < 0){
              perror("Recv failed");
        break;
  }
  total += nbytes;
  if(++i % 100 == 0){
        printf("total %d\n", total);
        }
  }
}while (I<2000)

  printf("Total received = %d\n",total)
}
```

The code above creates a socket that may be thought of as residing on the host. It is now the target's responsibility to connect to that socket. In order to do this the address of the host must be incorporated into the application that is running on the target.

In an environment that consists of multiple hosts and only one target, this is not the most convenient method of using sockets. A better approach would be to put the server on the target side. The host software (because it is not an embedded application and may use command line arguments) can be told the address of the target.

On the target, the calls are identical except for the additional fields for the name of the Unison server and instead of returning an integer for the call to socket a UCB is returned. The target side of this demo (Sample No. 5) is:

Table t11
New System Calls Used Creating a Socket on the Client Side (Sample No. 5)

Name	Description
errno	obtain a thread's error code
sendto()	send a message to a socket
shutdown	shut down part of a fullduplex connection
pthread_exit	exit thread

Example e14
Creating a Socket on the Client Side

```
#include <sys.h>
#include <fcntl.h>
#include <mrendez.h>
#include <iorqst.h>
#include <sys/ioctl.h>
```

Example e14 (cont'd)
Creating a Socket on the Client Side (Sample No. 5)

```
#include <tcpinit.h>
#include <sys/socket.h>
#include <arpa/inet.h>

#define printfxprintf
#define perror(errno)  printf("perror: %s\n",(errno))

#define IP_ADDR_OF_HOST
inet_addr("192.168.16.227")//htonl(0xc0a810e3)

THREAD Main(), tcp_shell();

ConfigTableStruct ConfigTable = {Main, 1024, 4};

#define GW_IP_ADDR    0xf010a8c0      //"192.168.16.240"

#define ENC_ADDR      2       // SPI1
#define ENC_INT_VECT  4
#define ENC_INT_LEVEL 3

static char   tcpbuf[1024*8] ;

struct tcpinit etherinit = {
  sizeof(struct tcpinit),      /* message size              */
  GW_IP_ADDR,                  /* default gateway           */
  ENC_ADDR,                    /* address of enc28j60 chip */
  ENC_INT_VECT,                /* interrupt vector          */
  0xc710a8c0,                  /* internet address
                               *////"192.168.16.199"
  {0x00,0x04,0xa3,0x00,0x4a,0x19},/* ethernet address     */
  {0,0},                              /* padding          */
  ENC_INT_LEVEL,                      /* interrupt level  */
  0,                                  /* unit number      */
  tcpbuf,                             /* buffer pointer   */
  sizeof(tcpbuf),                     /* size of buffer   */
  {'l','e',0},                        /* interface name   */
};

THREAD Main()
{
  pthread_t pid;
  pthread_attr_t attr;
  struct sched_param myNewPriority;

  int source_port;
  struct sockaddr_in source_addr, sink_addr;
  long sndamount = 0;
  char buf[512];
  unsigned long inetaddr;
  int i, n, status;
```

Example e14 (cont'd)
Creating a Socket on the Client Side (Sample No. 5)

```
   xprintf("\nMain\n");

pthread_attr_init(&attr);
myNewPriority.sched_priority = 5;
pthread_attr_setschedparam(&attr, &myNewPriority);
pthread_attr_setstacksize(&attr, 2048);

if(pthread_create(&pid, &attr, &tcp_shell, 0)!=0)
{
      #if DEBUG
                    xprintf("pthread_create = %d\n", errno);
      #endif
      pthread_exit(0);
}

if(dir_register("/dev/tcpd", pid)==0)
{
      #if DEBUG
            xprintf("dir_register = %d\n", errno);
      #endif
      pthread_exit(0);
}

/* create the connection to the socket server */
source_port = socket(AF_INET, SOCK_DGRAM, 0);
if(source_port < 0){
      perror("Error Occurred Opening Socket");
}

/* zero the address data structure */
memset(&source_addr, 0, sizeof(source_addr));
source_addr.sin_family = AF_INET; /* Use Internet domain*/
source_addr.sin_port = 0; /* The TCP/IP software will
                          assign our side of the
                    connection with a port
                    number if this element
                    is a zero */
source_addr.sin_addr.s_addr = INADDR_ANY;


status = bind(source_port, (struct sockaddr *)&source_addr,
                             sizeof(source_addr));
/* source_addr.sin_port will now contain the port
   address that was assigned by the TCP/IP server.

   This is the port address that must be used by
   programs running on other machines if they
   want to talk to this program (i.e. if we
   were sitting on a recvfrom socket call) */

printf("Source bind: status = %d, error code = %d\n",
```

Example e14 (cont'd)
Creating a Socket on the Client Side (Sample No. 5)

```
status, errno);

  /* now set up the address structure to communicate with the
program
  running on the other machine */

  memset((char*)&sink_addr, 0, sizeof(sink_addr));
  sink_addr.sin_family = AF_INET;
  sink_addr.sin_port = htons(2000);
  /* The next argument is the IP address of
     the host machine that the host side
           of this application is running on.
     This argument will have to change for your
           system */
  inetaddr = IP_ADDR_OF_HOST;
  memcpy((char*)&sink_addr.sin_addr, (char*)&inetaddr,
sizeof(long));


  /* setup a pattern in the buffer we are transmitting */
  for(n = 0; n < sizeof(buf); n++){
        buf[n] = '0' + n % 10;
  }

  printf("About to start transmitting \n");
  i = 0;
  do{
        n = sendto(source_port, (char *)buf, sizeof(buf), 0,
                        (struct sockaddr*) &sink_addr,
sizeof(sink_addr));
        sndamount += n;
        if(++i % 100 == 0){
              printf("Source >> sendto, amount %ld\n",
sndamount);
        }
  }while(i < 2000);

  printf("Example udp complete.\n");
  sleep(1);

  shutdown(source_port, 0);
  exit(0);

}

/*(<H>)*******************************************************
 *  Subroutine   : tcp_shell
 *  Purpose      : Task shell for starting device specific
pic32 tcpserver
 *(<E>)*******************************************************/
```

Example e14 (cont'd)
Creating a Socket on the Client Side (Sample No. 5)

```
THREAD tcp_shell(void *arg)
{
  tcpd(&etherinit);
  return 0;
}
```

Compiling and Running the Application

This example must be done on the target. Compile the host side (using the native C compiler on your workstation) then the target. The host side must be running before the target side may run. The Unison servers required on the target are **tcp.a, if_xx.a,** and **iolib.a**.

Running and debugging were discussed in previous chapters on moving to the target.

Using Stream Sockets

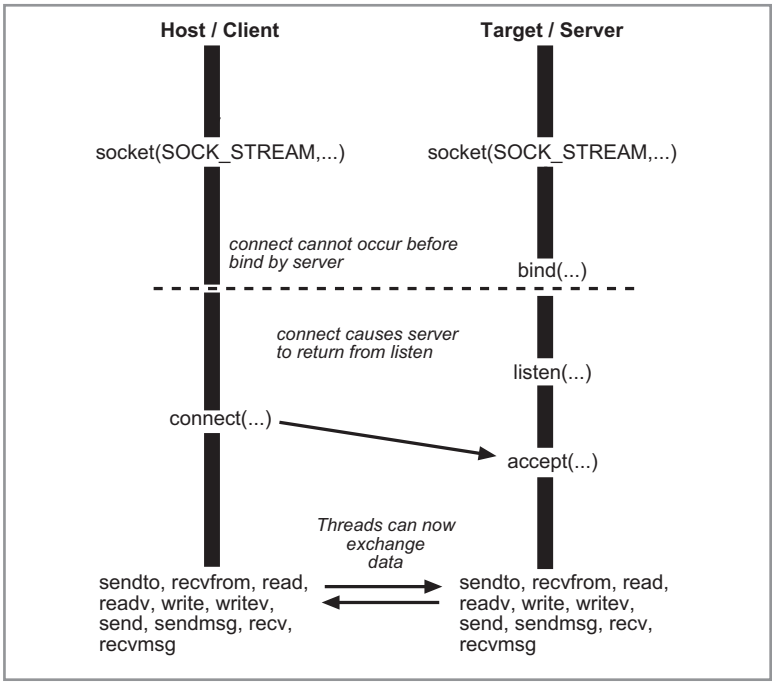
TCP/IP sockets are a little more complicated to setup but have the advantage of an error free link between applications. In TCP/IP sockets, the server executes a different set of Operating System calls than does the client. In our example (Sample No. 6), we will be treating the host as the client and the target as the server. This is the opposite of what was done with the UDP connections. Code for this can be found in $RRI_DIR/Unison/demo/stream.

As with the UDP sockets, the server must create a socket and bind an address to it using the calls *socket()* and *bind()*. There are however two additional calls that must be performed by the server. The socket that is created using the *socket()* call becomes the connection point for all other clients to connect to. After the *bind()* call, the server side uses the call *listen()* to listen for connection requests from client applications.

A client makes a request for a communication channel with the server using the *connect()* system call. If multiple clients make requests for a connection, they are queued up by the Ethernet software until the server side accepts them using the *accept()* system call.

Each accept() call done by the server results in a new file descriptor (fd) being issued. It is this new fd that is used for future communication between the client and server applications. This is shown in the diagram below:

Figure f10
Steps Required to Communicate Via TCP/IP Protocol



Server Side

The software required by the server application is shown below. The socket and bind calls are the same as in the UDP case. The *accept()* call is a blocking call and the application thread will not return from this call until a client makes the request for a connection. The arguments to this call are:

- the socket created by the *socket()* call
- the maximum number of simultaneous client connections

Once returned from the *listen()* call the server then calls the *accept()* system call. The arguments to this call are:

- the socket created by the *socket()* call
- an optional pointer to a structure that will return the address of the client that is connecting
- a pointer to a long indicating the size of the address structure.

Table t12
New System Calls Used with Stream Socket on the Server Side (Sample No. 6)

Name	Description
listen()	listen for connections on a socket
accept()	accept a connection on a socket
send()	send a message to a socket

The code for the server side is:

Example e15
Stream Socket on the Server Side (Sample No. 6)

```
#include <sys.h>
#include <fcntl.h>
#include <mrendez.h>
#include <iorqst.h>
#include <sys/ioctl.h>

#include <tcpinit.h>
#include <sys/socket.h>
#include <netinet/in.h>

#define printfxprintf
#define perror(errno)  printf("perror: %s\n",(errno))

THREAD Main(), tcp_shell();

ConfigTableStruct ConfigTable = {Main, 2048, 4};

#define GW_IP_ADDR    0xf010a8c0      //"192.168.16.240"

#define ENC_ADDR      1       // SPI1
#define ENC_INT_VECT  2
#define ENC_INT_LEVEL 3

static char   tcpbuf[1024*10] ;

struct tcpinit etherinit = {
  sizeof(struct tcpinit),      /* message size            */
  GW_IP_ADDR,                  /* default gateway         */
  ENC_ADDR,                    /* address of enc28j60 chip */
  ENC_INT_VECT,                /* interrupt vector          */
  0xc710a8c0,                  /* internet address
                               *////"192.168.16.199"
  {0x00,0x04,0xa3,0x00,0x4a,0x19},     /* ethernet address  */
  {0,0},                               /* padding           */
  ENC_INT_LEVEL,                       /* interrupt level  */
  0,                                   /* unit number      */
  tcpbuf,                              /* buffer pointer   */
  sizeof(tcpbuf),                      /* size of buffer   */
  {'l','e',0},                         /* interface name   */
};

THREAD Main()
{
  pthread_t pid;
  pthread_attr_t attr;
  struct sched_param myNewPriority;

  int  s, data_s;
  struct sockaddr_in myaddr, clientaddr;
  long sndamount = 0;
```

Example e15 (cont'd)
Stream Socket on the Server Side
(Sample No. 6)

```
  char buf[512];
  socklen_t namelen;
  int i, n, status;

    xprintf("\nMain\n");

  pthread_attr_init(&attr);
  myNewPriority.sched_priority = 5;
  pthread_attr_setschedparam(&attr, &myNewPriority);
  pthread_attr_setstacksize(&attr, 2048);

  if(pthread_create(&pid, &attr, &tcp_shell, 0)!=0)
  {
        #if DEBUG
                        xprintf("pthread_create = %d\n", errno);
        #endif
        pthread_exit(0);
  }

  if(dir_register("/dev/tcpd", pid)==0)
  {
        #if DEBUG
              xprintf("dir_register = %d\n", errno);
        #endif
        pthread_exit(0);
  }

  /* create the connection to the socket server */
  s = socket(AF_INET, SOCK_STREAM, 0);
  if(s < 0){
        perror("Error Occurred Opening Socket");
  }

  /* zero the address data structure */
  memset(&myaddr, 0, sizeof(myaddr));
  myaddr.sin_family = AF_INET; /* Use Internet domain*/
  myaddr.sin_addr.s_addr = INADDR_ANY;
  myaddr.sin_port = htons(2000);

  status = bind(s, (struct sockaddr *)&myaddr,
sizeof(myaddr));

  printf("Source bind: status = %d, error code = %d\n",
status, errno);

  /* wait for a client to request a connection */
  status = listen(s, 5);
  printf("Source listen: status %d, error code %d\n", status,
errno);

  /* accept the client application */
```

Example e15 (cont'd)
Stream Socket on the Server Side
(Sample No. 6)

```
    namelen = sizeof(clientaddr);
    data_s = accept(s, (struct sockaddr*)&clientaddr, &namelen);
    printf("source accept: error code %d\n", errno);

    /* setup a pattern in the buffer we are transmitting */
    for(n = 0; n < sizeof(buf); n++){
          buf[n] = '0' + n % 10;
    }

    printf("About to start transmitting \n");
    i = 0;
    do{
          n = send(data_s, (char*)buf, sizeof(buf), 0);
          sndamount += n;
          if((++i % 100) == 0){
                printf("Source >> send, amount %ld\n",
  sndamount);
          }
    }while(i < 2000);

    printf("Example stream complete.");

    shutdown(s,0);
    close(s);
    shutdown(data_s,0);
    close(data_s);
    exit(0);
  }

  /*(<H>)*****************************************************
   *  Subroutine  : tcp_shell
   *  Purpose     : Task shell for starting device specific
  pic32 tcpserver
  *(<E>)*****************************************************/

THREAD tcp_shell(void *arg)
{
    tcpd(&etherinit);
    return 0;
}
```

Whenever a thread makes a send or receive on a socket there is no guarantee that the entire message will be received or sent on one system call. The Ethernet server will give the application whatever is currently sitting in the Ethernet server's buffer. Multiple reads or writes may be required to transmit the entire message. You should always check the return code from these calls to determine the exact number of characters that where transmitted to or received from the buffer.

Client Side

The client side of the example is:

Table t13
New System Calls Used for Stream Socket on the Client Side (Sample No. 8)

Name	Description
connect()	initiate a connection on a socket
recv()	receive a message from a connected socket

Example e16
Stream Socket on the Client Side (Sample No. 8)

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <errno.h>

/* The following define must be changed to be the IP
    address of the target that the server side of this
    demo will be running on. */

#define IP_ADDR_OF_THE_TARGET  inet_addr("192.9.200.94")

#define BUFSIZE                65536
#define MAX_PACKET             65536

char sndbuf[BUFSIZE];
char rcvbuf[BUFSIZE];

main()
{
  int s, data_s;
  struct sockaddr_in srcaddr;
  int status;
  int nbytes;
  int total = 0;
  int i;

  /* Open sockets */
  s = socket(AF_INET, SOCK_STREAM, 0);
  printf("Sink creating socket %d\n", errno);

  /* bind the address to the socket - first zero the
structure*/
  memset(&srcaddr, 0, sizeof(srcaddr));
  srcaddr.sin_family = AF_INET; /* Use the Internet
  domain*/
  srcaddr.sin_port = htons(2000); /* Use port number 2000*/

  srcaddr.sin_addr.s_addr = IP_ADDR_OF_THE_TARGET;

  /* bind the name above to the socket */
  status = connect(s, (struct sockaddr*)&srcaddr,
```

Example e16 (cont'd)
Stream Socket on the Client Side (Sample No. 8)

```
                sizeof(srcaddr));
  if (status == -1){
                perror(Connect failed")
                exit(0);

  printf("Connected to socket \n");

  /* now begin to communicate with the sockets */
  i = 0;
        nbytes = recv(s, rcvbuf, sizeof(rcvbuf), 0;
        if(nbytes < 0){
                        perror("Recv failed");
                break;
  }

  if (nbytes ==0) break;/*socket was closed*/

        total += nbytes;
        if(++i % 100) == 0){
                printf("total %d\n", total);
        }

  }while(1);
}
```

Compiling and Running the Application

Compile the host side (using the native C compiler on your workstation) then the target. The target side must be running before the host side may run. The options for linking the target must include **tcp.a, if_xxx.a,** and **iolib.a**.

Chapter 9
Unison Servers

All Unison servers must be explicitly created. The basic servers (i.e. the TTY and Ethernet) are started by the user modifiable routine *posix_compat()* located in the hardware support directory for your Unison node (e.g. $RRI_DIR/Unison/demo/hardware /part_or_board_number). Any other services that you wish to use must be started by either modifying the *posix_compat* routine or by having the first application thread start the services that you require separately.

Using Unison Servers

Unison servers are threads that understand client requests. Each client request is packaged into a message and then sent to the server in a standardized format. The server, being a Nanoexec thread, will process the message once it becomes the next ready to run thread. Servers are the same as any other Nanoexec thread. They have no special privileges within the kernel except that they are generally given the highest priorities. Users can create their own servers by conforming to the Unison server protocol.

Both reads and writes by a client thread result in a synchronous message transfer to a Unison server. This happens if the client is performing a read or a write. The client always calls the server – the server never calls the client.

Buffered I/O

A synchronous rendezvous between a server and a client can take a significant amount of time. To increase performance, a buffer is maintained by Unison for each open connection. Any small writes (or reads) work from this buffer. This allows the server to transfer large blocks of data to or from the client. This form of I/O is referred to as *buffered I/O* and has the advantage of reducing both the message traffic and the number of context switches required to output or input data.

Buffered I/O may not be the fastest approach if the application is writing large blocks; it may be faster to simply use *write()* or *read()* system calls. These primitive calls **do not use this buffer**. The contents of the buffer passed into these calls are not copied by the operating system. Instead a pointer to the data is passed onto the server and it outputs the data using the fastest possible method. This is shown in Figure f11.

Whenever a ***stdio*** routine is used to read from a device, the stdio package will always attempt to fill the private I/O buffer and then process the data locally. As a result, this buffer contains the last information read (or written) from disk, tape, etc. For this reason, users should flush this buffer before closing the connection.

Differences Between UCB, FILE, and fd

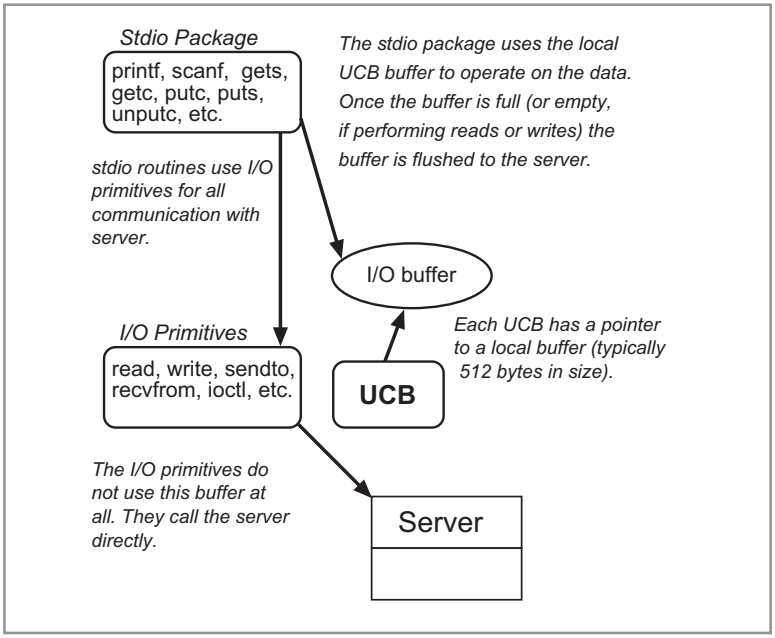
In Unison there are three different ways to reference an open connection to a server. One method is to use FILE which is a pointer to a data structure used to buffer data. The actual name for the data structure is UCB, but with POSIX compatibility you use FILE. Each open file that is created is assigned a unique number. This number, called a file descriptor (fd), is specific to that thread and may not be shared. All POSIX iolib primitives use file descriptors while all stdio primitives use FILEs. Thus the call to *fopen()* returns a FILE while the call to *open()* returns an fd. Be careful when using POSIX system calls interchangeably; there are some differences, most notably buffering.

You cannot interchange *stdio* and *iolib* calls unless they are first converted to the form required by the call. To get the fd number from a FILE, use the system call *fileno()*. To convert from an fd back to a FILE, call *fd2open()*. Normally, all work with Unison servers is done through the FILE primitives. This section will use the FILE primitives.

Sharing FILEs

In general it is not advisable for application threads to share FILEs. If the application has to have multiple threads using the same FILE, care should be taken to ensure that **neither thread attempts to use the stdio package**. If more than one thread is using the stdio package at a time, both threads will overwrite the data previously written by the other thread. These application threads should limit their requests to *read()* or *write()*.

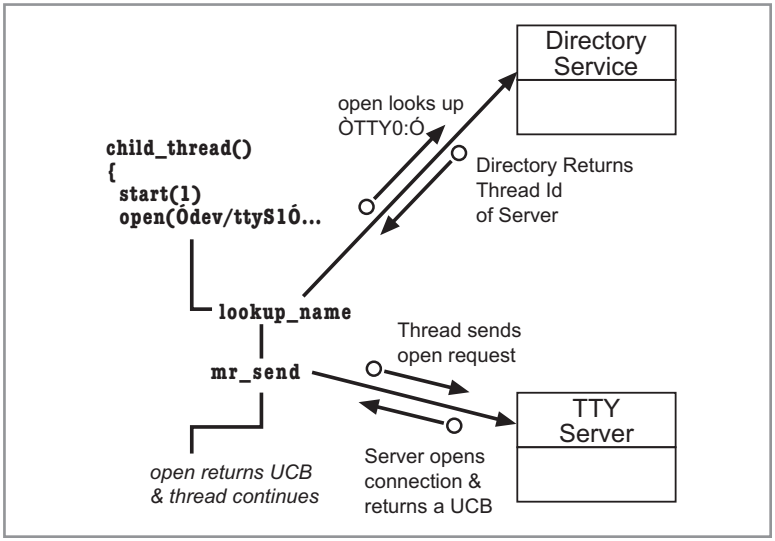
Figure f11
Buffers Used by the Unison I/O Package



Opening Connections

Whenever an application opens a connection with a server, the *open()* routine first looks up the name of the server with the directory service. The directory service then returns the thread id that was registered with that name. The *open()* call then sends a message to that thread requesting a connection to be established. This is shown in the following figure:

Figure f12
Sequence of Events That Occur Upon Execution of an Open Command



The names that are assigned to a server are completely determined by the application program. Each server must be explicitly created and named by the application. The **convention** that is placed on the name is that **every server name must start with *./dev/*** and have a unique server name of a few characters followed by a single character. This is done to emulate Linux device names. Names pass on their single character parameter to the server if the single character is not registered. Servers don't have to have a parameter, and if the parameter is passed on it need not be used. Once registered with the directory service, the name becomes known system wide and any thread on any processor is allowed to establish a connection with that server.

Some servers require specific names. The two servers in this category are the NFS server and the tcp server. The required names are given in the following table.

Table t14
Servers Requiring Specific Names

Name	Device Registrations
NFS	*/dev/nfs* or */dev/NFS*
TCP / UDP	*/dev/tcpd*

NFS

In addition alias names can be established for device servers. By registering an alias name using a server ID, requests to servers can have more organized path names.

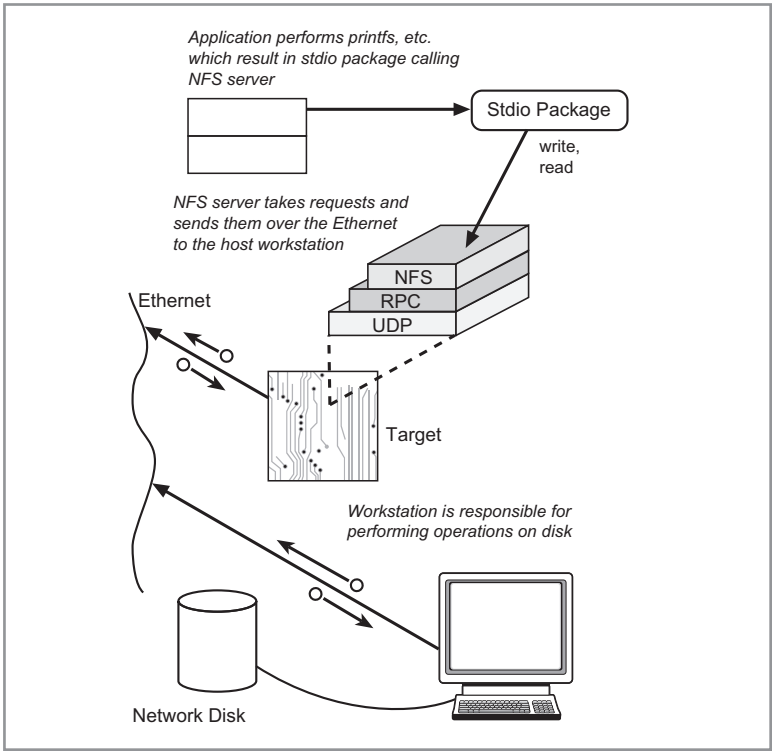
This section focuses upon Unison servers. In earlier sections we have discussed both the TCP/IP and TTY servers. We will now use other Unison servers and in doing so, describe in more detail how Unison servers work. We begin the discussion by describing a very simple **NFS** server.

*** Not all releases contain nfs and rpc for size reasons. Check processor specifics before using.*

RoweBots provides both an NFS server and a local file server. This section describes the usage of both of these servers and provides an example. Code for this example can be found in $RRI_DIR/Unison/demo/nfs.** Code for the file server example may be found in $RRI_DIR/Unison/demo/fsys-ram.

The NFS server allows applications to read and write data to or from a disk mounted on a workstation supporting the NFS protocol. The NFS server is different from other servers in that it does not manage any devices. Instead, it is a series of libraries that use both the RPC and the Ethernet services. As a result it is not possible to use the NFS server if either of these services is not linked into the application.

Figure f13
Sequence of Events When Using NFS Server



Due to the amount of communication between the NFS and Ethernet server, you will see performance improvements if the RPC, TCP/IP and NFS servers are all running at high priority. The NFS server does not buffer any data; any reads or writes are immediately transmitted over the Ethernet.

NFS Protocol

The NFS protocol is stateless; this increases the reliability by ensuring that each transaction with the server is independent of the previous transaction. This gives the host's file server the ability to continue to run even if the target crashes and reboots. Information already sent to the host workstation will not be lost.

To mount a disk on the Ethernet, the application must first create the NFS server. Whenever creating a server, the application is always responsible for assigning a name to it. In the case of the NFS server, the name is assumed by the POSIX compatible *opendir()*, *closedir()*, *readdir()*, etc. routines, to be either ***/dev/nfs*** or ***/dev/NFS*****. It is therefore important to name the server with one of these two names**.

Creating The NFS Server

The NFS server, as with all other servers, requires an initial message to be filled out by the application upon creation. The initial message required by the NFS server contains the following elements:

- the user id that all files are to be created with (uid)
- the group id that all files are to be created with (gid)
- the maximum number of files that are allowed to be open at any one time
- the maximum number of directories that can be mounted
- the name of the target.

Only one user and group id is allowed per NFS server. You must therefore ensure that the group and user id is a valid id on every host machine that you are attempting to use. The code to create the server would be:

Example e17
Creating a NFS Server

```
#include <stdio.h>
#include "nfsinit.h"

#define TARGET_NAME "target"

... Other Initialization Code...
  struct nfsinit;
  threadId nfs;

  /* set the size of the message */
  nfsinit.nfsi_msgsize = sizeof(nfsinit);
```

Example e17 (Cont'd)
Creating a NFS Server

```
  nfsinit.nfsi_uid = 104;  /* setup user id */
  nfsinit.nfsi_gid = 120;  /* setup the group id */
  nfsinit.nfsi_nfiles = 100; /*max. num of files
                      allowed to be open */
  nfsinit.nfsi_mnt_pts = 10;/*max num of mount points allowed */

  /* copy the name of the target into the structure */
  strcpy(nfsinit.nfsi_hostname, TARGET_NAME,
  strlen(TARGET_NAME));
    struct nfsrply { int size = sizeof(nfsrply);
  int status;
                  };

  /* create and initialize  the server */

  /* setup thread initialization parameters */
  pthread_attr( ... );

  /* create the server    */
  if(nfs = pthread_create( ...      )){

        /* send the initial message to the server if not using
NSFshell
         if ( mr_send(&nfsinit,  &nfsrply    , NFSINITPORT,
FOREVER)
              {  xprintf(" NFS initial message error \n");
              };
        if(!dir_register("/dev/NFS:", nfs))
        { xprintf("Unable to register NFS server");
        }
  }else
        { xprintf("Unable to create the NFS server");
        }
... The Rest Of The Application ...
```

Mounting Directories

** If you are unfamiliar with this procedure, you should check the manual set that came with your workstations.*

Once created, we can mount any number of network disks by using the system call *nfsmount()*. The NFS server can only mount disks that are local to the workstation's machine. Each disk that is mounted by the target must have permission from the host workstation. Permission is granted by modifying the file ***/etc/exports*** on the workstation if it is Unix or Linux.*

Each mount point is given a local name. This local name does not have to be the same path name that is present on the host workstation. Parsing of pathnames is done locally on the target system by the NFS server.

The *nfsmount()* call requires:

- the IP address of the machine that is physically connected to the disk

- the directory on the remote machine to be mounted
- the local mount name that is to be used by the Ethernet server. This must include the name of the server (i.e. */dev/nfs* or */dev/NFS*)
- A flag indicating how to mount the directory (non-zero indicates directory should be mounted as read only)

The code to mount a directory on a host machine would be:

Example e18
Mounting a Directory on the Host

```
#include "stdio.h"

/* the following defines must be changed for your environment
to be the host id of the machine that contains a disk that you
wish to mount. */
#define HOST_IP_ADDR   192.9.200.44
#define REMOTE_DIR     /usr/fred

... Other Initialization Code...

  /* mount a valid NFS file system */
  if(nfsmount(inet_addr(HOST_IP_ADDR), REMOTE_DIR,

              "/dev/NFS/home/fred", 0))
        { xprintf("nfsmount failed");
        }
  printf("Mounted the file system %s\n", REMOTE_DIR);

... The Rest Of The Application ...
```

Figure f14a
Sequence of Events When Using the NFS Server

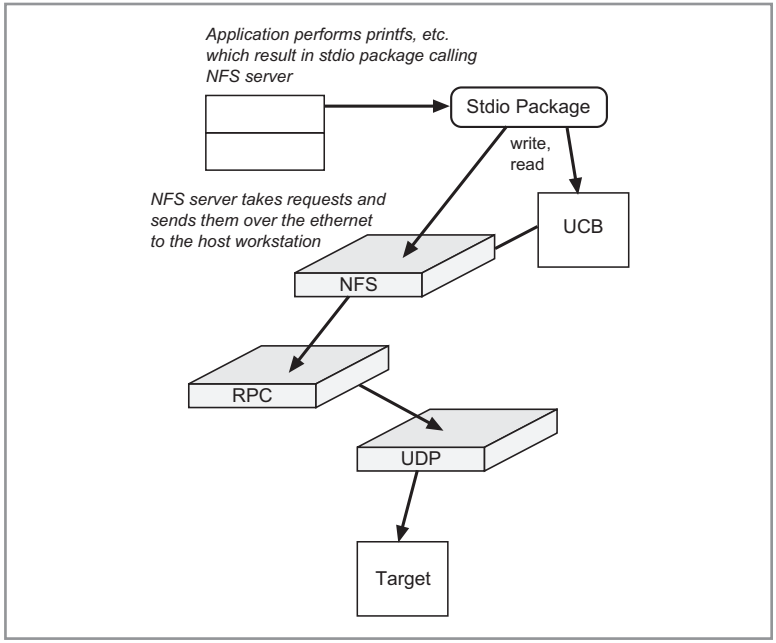
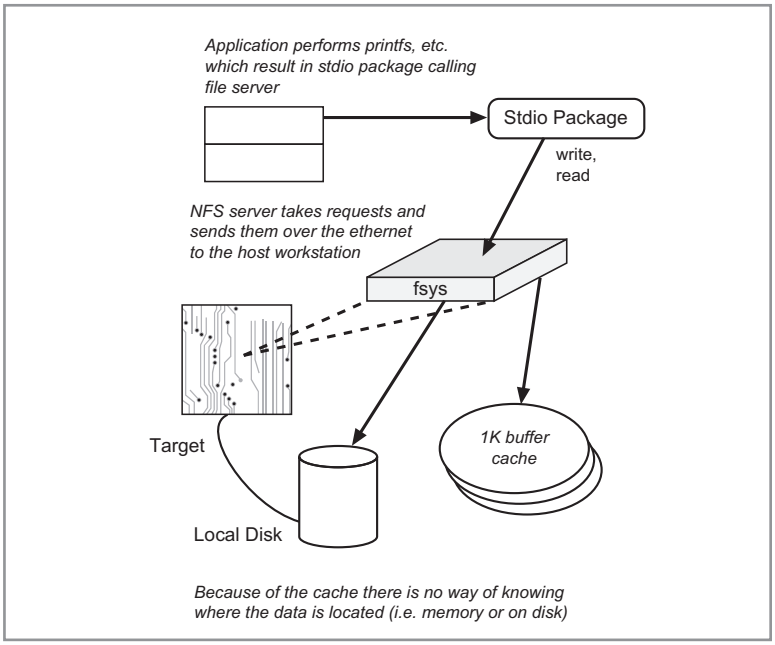


Figure f14b
Sequence of Events When Using the File Server



You can only have one NFS server operating. Each NFS server is capable of mounting any number of host disks on any number of machines. The only limitations are that there must be a sufficient number of unodes and a sufficient amount of memory and resources.

Many routines are generic to the file server and the NFS server. Before we talk about other calls that may be done with the NFS server we will discuss how to create the file server (sometimes referred to as fsys) and then return to discussing how to perform operations on the file system just mounted.

File Servers

Mounting Local Disks

The file server allows applications to mount a local disk. The local file server is faster than the NFS server. It was designed to be the fastest method of storing and retrieving large amounts of data. It does this by maintaining a large disk cache.

Whenever a read or write is performed, the file system does not immediately write the data to the disk. Instead it writes the data to memory and only after the cache is full will it dump it to the disk. Because the server maintains this disk cache, **you must not turn the target off without first syncing the disks**. Syncing the disks forces the server to flush the data from memory to the disk.

O_DIRECT

If the application transfers large blocks of data at a time, it is sometimes faster to by-pass the buffer cache. This is possible by using the **O_DIRECT** flag upon opening the file. This flag informs the file server to by-pass the cache where possible.

This means the data is copied directly between the user's buffer and the disk controller without going through an intervening cache of fixed size buffers. This can improve disk throughput immensely for large I/O requests because the device driver is passed only a few requests rather than one for each cached buffer (currently these buffers are 256 bytes in size).

Disk Cache

In addition, time is saved by avoiding the extra copy to/from a cache buffer. Assume the cache buffer size is 1K for simplicity. Requests to the disk driver will still occur for 1 Kbyte aligned disk addresses, and will still be a multiple of 1 Kbyte. Thus, for instance, if a user requests a transfer of 4 Kbytes starting at offset 512 in a file, the first 512 bytes of the transfer will use the buffer cache because the starting offset is <u>not</u> a multiple of 1 Kbyte. The next 3 Kbytes may be transferred directly between the disk and the user buffer and the remaining 512 bytes will again use the buffer cache. Without the O_DIRECT, all request to the driver would be for 1 Kbyte.

There are two reasons why an aligned transfer may result in more than a single request to the device driver. The first is that the blocks requested may <u>not</u> be contiguous within the file. In this case a separate request must be made for each contiguous region of the file. The second reason is that one of the disk blocks involved may already be in the cache. Before making a request directly to the device, the server searches the cache for a block already present. If one is found, data is copied between the cache and user space, and the driver is then called twice, once for each region on either side of the cached block. (This is a simplification; there may be more than one cached block involved). In this way, cache consistency is maintained and the user never reads stale data from the disk when there is newer data in the cache.

There can be problems with the use of the O_DIRECT. Since data is copied directly between the disk controller and the user thread, Unison <u>cannot</u> protect the system in the event that the client thread is destroyed during the copy. For instance, if the user issues a read request, the controller will eventually copy data directly into the user address space. If the user thread is destroyed after the request has been submitted to the controller, but before it has completed the copy, the copy may be scribbling on memory which shouldn't be scribbled on. A system crash is likely in this case. With the cache, the controller is always copying data between a local buffer cache and the disk.

Cloning the File Server

The file server performs all disk accesses synchronously. When it makes a request for disk I/O it waits until the I/O is complete. This means that a single server will never have more than one disk I/O request pending at a time. This can seriously impede throughput if there is more than one disk drive or if the disk controller hardware is able to put several pending requests into an optimal order.

In order to get around this problem, multiple copies of the file server thread may be created. Only one copy is created by the user; an initialization message is passed indicating, among other things, the number of copies of the server to create. Upon creation, the initial server will make multiple copies of itself. Only the initial server is registered with the Directory service (this must be done by the server's creator).

When a user thread issues an *open()* on a file, Unison asks the directory service for the thread id corresponding to the server name portion of the filename. This will be the id of the initial server. This server then sets up the connection, and if all is successful, determines which one of its clones (or itself) is the least busy. It then tells the client that all further requests on the open connection should be made to this least-busy server. This binding is in effect until the connection is closed with the *close()* call.

Note that this means that file system requests which do not take an open fd as parameter (such as open(), mkdir(), stat()) are all fielded by the initial server which can therefore still be somewhat of a bottleneck. However, the most disk intensive calls (read() and write()) can be handled by clone servers. Currently, the initial server determines the least busy server by finding the one that is handling requests on the least number of open connections. This may not be the best metric.

Note also that I/O requests are still synchronous from the user thread's point of view. If this is not the desired behavior, extra transporter or buffer threads could be set up by the user to do the I/O on behalf of the producer/consumer thread.

Creating the File Server

When the initial server thread is created, it must be passed an initialization message using mr_send. The format of this message is defined in the file "**fsysinit.h**" and requires the following information:

- the total number of servers which should be created
- the number of 256 byte buffers to configure into the buffer cache (The server will not allow less than 4.)
- the maximum number of different files that can be opened simultaneously (The same file opened more than once counts only once. The server will not allow less than 16.)

A code fragment to create the file server would be:

Example e19
Creating a File Server

```
#include "stdio.h"
#include "fsysinit.h"

... Other Initialization Code...
  threadId tid;
  struct fsysinit finit;

  finit.fsi_msgsize = sizeof(finit);
  finit.fsi_nclones  = 2; /* two copies of the file server */
  finit.fsi_nbuffers = 32; /* Make 32 x 256 byte or 8K buffer
cache */
  finit.fsi_maxopen = 32; /* Allow up to 32
                      different files to be opened */

  if( pthread_create( ... attributes ... )){

        /* setup parameters */
        pthread_attr( ... );

        /* File server can have any Linux name */
        if((dir_register_name("/dev/hda", tid)){
              xprintf("Unable to register server name");

        }
  }else{
        xprintf("Unable to create File server */
  }
... The Rest Of The Application ...
```

In the case of the NFS server, the file server simply mounted the disk that was local to another machine. The file server however, must manage the disk itself. This includes initializing the disk. Before any application can mount the disk it must have been previously initialized with the command *mkfs()*. This call only has to be performed once. From then on in, it only has to be mounted by using the call *mount()*.

Major & Minor Device Numbers

Both the *mount()* calls and the *umkfs()* system calls require a device number. The device number is made from a combination of two numbers. The two numbers are referred to as the **major** and **minor device** numbers. (See Figure f15).

Each device that is supported by the file server requires its own hardware specific routines that describe how to open, close, read, and write to the device. A pointer to these routines is typically placed into a table in the hardware directory for the processor that the file server will be running on. The **major** device number is the offset into the bdevsw table that contains the hardware specific routines to operate on that

device. A typical Device Table would be as follows:

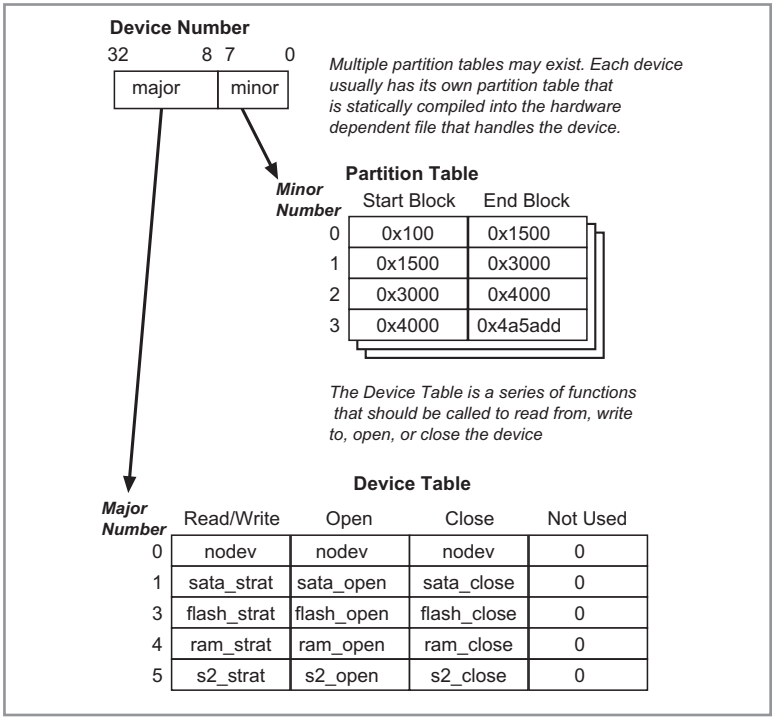
Example e20
A Typical Device Table

```
/* This structure is defined in the file “device.h” */
struct bdevsw {
  int   (* bdev_strategy)(int, int, int, char*, int);
                      /* routine used for reading/writing */
  int   (* bdev_open)(int);  /* routine used to init & open
device */
  int   (* bdev_close)(int);  /* routine used to shutdown
device */
  int    (* bdev_reservered)(int); /* Always set to zero */
}

/* must define an invalid device routine */
int nodev() {
  return (E_NO_DEV);
}

/* define the device table */
struct bdevsw[] =
{
  {nodev, nodev, nodev, 0},/* zero is an invalid device number
*/
  {scsi_strategy, scsi_open, scsi_close, 0},
  {ram_strategy, ram_open_close, ram_open_close, 0}
  {flash_strategy, flash_open_close, flash_open_close, 0}
};

/* fsys also needs to know the size of this table */

int bdevswmax = sizeof(bdevsw) / sizeof(struct bdevsw);
```

How Unison Determines a Device Number

The minor device number is used by the file server to partition a disk into several file systems. This minor device number is not interpreted by the file server. It is only used by the hardware specific routines and, as a result, the definition of this table is left to the person maintaining the Chip or Board Support Package for that hardware device.

Figure f15
How Unison Determines a Device Number



A typical **minor** Device Table definition would be:

Example e21
A Typical Minor Device Table Definition

```
static struct partition {
  int   pt_base;      /* First block of partition */
  int   pt_end;       /* Last block + 1 of partition */
} parttab[] = {
  0x100,0x1500,
  0x1500, 0x3000,
  0x3000, 0x4a5dd,
};

#define num_minors (sizeof(parttab) / sizeof(struct
partition))
```

Since the minor device number is interpreted by the hardware specific routines, its existence is not important in all configurations. The file server is capable of mounting any number of devices at any one time.

Creating a RAM Disk

We will construct a RAM disk in order to illustrate some of the features described above. The first item that must be done is to create a series of routines that allow the file server to open, close, read and write to RAM. The routines would be:

Example e22
Creating a RAM Disk

```
#include “stdio.h”
#include “device.h”

/* We are not going to partition the ram into small sections -
Instead, just create a large block of memory and use the
entire block */
#define DISK_SIZE (0x1024 * 0x1024 * 1) /* 1 Meg RAM disk */
static char * ram_disk[DISK_SIZE];

/* now require a routine to open/close the RAM  - this is a
trivial routine since RAM always exists and there is no
hardware initialization to do - Simply return EOK */
int ram_open_close(dev)
udev_t  dev;
{ return (EOK); /* ram disk is now initialized */
}

/* Next routine is used to read/write data from/to the disk */
int ram_strategy(dev, off, len, buf, rwflag)
uint_16 dev;
unsigned int off;
unsigned len;
char * buf;
int rwflag;
{ char * ram, * host;
  int verdict;

  if((off + len) > (sizeof(ram_disk))
  /* error - file server has attempted to go off the
      end of the disk - return disk is full */
       {return EBADBLOCK; /* NOTE: should never happen */
        }

  /* convert pointers into External addresses - required by
   vmecpy routines - vmecpy routine is used instead of
   memcpy routine to ensure all data coping is handled by one
hardware specific routine - not required for niprocessors */
  ram = (char*)addr_to_net(ram_disk + off);
  host = (char*)addr_to_net(buf);

  /* now copy data to ram disk */
  if(rwflag == BDEV_WRITE)
        { verdict = vmecpy(ram, host, len);
        }else{verdict = vmecpy(host, ram, len);
              }

  /* if verdict is non-zero the memory copy routine worked */
  return (verdict == 1 ? EOK : ESTALE);
}
```

Since RAM always exists, there is no initialization of the device required. All that is necessary for the open routine is to return EOK (indicating device was opened with no errors). If we had a real device we would have to initialize the hardware and prepare it for the reads or writes that will be occurring in the *ram_strategy* routines.

The closing is also simple in RAM. If a real device was used, it would be necessary to *idle* the hardware. This routine will be called by the file server when the application thread has requested to unmount the disk.

The *ram_strategy* routine is the routine called by the file server whenever it wants to read or write a block of data. This routine is not concerned with the strategy used by the file server in writing data to the file. The routine should simply write (or read) the data to (or from) disk. The **off** parameter is the location on the device that the file server is attempting to write to (or read from). Once the device has been moved to that location, this routine is responsible for writing the data specified in the buffer pointed to by **buf** to the device.

The next item is to use these routines in building a Device Table. The code required to build the Device Table would be:

Example e23
Building a Device Table

```
#include “stdio.h”
#include “device.h”

/* must define an invalid device routine */
int nodev() {
  return (ENODEV);
}

extern int ram_strategy(), ram_open_close();

/* define the device table */
struct bdevsw[] =
{
  {nodev, nodev, nodev, 0},/* zero is an invalid device number
*/
  {ram_strategy, ram_open_close, ram_open_close, 0}
};

/* fsys also needs to know the size of this table */
int bdevswmax = sizeof(bdevsw) / sizeof(struct bdevsw);
```

Once a file system has been made, it can then be mounted. This is done by using the *mount()* system call. This call requires:

- The device number to be mounted
- The file server name including the directory mount point for that device

- Whether or not the file system should be mount read only

In both cases, the device number is the offset in the device table shifted by 8 or more specifically:

```
#define RAM_DEVICE_NO  (0x1 << 0x8)
```

The minor device number occupies the lower 8 bits and can be ignored. The code required to create a file system on the RAM disk above would be (assuming that the file server has been previously created with the name **"/dev/hda"**).

Example e24
Creating a File System on RAM Disk

```
#include "sys.h"

#define RAM_DEVICE_NO  (0x1 << 0x8)
#define DISK_SIZE (0x1024 * 0x1024 * 4) /* 4 Meg ram disk */
#define MAX_FILES 300

...Rest Of Initialization Code...
int status;
int device;

status = mkfs("/dev/hda", RAM_DEVICE_NO,  DISK_SIZE,
MAX_FILES);
if(status == -1){
  xprintf("Unable to make a file system");
}

if((status = mount(RAM_DEVICE_NO, "/dev/hda/usr", 0)) == -1){
  xprintf("Unable to mount ram disk");
}
```

*** Note that a file path which is unique must be used for each mount command to avoid confusion.*

With the above code, the ram disk is now made and mounted. Any file may be opened in the directory "/dev/hda/usr/..." **.

Remember that the file server maintains some of its data structures in a buffer cache. Before the disk is closed or power is turned off the file server must be synchronized with the operating system call *sync()*. This may not be an important step in a RAM disk since the data will be lost when the power is turned off but it will be **essential on a non-volatile storage device**.

Controlling the Amount of Space Allocated to a File

When a file is created, the user must specify the size of the new file and the file system will allocate that much contiguous space on the device. As the file is filled with data past the initial specified size, it will be *extended* to allow the user to continue writing to that file. This type of scheme is known as an **extent based file system**. Currently the Unison file system allows for 13 extents. If the user attempts to write more than

13 times the original file size, an error will be returned indicating that the file is filled. The maximum number of extents is controlled by the unode data structure.

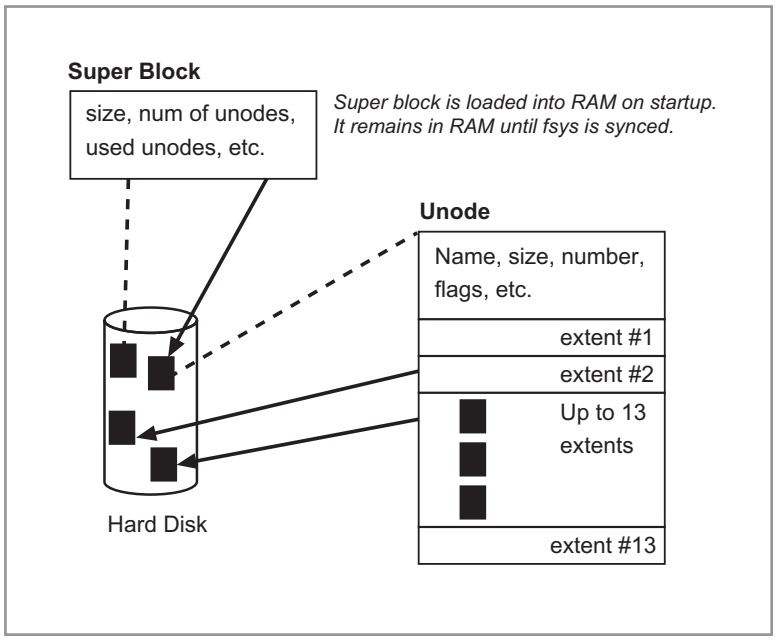
Each unode represents one file or directory. It is comprised of data that represents that file–including name, size, flags, owner, etc. Each unode is capable of pointing to multiple locations on the disk. (This is illustrated in Figure f16).

When the file server first starts up, it loads the super block into memory. The super block is the first block on the partition and contains:

- all the information concerning which blocks are in use
- general disk information (size, etc.)
- a pointer to the root directory

This super block is maintained in memory and is only copied back onto the disk during the synchronizing operation. If the synchronizing operation is not performed on the disk then any updates to that disk may be lost.

Figure f16
How Unison's File Server Uses the Disk



A thread opening a file for writing can pass the **O_CONTIG** flag, and then tell the server the amount of contiguous space which should be allocated to a file whenever it needs to be grown due to a *write()* call, even if this is more than the amount actually needed to satisfy the *write()*. However, it may sometimes be desirable that a large contiguous

region be initially allocated to a file and that the space be grown by somewhat smaller amounts when required. This can be arranged as follows:

Example e25
Allocating a Contiguous Region to a File

```
fd = open(name, O_CREATE|O_CONTIG|O_WRITE|O_DIRECT,  mode,
grow);
seek(ucb, initalloc, SEEK_SET);
if (write(ucb, (char *)0, 0) < 0)
/* Probably could not allocate “initalloc” bytes */
seek(ucb, 0, SEEK_SET);
```

The point is that a seek to offset “**initalloc**” in the file followed by a write of 0 bytes causes “initalloc” bytes of contiguous space to be allocated to the file. Note though that this also causes the file size to be set to “initalloc” which may be undesirable. There is no system call that can shrink the amount of space allocated.

Fat File System – fatfs

The Windows compatible file system fatfs is ideal for use in embedded applications which require removable media like SD, MMC, µSD and USB data sticks. Unison offers support for the standard Windows file system with short file names. Optional user recompilation will provide long file name support with full compatibility.

There are two versions of the fat file system. The first is based on 16 bit words and the second is based on 32 bit words. These two formats are similar but not compatible. Recompilation with conditional compilation allows users to switch from fat16 to fat32 and back.

Similar to any other file system, fatfs requires the following steps to make the file system useful to applications in the Linux/POSIX environment of Unison.

- Create file server.
- Register file server.
- Mkfs makes a file system on the media.
- Mount mounts the file system to make it accessible.

The net result is the availability of the file server at the mount point. Now files can be created and manipulated on the media.

The fatfs file server has several similarities and differences from the fsys file server. The similarities are:

- Identical initialization records are used, however, fatfs does not use the number of clones (always 1) or the number of buffers.
- Fsys and fatfs can run identical application code.
- Fsys and fatfs both offer POSIX error codes and compliance.

However, fatfs has some differences as well.

- Fsys is less POSIX compliant in its error codes than fatfs.
- Fsys starts with _file_server and fatfs starts with _fatfs_server as a root function.
- Fatfs has no disk cache.
- Fsys supports extents for contiguous file allocation while fatfs has no such feature.
- Fsys is optimized for disk based file systems while fatfs is best used with removable media.

The tutorial for Fsys applies equally to Fatfs as long as the fatfs server is initialized to replace Fsys. You can find the demonstration for the Fatfs in the demo directory at $RRI_DIR/Unison/demo.

Appendix A
Chip (and Board) Support Packages (CSPs)

Description

This section provides details about Unison Chip Support Packages and Servers. Unison is a flexible, versatile, multicore real-time kernel which, together with its servers and interprocessor communications mechanisms provides a complete run-time environment for embedded applications which adheres to the POSIX standards. Due to its POSIX roots it has complete run time compatibility with:

- Linux
- Windows
- Solaris
- Other pure POSIX RTOS solutions

Unlike these other solutions though, Unison offers an ultra tiny footprint allowing it to run on many System on Chip (SoC) processors that these other operating systems are unsuitable for.

Unison provides access to the most standard set of board services through "servers". These servers take general requests (such as writing data to a serial line or a file) and do the low-level transactions needed to implement the request.

Each chip supported by Unison implements the low-level functionality needed by Unison or the server differently. To handle this multitude of interactions, Unison and its servers have defined a standard set of low-level calls which implement the required functionality but which are implemented on each chip by accessing the low level routines.

Philosophy of Unison Board Support Package

Unison's philosophy of a Chip Support Package is to provide a general purpose interface to the hardware on the board for common hardware and functionality and to provide the tools and hooks to access those likely to be accessed directly by applications. High level abstractions and interactions tend to be general purpose and must be prepared to handle all situations and states which could occur. The extra overhead required often results in servers which are not optimal for the application. The approach of providing the tools and hooks needed by embedded applications for complete access to the low-level functionality of the board (or boards) gives applications the streamlined access where throughput and response time are critical.

This philosophy has led RoweBots to not provide direct support or usage of some of the higher-performance capabilities of a chip, such as processor-processor communications ports, or general purpose interfaces. If Unison and the IPC mechanisms used these interfaces for kernel to kernel communications it could create significant difficulties for an application which also uses that interface.

Features in a Standard Unison Chip Support Package

A Unison Chip Support Package consists of the following components:

- For this processor, linker and compiler-specific options
- Integrated development environment integration (IDE)
- Implementation of low-level routines to support a monotonic clock, time-of-day clock, and debug output as required or available on the board
- Implementation of routines to support multicore, such as mailbox, IPC interrupts, and Direct Memory Access (DMA) devices (to the extent needed to support interprocessor communications)
- Chip setup for initial Memory Management Unit (MMU) state, instruction cache, and bus (e.g. VME) setup
- Saving and restoring extra state for embedded processors or coprocessors such as floating point units (FPU) or tandem processors
- Hardware-specific routines for servers (tty, TCP/IP) to use device-specific addressing and protocols
- Initial (setup) message for each server, to define specific information such as interrupt vector and level, location of data and control ports, baud rate, or network addresses

For further detail, see "Chip Support Assumptions", page 122, in this Appendix.

Items Not in a Standard Unison CSP

The following items are not considered part of a standard Chip Support Package. Consult the specific CSP section for details and applicability.

Memory Management Unit (MMU)

On most Chips the MMU is not enabled or used by Unison. This is largely a performance issue as MMU overhead on many processors is significant and affects every memory access. A few processors or chips require the MMU to be enabled. For these chips the MMU page tables reflect actual memory except for processor-dependent requirements.

Data Cache

Unison is primarily a heterogeneous multicore kernel. Enabling data cache in a multicore environment requires support on every processor in the system for cache invalidation and cache snooping. Since some of the processors and chips supported by RoweBots do not completely support data caching in a multiprocessing or multicore environment, the only choice is not to use cache in general. For some chips Unison provides the facility to enable cache, but users are always cautioned to be very aware of data inconsistency issues.

Floating Point Unit

On processors and chips where the FPU may be optional the FPU is not configured by default. This is to avoid the proliferation of libraries and directories that must be supported in the Unison standard release. Contact RoweBots if you need such support.

Auxiliary or Coprocessors

On systems where auxiliary processors are supplied, but these processors are not independent enough to run a separate Unison kernel, Unison makes no attempt to use the auxiliary processor. Generally, DSC processors with attached DSP units are an exception.

Secondary Buses

Alternate buses connecting some or all processors in a system sometimes are not supported. Unison assumes that on systems where additional buses are connected, the application will be using them to transfer data for improved performance. Unison and the multiprocessing support therefore do not set up or use the PCIe for example. Should you wish to use this or other buses for kernel interfacing, RoweBots can make the necessary modifications for an appropriate remuneration or you can do it yourself.

High Speed Serial Links

These links are usually used by applications to move data from processor to processor or chip to chip at high speeds. Unison uses other interprocessor communication mechanisms to ensure that the full capability is available to the application. Should one wish to use these links for kernel interfacing, RoweBots can make the necessary modifications for an appropriate remuneration or you can do it internally.

Host Backplane Drivers

Some systems, available as part of a host system, are accessed from the host using the PCIe backplane. Depending on the system, RoweBots does provide a host driver along with a target driver to allow users to communicate using TCP/IP protocols over the backplane. This is not supported for MCUs but may be supported for MPUs and FPGAs. Consult the factory if it is not included in your demonstration programs.

Complex and Non-Standard Servers

Many chips and systems provide additional functionality such as parallel ports, SCSI, video interfaces, and alternate buses. Unison does not support such devices as part of the standard release. On chips, such as the PIC families where common devices exist, we provide servers for the entire collection of peripherals which interfaces to these devices. Servers for additional hardware can be developed by RoweBots for the customer for an appropriate remuneration, ported from other platforms or developed by others.

Chip Support Assumptions

Unison makes certain assumptions about a supported chip or a board for which a customer needs a CSP. These assumptions and constraints are related to the ability of a chip, or combination of chips, to work together in a multiprocessing and/or multicore environment. Please refer to the specific processor in Appendix C in this document.

RAM Memory

A Unison application, complete with all servers and a very large amount of application code and data should have 2 MB of memory on each processor. This will generally be split between flash and RAM in an 8:1 to 16:1 ratio. A minimal practical system can be built with 7K Flash and 1000 bytes of memory or less depending on the processor.

Clock or Multicore Interprocessor Communication Support

Unison needs to communicate with Unison kernels to implement a multiprocessing/multicore solution. This communication can use processor or mailbox interrupts, or polling. Polling requires a clock on each processor which is not using an IPC interrupt.

Distributed Interprocessor Communication Support

If a completely distributed system is required, Unison generally assumes that the communication for the distributed IPC will be done with an error checking and recovery protocol which guarantees end to end transport. It requires specification of client and server nodes for initialization and substantially more space in memory.

No Architectural Issues

Some processors or boards or board combinations have specific architectural issues which make multiprocessing impossible, difficult, or inefficient. Examples of such issues are:

- Some processors provide no dual-ported memory, or only provide a limited window to the bus. A DMA only transfer might be supported. Such processors may be supportable by Unison, but usually with additional overhead and cost.
- Some processors have non-contiguous memory. This may require custom linker modifications, FLASH, downloader or IPC modifications.
- Some chips require byte-only access to specific registers or memory while others support or generate word or longword access only. Such chip pairs may not function at all together, or may function with reduced capability. Similar situations exist for word-vs-longword, longword-vs-word, byte-vs-word, word-vs-byte and longword-vs-byte pairings.

- Bus timings and support for burst transfers can be issues when different processors are combined. Speed and throughput increases have forced board manufacturers to be less tolerant of deviations from the bus specification. Bus timeouts and missed interrupts can sometimes be traced to incompatible bus issues.

Chips Currently Supported

Microchip (without restriction)

- PIC 24 Family (PIC 24H consult factory)
- PIC 30 Family (PIC 30F consult factory)
- PIC 33 Family
- PIC 32 Family
- **Boards**
  - Explorer 16 with 24/30/33/32 families
  - PICtail Plus Ethernet
  - PICtail Plus ADPCM
  - PICtail Plus Graphics
  - PICtail Plus SD
  - PICtail Plus USB
  - pic32 starter kit
  - pic32 usb starter kit
  - pic32 network starter kit, PICtail 802.15.4

Renesas

- SH2A
- M16C Family
- R32C
- Rx Family
- **Boards**
  - H8S & H8SX Graphics
  - H8SX RSK
  - M16C RSK
  - R8C Subatomic Evaluation
  - R32C RSK
  - SH2 RSK
  - Renesas I/O Board
  - SH2A RSK 7286
  - SH2A RSK 7203
  - SH2A RSK 7216
  - Rx RSK

Xilinx

- MicroBlaze

- **Boards**
  - sp605

ARM

Cortex-A8

Chip Vendors

- **TI MPU**
  - Sitara Family
  - DaVinci Family

- **Boards**
  - LogicPD baseboard with SOM

Cortex-M3

Chip Vendors

- **Microsemi**
  - SmartFusion
  - SmartFusion2

- **Boards**
  - SmartFusionTM Evaluation Kit
  - SmartFusion2 Development Kit

- **TI MCU**
  - LM3S6965
  - LM3S3748
  - LM3S9792 (supported with little or no reconfiguration)
  - LM3S9B90
  - LM3S9B92
  - LM3S8962
  - LM3S9B96
  - LM3S9D96

- **Boards**
  - LogicPD Zoom with OMAP 3530
  - EKC-LM3S6965
  - EKC-LM3S3748
  - EKC-LM3S9B92
  - EKC-LM3S9B96
  - EKC-LM3S9D96

ARM

***Chip Vendors* (*continued*)**

- **ST**
  - STM32F10x_*
  - STM32F2xx
  - STM32L1xx_md
  - STM32F4xx

- **Boards**
  - STM3210E-Eval
  - STM3210C-Eval
  - STM322X Mangusta
  - STML152
  - STM32L152
  - STM322xG
  - STM324xG

- **NXP**
  - LPC1768
  - LPC1788

- **Boards**
  - NXP or Kiel MCB1760 Evaluation Board (populated with 1768)
  - IAR 1788 Kickstart

- **Fujitsu**
  - MB9XXX

- **Boards**
  - MB9XXX

Appendix B
Microchip Processors and Solutions

Microchip Overview

Microchip offers a complete line of 16 and 32 bit processors with standardized peripherals across the line of processors. This approach allows users to migrate up in performance very easily or reuse existing device drivers on other processors without modification or with minor modification.

The PIC24 offers 16 bit performance with a complete set of peripherals. The dsPIC 30 and 33 offer compatibility with the PIC24 with additional DSP capabilities. The PIC 32 offers a 32 bit MIPS 4K core with 80MHz operation. Almost 100% portability exists in the 16 bit line while minor changes are required for the 32 bit line.

PIC 24/30/33 Families IDE

The MPLABX environment using the Microchip C compiler is used for these families of parts. All projects for the system are in MPLABX. Please contact us for other special development environments.

Architectural Issues

Interrupt Handling

All interupt handling is setup to be done in C language. For this reason, special calls are used to setup vectored interrupt handling. However, interrupt handling is not done using i_set_interrupt() the standard call for this procedure due to limitations in the IDE, compiler and processors. The function has been replaced by a compiler macro.

```
#define NANOEXEC_ISR()

__attribute__((interupt(preprologue("\tinc.w\t__isrlevel\n
\tcall\t_i_prologue"))))
```

This definition takes the macro and appends the function that will be called to handle the interrupt. This handler often has a secondary handler to setup an initial message (particularly for standard Unison servers).

On these families of processors, the functions which the macro applies to that handle the vectored interrupts are predefined. You can look up these functions in the various architectural and programming guides. The following example highlights the use of the macro for timer 1.

```
void __NANOEXEC_ISR() _T1Interrupt();
```

Interrupt Initialization

Additional steps must be taken to setup the device so interrupts are properly handled by the function. This would include:

- setting up the device priority level using registers IPCx and IPL<2:0>
- clearing the interrupt flag for the device using IFSx.

Interrupt enable and disable functions are provided by Microchip for these families of parts. This means that the generic routines used by Unison are not used.

Memory Map

One memory mapping issue for the PIC 24/30 and 33 families is the issue of access to constants which are loaded into FLASH. To enable access, CORCON |= 4 must be set for each thread. This is done automatically in the software.

Additional memory map control may be done using malloc_init to ensure that the maximum stack space is available for the user application.

For the dsPIC architecture things get more difficult. The multiply accumulate requirements of the architecture make the need to feed the arithmetic unit with both X and Y buffers for proper operation. The allocation and management of these buffers is often best accomplished using a linker script. By taking this approach, the user can preallocate some buffers and create partitions for other buffers to meet the application need for buffer placement while still keeping all remaining memory for user applications. A demo called adpcm_dsp illustrates this type of buffer management and script.

MMU

It is not present on this processor.

Multicore Considerations

These are not present on this processor.

Context Switching

When context is switched, all context is preserved for all threads on all families. This can be relaxed in the case of the dsPIC to eliminate the ALU registers if they are only used by a single thread.

Note that on context *switch, none of the peripheral registers are saved and restored.*

- W0-W15 (includes stackpointer)
- SPLIM
- PC
- CORCON
- On dsPIC – All ALU registers (with pipe flushed and restored)

PIC 32 Families IDE

The MPLABX environment using the Microchip C compiler is used for these families of parts. All projects for the system are in MPLABX.

Please contact us for other special development environments.

Architectural Issues

Interrupt Handling

All interrupt handling is setup to be done in C language. For this reason, special calls are used to setup vectored interrupt handling. Interrupt handling is done using i_set_interrupt() the standard call for this procedure. Consult the man pages on this function to see the correct operation of this function to setup interrupt vectors.

On these families of processors, the functions which the macro applies to that handle the vectored interrupts are user defined. You can look up these functions in the various servers or setup files. The following example highlights the use of the macro for the tty server.

```
i_set_interrupt(_UART1_VECTOR, Uart1_Body_Interrupt, 0);
```

Interrupt Initialization

Additional steps must be taken to setup the device so interrupts are properly handled by the function. This would include:

- setting up the device priority level
- setting up device control registers
- clearing the interrupt flag for the device.

Interrupt enable and disable functions are provided by Microchip for these families of parts. This means that the generic routines used by Unison are not used.

Memory Map

One memory mapping issue for the PIC 32 family is the issue of access to constants which are loaded into FLASH. This is done automatically in the software.
Additional memory map control may be done using malloc_init to ensure that the maximum stack space is available for the user application. A linker script can be added if tighter control over the memory on the chip is required. In general this should not be required.

MMU

It is not present on this processor.

Multicore Considerations

These are not present on this processor.

Context Switching

***Note that on context** switch, none of the peripheral registers are saved and restored.*

When context is switched, all context is preserved for all threads on all families.

- R0-31 (using GPR shadow register set)
- PC
- CP0 register subset
- MDU
- ALU
- Adder
- Branch unit
- Mux and Bit Detect

Operating Modes

All operations are done in kernel mode. User and debug modes are not used.

Cache Management

Program memory and data cache is enabled. The unicore operation does not required control of these registers.

Appendix C
Chip and Board Support for Microchip

Supported boards and components

Boards

- PIC32 Starter Kits
- PIC32 Ethernet Starter Kit
- Explorer 16 with 24/30/33/32 families
- dsPICDEM MCLV
- PICtail Plus Ethernet
- Speech Playback PICtail Plus
- PICtail Plus Graphics
- Pictail Plus SD/MMC
- PICtail Plus USB
- Buck/Boost Converter PICtail Plus

Tool Support

The Explorer 16 offers great prototyping capabilities. The serial port is connected on the tty server channel 1. All configuration bits are set up in all demos. You have to select the correct linker script and processor in the demo, build and run. Other details are in the demo readme files. All other boards are supported connected to the Explorer 16. The demos are all in the demo directory and are preconfigured.

Note that there is separate projects under each demo for Explorer 16 and PIC32 starter kits.

Appendix D
Device Specific Servers

Overview

This section provides an overview of all the various servers that are available. This set of servers will not necessarily run on your specific configuration because both peripheral support and adequate space is often required. Check specific chip support packages and configurations for these details.

Serial I/O Server

The Serial I/O Server offers up to 115K bits per second operation with a variety of line disciplines. All standard flow control methods are supported along with buffer flushing and servicing limits for high performance operation.

TCP and UDP Server

The TCP and UDP Server is both flexible and feature rich. Conditional compilation is used to tailor the size to different configurations including UDP only, multiport drivers and buffer sizes. The server supports the following features:

- fixed or DHCP server derived IP address
- optional DHCP server support
- UDP only option
- configurable buffer sizes for optimal size performance tradeoffs
- multiple open connections
- static and dynamic routing tables
- ICMP and ARP support
- multiple network interfaces

The server is both fast and very easy to configure. See the numerous examples for details on feature use.

Multimedia File Server

The file server offers contiguous files with a buffer cache supporting high performance multimedia streaming file storage and recovery. It can be configured for a variety of media.

- RAM
- USB flash drive
- SD or µSD
- hard disk

The setup for specific devices requires different chip support drivers. Check the specific examples and demonstrations for available drivers for your configuration.

FAT File System

The Fatfs file server can be configured to operate with a variety of media. Its primary strength is its full compatibility with FAT16 or FAT32 file systems which are broadly used on other devices.

- RAM
- USB flash drive
- SD or µSD
- hard disk

Flash File System

The Flashfs file server can be configured to operate with a variety of media. Its primary strength is its ability to support both NAND and NOR flash devices with rapid writing and reading of files.

- SD or µSD
- NOR flash
- NAND flash

Please contact the factory for details – this file server is not yet fully released at the time of writing this manual.

NFS File Server with RPC and XDR

NFS, RPC and XDR are available only on devices with large memory footprints and options. They support host file access on Linux, Unix and Solaris systems using the TCP server.

USB Embedded Host Server

USB Embedded Host support includes USB host support along with options for a variety of device classes. These device classes include: mass storage, communications device classes, human interface device classes and more. Please consult the man pages for specific server options and information.

USB Device Server

USB Embedded Device support includes USB device support along with a single device class option. The options are: mass storage devices, bulk data transfer, communications devices, human interface devices and more. Please consult the man pages for specific device options and information.

Posh Server

The posh server provides a POSIX Shell with commands to manipulate files in the file system. This includes all the standard commands Linux might have but with limited options and a tiny footprint with login support in addition.

Telnet Server

The telnetd server provides telnet services with minimal formatting and a minimal footprint. It uses posh to provide file manipulation. For size reasons, telnetd may not be available on all configurations.

TFTP Server

The tftpd server provides trivial file transfer protocol support to import and export files from Unison using TCP networking. Files are transferred to a fixed mount point plus a path. Telnet can be used to move files within the Unison file system.

HTTP Server

The thttpd server offers an HTTP 1.0/1.1 server with a tiny footprint. It is suitable for user interfaces and file downloads using HTTP. Get, Post and Head commands are supported. Consult the factory for delivery information.

Bluetooth AT Command Server

Bluetooth may be implemented with a complete module which supports Bluetooth including a profile or profiles. In this case, common AT commands are used to initialize the device and start traffic flowing. A variety of modules are supported. Please consult the man pages for specific device options and information.

Bluetooth Multi-Profile Server

The Bluetooth Multi-Profile Server provides a variety of different profiles for Bluetooth. If you require a special profile, this profile will be provided for the cost of the existing profiles. The interface is done at the HCI level so the chosen module only requires HCI support. This can be setup via AT commands or SPI interfaces.

WIFI Server / Driver

WIFI support is provided on a case by case basis using either a driver for WIFI attached to the TCP/IP server or using a separate server interfaced to the TCP server. A variety of modules are supported. Please consult the man pages for specific device options and information.

Remedy GraphXGen Graphics Services

Using standard vendor libraries, Remedy GraphXGen provides the capability to automatically generate color touch panel graphics displays on the target. For further details, refer to the Remedy GraphXGen manual.

Graphics Library Services

Standard vendor graphics libraries are supported under Unison as a service. These libraries may be used standalone or with GraphXGen.

6loWPAN

The 6loWPAN emerging standard is supported with IPv6. It includes a size optimized v6 stack along with ROLL and RPL support.

GSM/GPRS/UHF

Access to cell networks including voice call potential is provided via the serial drivers, AT command pass through, line multiplexing and PPP driver for serial access to remote TCP servers. User customization is generally required.

DNS Client

DNS client services are provided as a standard component in Unison 5.2.

Datalogger

The datalogger is used to log data from the various target components. It has dynamic collection capabilities and is fully integrated as the data collection component of the Remedy RED analysis tool.

SNMP v1, v2c, and v3

The full set of functionality to support SNMP MIB for getting and setting variables along with complete trap support is provided in the SNMP server. The v3 support adds MD5 security.

Please consult the factory for other device and protocol support. A variety of solutions are continually being developed or can be added for specific applications.

Appendix E
RemedyTools and Applications

Overview

The tools and applications to support various use product requirements are provided in this section. It really covers all components which are not solely running on the target environment or are not a server or service within the operating system.

Remedy Object Viewer

The Remedy Object Viewer (OV) allows users to examine the detailed state of components which the operating system has created. These objects are:

- threads and their allocated resources
- named semaphores
- semaphores
- message queues
- mutexes
- memory
- rendezvous state and queues
- timers
- barriers
- servers
- environment variables
- other registered data
- system information

As shown below under Remedy RED, the two figures illustrate the architecture of the Remedy OV. The hardware diagram shows a debug solution providing breakpoint capabilities. The Object Viewer information is updated each time a breakpoint is hit. The software architecture figure illustrates where Remedy OV runs.

Please consult the detailed installation for the OV for your SoC for installation.

Remedy Flash Bootloader

The flash downloader/bootloader offers the ability to update files in the target system either under target control or under host control. This set of utilities requires customization for your specific communication scheme or boot device. Please contact the factory for details.

Remedy POST

The Remedy power on self test (POST) provides startup checking of the registers, pointers, alu functions, memory data and addressing features, interrupt functions and timer capabilities. It is available as a separate module.

Remedy Diagnostics

Diagnostics are available as a separate application for the MCU and all peripheral functions. Please contact the factory for this custom application.

Remedy RED

The Remedy Remote control, Event & Data Viewer (RED) is a tool that provides detailed dynamic time analysis on the target system. It utilizes a separate channel to the target as shown in *figure f55*, which must be a high performance connectivity link. The time based data is displayed by the event and data tool on the host while the dynamic display is controlled by the Remote Control component. In *figure f56* the software architecture, RED is shown as the Event component while Remedy OV is shown as the OS Viewer component.

Remedy GraphXGen

Using standard vendor libraries, Remedy GraphXGen provides the capability to automatically generate color touch panel graphics displays on the target. For further details, refer to the Remedy GraphXGen manual.

Figure f17
Remedy RED Hardware Architecture

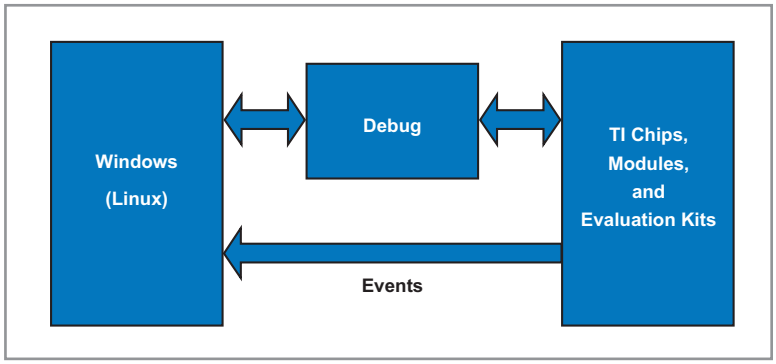
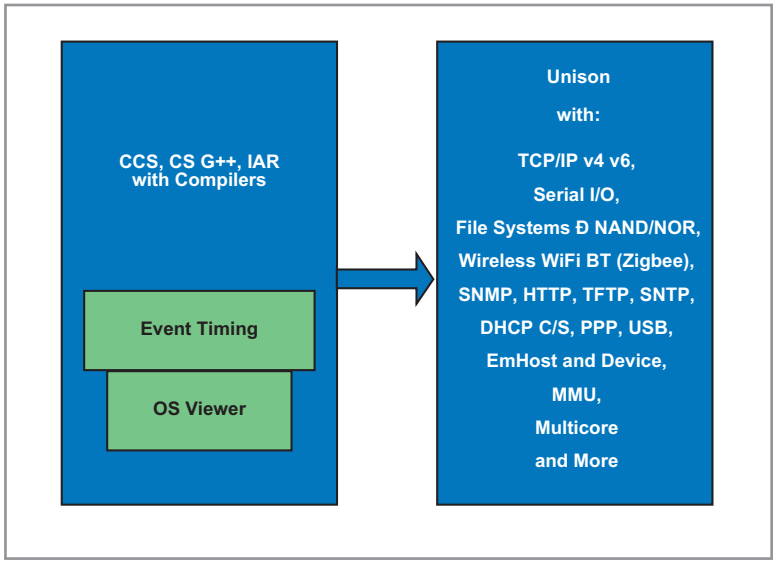


Figure f18
Remedy RED Software Architecture



Appendix F
Error Messages

Introduction

The following is a list of error messages that may be produced by the Unison/Nanoexec operating system during application execution.

Unison Error Codes

The following error codes are obtained by looking at the value of errno. Note that the initial thread error code is EOK and it may be reset to this value by setting the variable. These errors are found in "$RRI_DIR/include/errno.h".

Table t15
Unison Error Codes

Value	Name	Description
0	**EOK**	No error
1	**EPERM**	Not the owner of the file.
2	**ENOENT**	No entry in the tables
3	**ESRCH**	Search for thread failed. Or ?????
4	**EINTR**	Interrupted function call; an asynchronous signal occurred and prevented completion of the call. When this happens, you should try the call again.
5	**EIO**	A physical I/O error occurred.
6	**ENXIO**	No such device or address.
7	**E2BIG**	Argument list too long; used when the arguments passed to a new program being executed with one of the exec functions occupy too much memory space. (See Executing a File.)
8	**ENOEXEC**	Invalid executable file format.
9	**EBADF**	Bad file descriptor; for example, I/O on a descriptor that has been closed or reading from a descriptor open only for writing (or vice versa).
10	**ECHILD**	There are no child processes. This error happens on operations that are supposed to manipulate child processes, when there aren't any processes to manipulate.
11	**EAGAIN**	Resource temporarily unavailable – try later.
12	**ENOMEM**	No memory for this operation

Table t15 (continued)
Unison Error Codes

Valu	Name	Description
13	**EACCES**	No Access Permission
14	**EFAULT**	A major fault has occurred with the service.
15	**ENOTSUPPORTED**	No support for this feature
16	**EBUSY**	The device or service is busy and cannot service the request.
17	**EEXIST**	Referenced object does not exist
18	**EXDEV**	You can't link files that are on physically separate devices because each device has to be mounted or the link will not work.
19	**ENODEV**	No device exists to support this call
20	**ENOTDIR**	Not a directory
21	**EISDIR**	Is a directory
22	**EINVAL**	Invalid operation – assembly level
23	**ENFILE**	The file does not exist
24	**EMFILE**	The current process has too many files open and can't open any more. Duplicate descriptors do count toward this limit.
25	**ENOTTY**	The tty server was not found.
27	**EFBIG**	A file access was made which exceeded the maximum file size.
28	**ENOSPC**	No space exists for the operation
29	**ESPIPE**	Illegal seek. An lseek was issued to a socket or pipe. This error may also be issued for other non-seekable devices.
30	**EROFS**	The file system is read only but a write was requested.
31	**EMLINK**	Too many links have been created.
32	**EPIPE**	Illegal seek. An lseek was issued to a socket or pipe. This error may also be issued for other non-seekable devices.
33	**EDOM**	A domain or input error has occurred likely for a mathematical function.
34	**ERANGE**	A range error has occurred typically for a mathematical function.

Table t15 (continued)
Unison Error Codes

Value	Name	Description
45	EDEADLK	Deadlock condition cannot proceed
46	ENOLCK	The function call was not able to get the lock required to complete the call and failed as a result.
47	ECANCELLED	Operation cancelled.
48	ENOTSUP	A thread has requested an operation that is not supported by the particular server.
50	EBADMSG	The message format was incorrect causing an error.
51	ENAMETOOLONG	The name used has too many characters
52	ENOSYS	This feature is not supported in the system
53	ENOTEMPTY	An attempt has been made to delete an object which is not empty
54	EMSGSIZE	The message was too large
55	EINPROGRESS	The operation is in progress and cannot be requested again.
56	ETIMEDOUT	A timeout occurred
60	EPROTONOSUPPORT	Does not support this protocol option. The protocol has not been configured into the system or no implementation for it exists.
61	EPROTOTYPE	Protocol wrong type for socket. A protocol was specified which does not support the semantics of the socket type requested. For example, you cannot use the ARPA Internet UDP protocol with type SOCK_STREAM.
62	ENOBUFS	No buffer space available. An operation on a socket or pipe was not performed because the system lacked sufficient buffer space.
63	EOPNTOSUPP	Operation not supported on socket. For example, trying to accept a connection on a datagram socket.

Table t15 (continued)
Unison Error Codes

Value	Name	Description
64	EISCONN	Socket is already connected. A connect request was made on an already connected socket or a sendto request on a connected socket specified a destination other than the connected party.
65	ENOTCONN	Socket is not connected. A request to send or receive data was disallowed because the socket is not connected.
66	EALREADY	Operation already in progresss. An operation was attempted on a non-blocking object which already had an operation in progress.
67	EALREADYIN USE	Object already being used by another operation.
68	EDESTADDRREQ	Destination address is missing. A required address was omitted from an operation on a socket.
69	EWOULDBLOCK	Non-blocking I/O: request aborted
70	ENOPROTOOPT	Option not supported by protocol. A bad option was specified in a getsockopt() or setsockopt() call.
71	EAFNOSUPPORT	Address family not supported by protocol family. An address in-compatible with the requested protocol was used. For example, you shouldn't necessarily expect to be able to use PUP Internet addresses with ARPA Internet protocols.
72	ENETUNREACH	Cannot reach the network. A socket operation was attempted to an unreachable network.
73	EADDRNOTAVAIL	Can't assign requested address. Usually caused by an attempt to create a socket with an address not on this machine.

Table t15 (continued)
Unison Error Codes

Value	Name	Description
74	EHOSTDOWN	Host is down. A socket operation failed because the destination host was down.
75	EHOSTUNREACH	Host is unreachable. A socket operation was attempted to an unreachable host.
76	ECONNREFUSED	Connection refused. No connection could be made because the target machine actively refused it. This usually results from trying to connect to a service which is inactive on the foreign host.
77	ENETDOWN	Network is down. A socket operation encountered a dead network.
78	ECONNRESET	Connection reset by peer. A connection was forcibly closed by a peer. Usually caused by the peer executing a shutdown call.
79	ECONNABORTED	Software caused connection abort. A connection abort was caused internal to your host machine.
80	EINPROGRESS	Operation now in progress. An operation which takes a long time to complete (such as a connect) was attempted on a non-blocking object (see ioctl).
81	ETIMEDOUT	TCP connection timeout out
82	EAGAIN	Resource is temporarily unavailable.
83	E_DOMAIN	Floating point input or parameter error.
84	E_RANGE	Floating point output range error.
100	EBADINDEX	Bad Unison table index
101	EBADID	Bad Unison object id
102	EBADPORT	Bad Unison port number
104	ETHREADDIED	The thread died as an operation was in progress and could not be completed.
105	EBADSERVERNAME	Server name was bad.
106	ENOSERVERFOUND	Server name was not found.

Table t15 (continued)
Unison Error Codes

Value	Name	Description
107	**EBADUCB**	Bad file descriptor
110	**EBADFSYS**	The file system is corrupted.
111	**ENOEXTENTS**	The current file has no more extents remaining. Copy the file to a new file to overcome this problem.
112	**EDQUOT**	Disk quota exceeded.
113	**ESTALE**	A file or ucb handle was stale and could not be used.
114	**ENFSERR**	NFS error – The network file system experienced an error.
115	**ERPCFAILED**	The RPC call failed
116	**ECANTSPAWNPROCESS**	It was not possible to spawn a thread to deal with this request.*
119	**ENAMLEN**	The name length is too long for this call.*
120	**EBADBLOCK**	Bad block from low level routines.*

* Not used in all implementations

Table t16
Unison Fatal Error Codes

Value	Name	Fatal Error
0x40000001	**FAT_HEAP**	Heap error
0x40000002	**FAT_PRIORITY**	Incorrect thread priority
0x40000003	**FAT_NPORTS**	Ports incorrectly setup for a thread or couldn't be created.
0x40000004	**FAT_DUPINDEX**	Duplicate object indices.
0x40000005	**FAT_READYQS**	ReadyQ error
0x40000010	**FAT_IDLE**	No Idle thread creation possible.
0x40000011	**FAT_ROOT**	Root configuration not valid or couldn't be started.
0x40000012	**FAT_DIRECTORY**	Directory error.
0x40000013	**FAT_NTHREAD**	Number of threads
0x40000100	**FAT_BRIDGE**	Shared backplane bridge error
0x40000101	**FAT_MULTIPROC**	Multiprocessor or multicore fatal error.
0x40000200	**FAT_KERNEL**	Internal kernel error with inconsistent state.

Known Limitations

Version 4.1

1. *mq_notify* is not implemented in this architecture at the time of rendering this manual. If you need this function, check the support site for updates and contact the factory if it is not available.
2. *mr_send* does not support sender timeouts. This feature adds extra size and complexity not suitable for ultra tiny kernels at this time.

Appendix G
libc

The ***libc*** library, which is provided with the compiler, provides functions that support basic C programming within the Unison environment. It contains all of the functions required by ANSI as well as some extensions provided for compatibility with BSD POSIX. It is sometimes extended by RoweBots in the event that these basic functions are not available from the manufacturer of the development environment.

Math Functions

The ***libc*** math functions will use hardware floating point on processors that have a built in floating point coprocessor (FPU). Processors that may have an optional FPU must explicitly inform the linker that an FPU is present.

Error handling by the ***libc*** math functions is typically performed via the standard "*matherr*" function.

The details of the ***libc*** provided are included with the compiler documentation. You should refer to this documentation for the specifics of the calls which are handled and the details of error handling. The details of ***libc*** are provided in this appendix only in the case where the compiler support has been extended or implemented by RoweBots.

Appendix H
Glossary

API Application Programming Interface

BSS Base Stack Segment, which contains uninitialized variables for a single linked image of an executable program

CAN Control Area Network, a network often used in automobiles and industrial applications

CS Code Sourcery, a company that develops the Sourcery G++ IDE

CSP Chip Support Package is the software that customizes a release for a specific System On a Chip (SOC).

DMA Direct Memory Access is the direct transfer of data from memory to memory or from memory to device without processor intervention.

DSC A Digital Signal Controller is a small microcontroller that is capable of doing fast multiply-accumulates as well as providing good control.

DSP Digital Signal Processor or Digital Signal Processing

dsPIC A DSC made by Microchip

Eclipse An IDE that is open source and being widely adopted for embedded systems

endianess The ordering of individually addressable sub-units (words, bytes or even bits) within a longer data word stored in external memory. The most typical cases are the ordering of bytes within a 16-, 32-, or 64-bit word, where endianess is often simply referred to as byte order. The usual contrast is between most versus least significant byte first, called big-endian and little-endian respectively.

EPROM An electrically or light erased read-only memory, often called flash

FIFO First In First Out queue in software or hardware

FLASH Read-only memory that is electrically writable. It comes in several forms.

FPGA Field Programmable Gate Array, which provides a programmable set of gates that is software configurable at run time

FPU Floating Point Unit used to provide faster floating point support

gcc GNU C compiler

gdb GNU debugger

gid Group Id is a group identifier often associated with Linux or POSIX file systems.

HEW An IDE from Renesas supporting the Renesas processors

IAR IDE A company that develops one of the leading IDEs for embedded systems and their IDE

I/O Input Output for a microcontroller in a variety of forms such as Ether

net, serial and parallel

IDE An Integrated Development Environment is a combined software project manager, file manager, compiler, linker, editor and debugger.

IP Internet Protocol or Internet Packet or an internal packet to the TCP/UDP/IP protocol stack used for communication over the Internet and internal networks

ISR Interrupt Service Routine is a small routine that services an interrupt and typically signals a server to process the interrupt

LIT LIT is an automotive specification for serial communication

MCU A microcontroller or a System On a Chip is a complete tiny computer with everything on a single chip, which does dedicated applications

Microchip An MCU or microcontroller and DSC manufacturer of PIC24, dsPIC and PIC32 devices

Microsemi A microcontroller manufacturer of ARM Cortex M3 SmartFusion parts

MMU Memory Management Unit is a piece of hardware used to map addresses in very large microcontrollers.

MMC A data card format for removable media It is compatible with SD and µSD formats.

MPLAB Microchip's IDE for PIC24, dsPIC and PIC32

MPLABX Microchip's IDe for pIC24, dspIC and pIC32

MPU Memory Protection Unit for protecting regions of memory on a microcontroller without an MMU

µSD µSD, MicroSD or MSD, a data storage card format electrically compatible with SD and MMC but in a smaller physical form factor (1cm x 1 cm). These cards come with small plastic adapters for use in standard SD slots.

NAND A type of flash memory or read-only memory that provides very high density but is less reliable and slower than NOR flash.

NFS Network File System is a file system that runs over a local area network using TCP/UDP/IP to provide remote file access.

NOR A type of flash memory or read-only memory that provides high density, reliable writes and is faster than NAND flash.

OO Object Oriented is a style of programming.

PIC A line of microcontrollers made by Microchip including the PIC24, dsPIC and PIC32

Renesas The leading manufacturer of microcontrollers including the H8, M16C, R32C, RX, SH2A and other MCUs

RPC Remote Procedure Call is a means to communicate over a local area network at a higher level than socket communication.

RTOS Real Time Operating System is an embedded operating system dedicated to a specific task that must be done in a timely fashion.

SD A data card format for removable media It is compatible with MMC and µSD (micro SD) formats.

SmartFusion/ SmartFusion2 A line of ARM Cortex M3 microcontrollers made by Microsemi

SOC System On a Chip is a microcontroller that has a complete tiny computer on a chip including processor, memory and all I/O.

Sourcery G++ Code Sourcery's IDE, which utilizes GCC and Eclipse IDE

ST ST Microelectronics, a microcontroller manufacturer of ARM Cortex M3 STM32 parts

STM32 A line of ARM Cortex M3 microcontrollers made by ST Microelectronics

Stellaris ARM Cortex M3 MCUs made by Texas Instruments

TCP Transmission Communication Protocol is the main protocol used to transmit data over Ethernet and the Internet. It may also refer to the server that provides TCP/UDP/IP protocols.

TCP/IP TCP with IP are the basic protocols that support Internet and Ethernet or local area network communication.

TI Texas Instruments, a microcontroller manufacturer of ARM Cortex M3 Stellaris parts

TTY Terminal server, which provides asynchronous character oriented I/O

UART Universal Asynchronous Receiver and Transmitter is the hardware that transmits asynchronous characters over the network.

UCB User Connection Block is a block that is used with an I/O server to buffer user data as it is sent to the server.

UDP Universal Datagram Protocol is the main protocol to send single messages over the Internet.

uid User Id is an individual's identifier often associated with Linux or POSIX file systems.

USB Universal Serial Bus is a common hardware format to connect devices such as a mouse, keyboard, flash drive or other device to a computer.

VME A bus used to connect many processors in parallel

XDR eXternal Data Representation is a protocol used on networks to overcome issues with Big Endian and Little Endian byte ordering. See endianess.